
STensor/IndexNotation

Setup

```
In[*]:= << mGRG`STensor`
```

```
In[*]:= $PreRead =  
  ReplaceAll[#, expr_String => StringReplace[expr, "SP" -> "mGRG`STensor`Private"]] &;
```

```
In[*]:= Off[AutoFlag]
```

Utilities

AllOptions

```
In[*]:= AllOptions::usage
```

```
Out[*]=
```

```
AllOptions[qHead][name, optL] checks if 'name' satisfies all conditions  
in the option list 'optL'. 'qHead' is typically HeadQs or IndexQs.
```

```
In[*]:= HeadQs::usage
```

```
Out[*]=
```

```
An option for specifying heads of  
expressions to which a function should be applied.
```

```
In[*]:= IndexQs::usage
```

```
Out[*]=
```

```
An option for specifying which indices to target in an operation.
```

Returns True if 'name' satisfies all the conditions for 'HeadQs'.

```
In[*]:= AllOptions[HeadQs][name, {HeadQs -> {True &}}]
```

```
Out[*]=
```

```
True
```

Returns True if 'idx' satisfies all the conditions for 'IndexQs'.

```
In[*]:= AllOptions[IndexQs][idx, {IndexQs -> {False &}}]
```

```
Out[*]=
```

```
False
```

ConstantQ

<code>In[*]:=</code>	ConstantQ::usage
<code>Out[*]=</code>	ConstantQ[x] returns True if x is a number or a numeric symbol or a constant symbol, and False otherwise.
<code>In[*]:=</code>	ConstantQ /@ {1, $\frac{2}{3}$, 3.14, 2 + 3 i}
<code>Out[*]=</code>	{True, True, True, True}
<code>In[*]:=</code>	SetAttributes [c, Constant] ConstantQ [c]
<code>Out[*]=</code>	True

FreePatternQ

<code>In[*]:=</code>	FreePatternQ::usage
<code>Out[*]=</code>	FreePatternQ[expr] checks if 'expr' contains pattern objects.
<code>In[*]:=</code>	FreePatternQ /@ {a, a : True, PatternSequence[a, b_, c], _, __, ___}
<code>Out[*]=</code>	{True, False, False, False, False, False}

PositiveIntegerQ

<code>In[*]:=</code>	PositiveIntegerQ::usage
<code>Out[*]=</code>	PositiveIntegerQ[n] yields True if n is a positive integer, and False otherwise.
<code>In[*]:=</code>	PositiveIntegerQ /@ {1, 0, -1, 3.14, some}
<code>Out[*]=</code>	{True, False, False, False, False}

SignOfTerm

In[*]:= **SignOfTerm::usage**

Out[*]=

SignOfTerm[expr] returns -1 if a symbolic
'term' has a leading minus sign (e.g., -x), and 1 otherwise.

In[*]:= **SignOfTerm /@ {-a, b, -1, 1}**

Out[*]=

{-1, 1, 1, 1}

SymbolJoin

In[*]:= **SymbolJoin::usage**

Out[*]=

SymbolJoin[s1, s2, ...] joins symbols or strings into a single symbol.

In[*]:= **SymbolJoin[{a, b, c}]**

Out[*]=

abc

In[*]:= **SymbolJoin[a, b, c]**

Out[*]=

abc

In[*]:= **SymbolJoin[a, "b", c]**

Out[*]=

abc

In[*]:= **% // Head**

Out[*]=

Symbol

Indices

인덱스에 대한 연산을 하는 함수는 옵션으로 인덱스가 만족해야 할 질의함수 리스트인 IndexQs를 가질 수 있다. 이 옵션을 갖는 함수는 그 질의함수들을 모두 만족하는 인덱스에 대해서만 작용한다.

인덱스 분류

인덱스는 (IndexType, Lower/Upper, IndexKind)에 따라 구별된다.

```
In[*]:= dnIndexL = {-1, la, lμ, lA, lba, lda, lha};
upIndexL = {1, ua, uμ, uA, uba, uda, uha};
indexL = Flatten[{dnIndexL, upIndexL}];
```

IndexType

IndexType은 Symbol로 표시되는 정규 인덱스와 더미 인덱스, 0을 제외한 Integer로 표현되는 성분 인덱스로 나뉜다.

RegularIndexQ

```
In[*]:= RegularIndexQ::usage
```

```
Out[*]=
```

RegularIndexQ[index] returns True if 'index' is a regular, non-dummy, symbolic index, typically defined with SetIndices.

인자가 정규 인덱스 인가를 판단하는 질의 함수이다. 인자가 defIndexKind 함수를 사용하여 설정된 인덱스이면 정규 인덱스이다.

```
In[*]:= RegularIndexQ /@ {hello, dhpark, la, lo, ua, uo}
```

```
Out[*]=
```

{False, False, True, True, True, True}

DummyIndexQ

```
In[*]:= DummyIndexQ::usage
```

```
Out[*]=
```

DummyIndexQ[index] returns True if 'index' is a system-generated dummy index.

인자가 더미 인덱스 인가를 판단하는 질의 함수이다. 더미 인덱스는 프로그램의 필요에 따라 자동적으로 생성됐거나 NewDummy 명령으로 생성된 인덱스이다.

```
In[*]:= NewDummy[Latin]
```

```
Out[*]=
```

{lLatin\$10322, uLatin\$10322}

```
In[*]:= DummyIndexQ /@ %
```

```
Out[*]=
```

{True, True}

ComponentIndexQ

In[*]:= **ComponentIndexQ::usage**

Out[*]=

ComponentIndexQ[index] returns True
if 'index' is a component index (a non-zero integer).

인자가 성분 인덱스 인가를 판단하는 질의 함수이다. 성분 인덱스는 0을 제외한 정수로 표현된다. 참고: Abstract Index 표기법에서는 기준 벡터 ∂_μ^a 로 벡터 ξ^a 를 $\xi^a = \xi^\mu \partial_\mu^a$ 로 표현한다. 이 때 a 는 abstract index이고, μ 는 성분 인덱스이다. 그러나 본 패키지에서의 성분 인덱스는 (컴퓨터 연산의 관례에 따라) 0이 아닌 정수를 의미한다. 특히 본 패키지에서의 정규(Regular) 인덱스는 Abstract Index 표기법에서의 abstract index이거나 성분 인덱스일 수 있고, 각각의 구분은 사용자의 몫이다.

In[*]:= **ComponentIndexQ** /@ {-1, -2, 0, 1, 2}

Out[*]=

{True, True, False, True, True}

Lower or Upper

인덱스가 텐서의 아래 첨자(lower, covariant)이면 DnIndex, 위 첨자(upper, contravariant)이면 UpIndex이다.

DnIndexQ

In[*]:= **DnIndexQ::usage**

Out[*]=

DnIndexQ[index] returns True if 'index'
is a valid lower (covariant) index, and False otherwise.

인자가 covariant(또는 lower, down) 인덱스 인가를 판단하는 질의 함수이다. 인덱스의 이름이 "l"(ell)로 시작하거나 (성분 인덱스인 경우) 음의 정수이면 DnIndex이다.

In[*]:= **Select**[indexL, DnIndexQ]

Out[*]=

{-1, 1a}

UpIndexQ

In[*]:= **UpIndexQ::usage**

Out[*]=

UpIndexQ[index] returns True if 'index' is
a valid upper (contravariant) index, and False otherwise.

인자가 contravariant (또는 upper, up) 인덱스 인가를 판단하는 질의 함수이다. 인덱스의 이름이

"u" 로 시작하거나 (성분 인덱스인 경우) 양의 정수이면 UpIndex이다.

```
In[*]:= Select[indexL, UpIndexQ]
Out[*]= {1, ua}
```

IndexKind

IndexKind는 새로이 생성하거나 제거할 수 있다.

Latin

Latin Kind는 기본적으로 이미 정의되어 있고, DefaultKind로 설정되었다.

```
In[*]:= SP`getCharacters[Latin]
Out[*]= {a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z}
```

```
In[*]:= IndexToKind::usage
Out[*]= IndexToKind[idx] returns the kind of 'idx', and NonKind otherwise.
```

```
In[*]:= NonKind::usage
Out[*]= NonKind represents the absence of a valid kind. It is returned by
        IndexToKind when an index is not associated with any defined kind.
```

```
In[*]:= {IndexToKind[la], IndexToKind[ua]}
Out[*]= {Latin, Latin}
```

DefaultKind는 제거할 수 없다. 따라서 Latin Kind를 제거하려면 DefaultKind를 다른 Kind로 먼저 변경해야 한다.

```
In[*]:= SP`undefIndexKind[Latin]
... Msg: Latin is DefaultKind which cannot be removed.
```

```
In[*]:= GetIndices::usage
Out[*]= GetIndices[ikind] returns all defined indices of the specified index kind.
```

In[*]:= **GetIndices[Latin]**

Out[*]=

```
{la, lb, lc, ld, le, lf, lg, lh, li, lj, lk, ll, lm, ln, lo, lp,
  lq, lr, ls, lt, lu, lv, lw, lx, ly, lz, ua, ub, uc, ud, ue, uf, ug, uh,
  ui, uj, uk, ul, um, un, uo, up, uq, ur, us, ut, uu, uv, uw, ux, uy, uz}
```

Latin Kind의 인덱스는 RegularType인 경우 인덱스 이름의 두 번째 문자가 영문 소문자이고, DummyType은 두 번째 문자부터 'Latin\$'로 시작하고 유일한 정수로 끝난다.

In[*]:= **NewDummy[Latin]**

Out[*]=

```
{lLatin$17041, uLatin$17041}
```

In[*]:= **IndexToKind /@ Join[{la, lb, ua, ub}, %]**

Out[*]=

```
{Latin, Latin, Latin, Latin, Latin, Latin}
```

In[*]:= **KindIndexQ::usage**

Out[*]=

```
KindIndexQ[ikind] returns a pure function
  that checks if an index belongs to the specified 'ikind'.
```

In[*]:= **KindIndexQ[Latin]**

Out[*]=

```
IndexToKind[#[1]] === Latin &
```

In[*]:= **KindIndexQ[Latin] /@ NewDummy[Latin]**

Out[*]=

```
{True, True}
```

In[*]:= **KindIndexQ[Latin][#] && DnIndexQ[#] & /@ {la, ub}**

Out[*]=

```
{True, False}
```

특정 IndexKind의 인덱스를 다른 목적으로 사용하기 위해 인덱스들을 변경하려면 SetIndices 명령을 사용한다:

In[*]:= **SetIndices::usage**

Out[*]=

```
SetIndices[{"a", "b", ...}, ikind]
  defines a new set of indices of a specified index kind.
```

In[*]:= **SetIndices[{"a", "b"}, Latin]**

```
In[*]:= GetIndices[Latin]
```

```
Out[*]:= {1a, 1b, ua, ub}
```

원래의 정의로 복귀한다:

```
In[*]:= SetIndices[Alphabet[], Latin]
```

```
In[*]:= GetIndices[Latin]
```

```
Out[*]:= {1a, 1b, 1c, 1d, 1e, 1f, 1g, 1h, 1i, 1j, 1k, 1l, 1m, 1n, 1o, 1p,
1q, 1r, 1s, 1t, 1u, 1v, 1w, 1x, 1y, 1z, ua, ub, uc, ud, ue, uf, ug, uh,
ui, uj, uk, ul, um, un, uo, up, uq, ur, us, ut, uu, uv, uw, ux, uy, uz}
```

Example: Zero

일차원 공간 또는 특별한 목적 (예를 들면 (3+1) decomposition)을 위한 IndexKind를 정의할 수 있다:

```
In[*]:= SP`defIndexKind[Zero, {"0"}];
```

```
In[*]:= SP`getCharacters[Zero]
```

```
Out[*]:= {0}
```

```
In[*]:= {IndexToKind[10], IndexToKind[u0]}
```

```
Out[*]:= {Zero, Zero}
```

1D Kind의 인덱스는 정의에 의해 DummyType이 없다.

```
In[*]:= GetIndices[Zero]
```

```
Out[*]:= {10, u0}
```

```
In[*]:= NewDummy[Zero]
```

```
Out[*]:= {10, u0}
```

```
In[*]:= IndexToKind /@ %
```

```
Out[*]:= {Zero, Zero}
```

```
In[*]:= KindIndexQ[Zero]
```

```
Out[*]= IndexToKind[##] === Zero &
```

```
In[*]:= KindIndexQ[Zero] /@ NewDummy[Zero]
```

```
Out[*]= {True, True}
```

```
In[*]:= KindIndexQ[Zero][#] && DnIndexQ[#] & /@ {10, u0}
```

```
Out[*]= {True, False}
```

```
In[*]:= OneDimKindQ::usage
```

```
Out[*]= OneDimKindQ[ikind] returns True if 'ikind' is a one-dimensional index kind.
```

인덱스로 사용할 수 있는 문자가 한 개인가를 묻는다:

```
In[*]:= OneDimKindQ[Zero]
```

```
Out[*]= True
```

1D Kind의 인덱스를 변경하여 가능한 인덱스가 두 개 이상으로 다시 정의하면 1D Kind가 아니다:

```
In[*]:= AddIndices[{"0", "9"}, Zero]
```

```
In[*]:= GetIndices[Zero]
```

```
Out[*]= {10, 19, u0, u9}
```

```
In[*]:= OneDimKindQ[Zero]
```

```
Out[*]= False
```

```
In[*]:= NewDummy[Zero]
```

```
Out[*]= {1Zero$17126, uZero$17126}
```

Zero Kind의 정의를 취소한다:

```
In[*]:= SP`undefIndexKind[Zero]
```

```
In[*]:= IndexToKind[10]
```

```
Out[*]= NonKind
```

Example: Greek

정의된 IndexKind를 제거하려면 undefIndexKind 함수를 사용한다:

```
In[*]:= SP`undefIndexKind[Greek]
```

```
In[*]:= ? Greek
```

```
Out[*]=
```

Symbol
Global`Greek
Full Name Global`Greek
^

새로운 IndexKind를 정의하려면 defIndexKind 함수에 첫 번째 인자로 Kind 이름, 두 번째 인자로 인덱스로 사용할 문자 리스트를 입력한다.

Greek Kind는 인덱스가 그리스 문자인 경우를 위한 것이다:

```
In[*]:= SP`defIndexKind[Greek, Alphabet["Greek"]];
```

```
In[*]:= SP`getCharacters[Greek]
```

```
Out[*]= {α, β, γ, δ, ε, ζ, η, θ, ι, κ, λ, μ, ν, ξ, ο, π, ρ, σ, τ, υ, φ, χ, ψ, ω}
```

```
In[*]:= {IndexToKind[1μ], IndexToKind[uμ]}
```

```
Out[*]= {Greek, Greek}
```

```
In[*]:= NewDummy[Greek]
```

```
Out[*]= {1Greek$19007, uGreek$19007}
```

```
In[*]:= IndexToKind /@ Join[{1α, 1μ, uα, uμ}, %]
```

```
Out[*]= {Greek, Greek, Greek, Greek, Greek, Greek}
```

```
In[*]:= KindIndexQ[Greek] /@ Join[{1α, 1μ, uα, uμ}, NewDummy[Greek]]
```

```
Out[*]= {True, True, True, True, True, True}
```

이미 다른 Kind에서 사용되는 인덱스를 추가할 수는 없다:

```
In[*]:= AddIndices[{"a", "b"}, Greek]
```

Msg: {a, b} includes character(s) which are already defined.

Greek Kind의 정의를 취소한다:

```
In[*]:= SP`undefIndexKind[Greek]
```

Example: Capital

Capital Kind는 인덱스가 영문 대문자인 경우를 위한 것이다:

```
In[*]:= SP`undefIndexKind[Capital];
SP`defIndexKind[Capital, ToUpperCase /@ Alphabet[]];
```

```
In[*]:= SP`getCharacters[Capital]
```

Out[*]=

```
{A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z}
```

```
In[*]:= {IndexToKind[lA], IndexToKind[uA]}
```

Out[*]=

```
{Capital, Capital}
```

```
In[*]:= NewDummy[Capital]
```

Out[*]=

```
{lCapital$19044, uCapital$19044}
```

```
In[*]:= IndexToKind /@ Join[{lA, lO, uA, uO}, %]
```

Out[*]=

```
{Capital, Capital, Capital, Capital, Capital, Capital}
```

```
In[*]:= KindIndexQ[Capital] /@ Join[{lA, lO, uA, uO}, NewDummy[Capital]]
```

Out[*]=

```
{True, True, True, True, True, True}
```

Capital Kind의 정의를 취소한다:

```
In[*]:= SP`undefIndexKind[Capital]
```

Example: BarLatin

Latin Kind의 인덱스에 Bar 기호가 붙는 경우이다:

```
In[*]:= SP`undefIndexKind[BarLatin]
SP`defIndexKind[BarLatin, "b" <> # & /@ {"a", "b", "c", "d", "e"}];
```

```
In[*]:= SP`getCharacters[BarLatin]
```

```
Out[*]= {ba, bb, bc, bd, be}
```

```
In[*]:= {IndexToKind[lba], IndexToKind[uba]}
```

```
Out[*]= {BarLatin, BarLatin}
```

```
In[*]:= NewDummy[BarLatin]
```

```
Out[*]= {lBarLatin$19070, uBarLatin$19070}
```

```
In[*]:= IndexToKind /@ Join[{lba, lbb, uba, ubb}, %]
```

```
Out[*]= {BarLatin, BarLatin, BarLatin, BarLatin, BarLatin, BarLatin}
```

```
In[*]:= KindIndexQ[BarLatin] /@ Join[{lba, lbb, uba, ubb}, NewDummy[BarLatin]]
```

```
Out[*]= {True, True, True, True, True, True}
```

원래대로 복귀한다:

```
In[*]:= SP`undefIndexKind[BarLatin]
```

Example: DotGreek

Greek Kind의 인덱스에 Dot 기호가 붙는 경우이다:

```
In[*]:= SP`defIndexKind[DotGreek, "d" <> # & /@ {"μ", "ν"}];
```

```
In[*]:= SP`getCharacters[DotGreek]
```

```
Out[*]= {dμ, dν}
```

```
In[*]:= {IndexToKind[lDμ], IndexToKind[uDμ]}
```

```
Out[*]= {DotGreek, DotGreek}
```

```
In[*]:= NewDummy[DotGreek]
```

```
Out[*]= {lDotGreek$19096, uDotGreek$19096}
```

```
In[*]:= IndexToKind /@ Join[{ld $\mu$ , ld $\nu$ , ud $\mu$ , ud $\nu$ }, %]
```

```
Out[*]:= {DotGreek, DotGreek, DotGreek, DotGreek, DotGreek, DotGreek}
```

```
In[*]:= KindIndexQ[DotGreek] /@ Join[{ld $\mu$ , ld $\nu$ , ud $\mu$ , ud $\nu$ }, NewDummy[DotGreek]]
```

```
Out[*]:= {True, True, True, True, True, True}
```

원래대로 복귀한다:

```
In[*]:= SP`undefIndexKind[DotGreek]
```

Example: HatZero

1D Kind의 인덱스에 Hat 기호가 붙는 경우이다:

```
In[*]:= SP`defIndexKind[HatZero, {"h0"}];
```

```
In[*]:= SP`getCharacters[HatZero]
```

```
Out[*]:= {h0}
```

```
In[*]:= {IndexToKind[lh0], IndexToKind[uh0]}
```

```
Out[*]:= {HatZero, HatZero}
```

1D Kind이므로 DummyType은 없다:

```
In[*]:= NewDummy[HatZero]
```

```
Out[*]:= {lh0, uh0}
```

```
In[*]:= IndexToKind /@ %
```

```
Out[*]:= {HatZero, HatZero}
```

```
In[*]:= KindIndexQ[HatZero] /@ {lh0, uh0}
```

```
Out[*]:= {True, True}
```

```
In[*]:= ComponentIndexQ /@ {lh0, uh0}
```

```
Out[*]:= {False, False}
```

원래대로 복귀한다:

```
In[*]:= SP`undefIndexKind[HatZero]
```

IndexToKind

```
In[*]:= IndexToKind::usage
```

```
Out[*]=
```

```
IndexToKind[idx] returns the kind of 'idx', and NonKind otherwise.
```

인자에 해당하는 IndexKind를 돌려 준다.

```
In[*]:= IndexToKind /@ {nonIndex, 0}
```

```
Out[*]=
```

```
{NonKind, NonKind}
```

```
In[*]:= IndexToKind /@ {1a, ubb, 1α, uA}
```

```
Out[*]=
```

```
{Latin, NonKind, NonKind, NonKind}
```

성분 인덱스는 모든 Kind에서 사용할 수 있기 때문에 두 번째 인자로 Kind를 지정해야 한다.

```
In[*]:= IndexToKind /@ {-1, 0, 2}
```

```
Out[*]=
```

```
{Latin, NonKind, Latin}
```

```
In[*]:= IndexToKind[#, Latin] & /@ {-1, 0, 2}
```

```
Out[*]=
```

```
{Latin, NonKind, Latin}
```

Local Functions

checkIndexCharacters

인덱스로 사용할 문자들이 이미 다른 IndexKind에서 사용중인가를 확인한다.

```
In[*]:= SP`checkIndexCharacters [{"a"}, {"μ"}]
```

⋯⋯ Msg: {a, μ} includes character(s) which are already defined.

```
Out[*]=
```

```
False
```

checkName

인자가 다른 목적으로 사용되도록 예약된 이름이거나 또는 이미 사용중인 인덱스의 이름이면 False를 돌려 준다.

```
In[*]:= SP`checkName /@ {la, uμ, C}
```

Msg: la is used as an index name!

Msg: C is reserved!

```
Out[*]=
```

```
{False, True, False}
```

indexKindDummy

returns IndexKind of a Dummy index in String form.

indexClass

indexClass는 IndexToKind와 거의 동일하지만 성분 인덱스에 대해서는 ComponentIndexQ를 돌려준다:

```
In[*]:= SP`indexClass /@ {nonIndex, 0}
```

```
Out[*]=
```

```
{NonKind, NonKind}
```

```
In[*]:= SP`indexClass /@ {-1, 2, la, ub}
```

```
Out[*]=
```

```
{ComponentIndexQ, ComponentIndexQ, Latin, Latin}
```

setIndices

internal engine for setting up index properties.

사용할 인덱스 설정

GetIndices

```
In[*]:= GetIndices::usage
```

```
Out[*]=
```

```
GetIndices[ikind] returns all defined indices of the specified index kind.
```

프로그램에서 사용 가능한 RegularType 인덱스들을 문자열 형태로 보여준다. 인자가 IndexKind이면 그 IndexKind에 해당하는 인덱스들, 인자가 All이면 모든 IndexKind의 인덱스들을 보여준다.

```
In[*]:= GetIndices[Latin]
```

```
Out[*]=
```

```
{la, lb, lc, ld, le, lf, lg, lh, li, lj, lk, ll, lm, ln, lo, lp,
  lq, lr, ls, lt, lu, lv, lw, lx, ly, lz, ua, ub, uc, ud, ue, uf, ug, uh,
  ui, uj, uk, ul, um, un, uo, up, uq, ur, us, ut, uu, uv, uw, ux, uy, uz}
```

```
In[*]:= GetIndices[All]
Out[*]= { {1a, 1b, 1c, 1d, 1e, 1f, 1g, 1h, 1i, 1j, 1k, 1l, 1m, 1n, 1o, 1p,
           1q, 1r, 1s, 1t, 1u, 1v, 1w, 1x, 1y, 1z, ua, ub, uc, ud, ue, uf, ug, uh,
           ui, uj, uk, ul, um, un, uo, up, uq, ur, us, ut, uu, uv, uw, ux, uy, uz} }
```

SetIndices

```
In[*]:= SetIndices::usage
Out[*]= SetIndices[{ "a", "b", ... }, ikind]
         defines a new set of indices of a specified index kind.
```

프로그램에서 사용 가능한 RegularType의 인덱스들을 설정한다. 첫 번째 인자는 인덱스로 사용할 문자 리스트이고, 두 번째 인자는 IndexKind이다.

기본적으로 설정된 문자 리스트를 바꿀 때에도 SetIndices 명령을 사용한다:

```
In[*]:= SetIndices[ToString /@ {a, b, c, d}, Latin]; GetIndices[Latin]
Out[*]= {1a, 1b, 1c, 1d, ua, ub, uc, ud}
```

사용 가능한 인덱스들은 다른 연산에 대해 보호된다:

```
In[*]:= 1d = "NOT POSSIBLE";
Set: Symbol 1d is Protected. ⓘ
```

```
In[*]:= ? 1d
Out[*]=
```

Symbol
Global`1d
UpValue Definitions
DnIndexQ[1d] ^= True
IndexToKind[1d] ^= Latin
RegularIndexQ[1d] ^= True
UpIndexQ[1d] ^= False
Attributes {Protected}
Full Name Global`1d
^

In[]:= **le = "POSSIBLE"**

Out[]:=
POSSIBLE

In[]:= **? le**

Out[]:=

Symbol
Global`le
Assignment
le = POSSIBLE
UpValue Definitions
UpIndexQ[le] ^= False
Full Name Global`le
^

In[]:= **SetIndices[ToString /@ {a, b, c, d, e}, Latin]; GetIndices[Latin]**

Out[]:=
{la, lb, lc, ld, le, ua, ub, uc, ud, ue}

주의: SetIndices 명령의 첫 번째 인자에 사용한 문자들에 “l(ell)” 또는 “u”를 덧붙인 기호들 (예를 들면 la 또는 ub 등등)의 원래 가지고 있던 값이 **삭제** 됨을 주의하자.

In[]:= **le (* previously le=="POSSIBLE" *)**

Out[]:=
le

In[]:= **le = "NOT POSSIBLE";**

Set: Symbol le is Protected. [i](#)

In[]:=

? le

Out[]:=

Symbol

Global`le

UpValue Definitions

DnIndexQ[le] ^:= True**IndexToKind[le] ^:= Latin****RegularIndexQ[le] ^:= True****UpIndexQ[le] ^:= False**

Attributes {Protected}

Full Name Global`le

^

SetIndices 명령의 첫 번째 인자로 빈 리스트 {}를 사용하면 그 IndexKind로 사용 가능했던 모든 인덱스가 제거된다:

In[]:=

SetIndices[{}, Latin]; GetIndices[Latin]

Out[]:=

{ }

In[]:=

le (* NB: The previous value of le is **CLEARED! *)**

Out[]:=

le

In[]:=

la = "POSSIBLE"

Out[]:=

POSSIBLE

In[]:=

? la

Out[]:=

Symbol

Global`la

Assignment

la = POSSIBLE

UpValue Definitions

UpIndexQ[la] ^:= False

Full Name Global`la

^

In[*]:= **SetIndices**[{"a", "b"}, Latin]; **GetIndices**[Latin]

Out[*]=
{1a, 1b, ua, ub}

In[*]:= **1a**

Out[*]=
1a

In[*]:= **1a = "NOT POSSIBLE";**

Set: Symbol 1a is Protected. [i](#)

In[*]:= **? 1a**

Out[*]=

Symbol
Global`1a
UpValue Definitions
DnIndexQ [1a] ^= True
IndexToKind [1a] ^= Latin
RegularIndexQ [1a] ^= True
UpIndexQ [1a] ^= False
Attributes {Protected}
Full Name Global`1a
^

In[*]:= **SP`undefIndexKind**[Greek];
SP`defIndexKind[Greek, ToString /@ { μ , ν , ρ , σ }];
GetIndices[Greek]

Out[*]=
{1 μ , 1 ν , 1 ρ , 1 σ , u μ , u ν , u ρ , u σ }

In[*]:= **1 α = "POSSIBLE"**

Out[*]=
POSSIBLE

In[*]:= **1 μ = "IMPOSSIBLE"**

Set: Symbol 1 μ is Protected. [i](#)

Out[*]=
IMPOSSIBLE

`In[*]:=``? l μ` `Out[*]=`

Symbol

Global`l μ

UpValue Definitions

`DnIndexQ[l μ] ^= True``IndexToKind[l $\mu$ ] ^= Greek``RegularIndexQ[l μ] ^= True``UpIndexQ[l μ] ^= False`

Attributes {Protected}

Full Name Global`l μ

^

`In[*]:=``SetIndices[Alphabet["Greek"], Greek]; GetIndices[Greek]``Out[*]=`

```
{l $\alpha$ , l $\beta$ , l $\gamma$ , l $\delta$ , l $\epsilon$ , l $\xi$ , l $\eta$ , l $\theta$ , l $\iota$ , l $\kappa$ , l $\lambda$ , l $\mu$ , l $\nu$ , l $\xi$ ,
  l $\omicron$ , l $\pi$ , l $\rho$ , l $\sigma$ , l $\tau$ , l $\upsilon$ , l $\varphi$ , l $\chi$ , l $\psi$ , l $\omega$ , u $\alpha$ , u $\beta$ , u $\gamma$ , u $\delta$ , u $\epsilon$ , u $\xi$ , u $\eta$ ,
  u $\theta$ , u $\iota$ , u $\kappa$ , u $\lambda$ , u $\mu$ , u $\nu$ , u $\xi$ , u $\omicron$ , u $\pi$ , u $\rho$ , u $\sigma$ , u $\tau$ , u $\upsilon$ , u $\varphi$ , u $\chi$ , u $\psi$ , u $\omega$ }
```

`In[*]:=``l $\alpha$` `Out[*]=`l $\alpha$ `In[*]:=``? l α` `Out[*]=`

Symbol

Global`l α

UpValue Definitions

`DnIndexQ[l α] ^= True``IndexToKind[l $\alpha$ ] ^= Greek``RegularIndexQ[l α] ^= True``UpIndexQ[l α] ^= False`

Attributes {Protected}

Full Name Global`l α

^

이미 사용하고 있는 인덱스는 다른 IndexKind의 인덱스로 설정할 수 없다:

```
In[*]:= SetIndices[ToString /@ {a,  $\mu$ , c}, Greek]
```

Msg: {a, μ , c} includes character(s) which are already defined.

영문 대문자 'AA'는 현재 어느 IndexKind에서도 사용하지 않으므로 사용 가능하다:

```
In[*]:= SetIndices[ToString /@ {AA, b, c}, Latin]; GetIndices[Latin] (* NO error! *)
```

```
Out[*]=
```

```
{1AA, 1b, 1c, uAA, ub, uc}
```

기본 설정으로 복귀한다:

```
In[*]:= SetIndices[Alphabet[], Latin]
SP`undefIndexKind[Greek]
```

AddIndices

```
In[*]:= AddIndices::usage
```

```
Out[*]=
```

```
AddIndices[{ "c1", "c2", ... }, ikind]
  adds new index characters to an existing index kind.
```

프로그램에서 사용할 수 있는 RegularType의 인덱스들에 새로운 인덱스들을 포함시킨다. SetIndices 명령에 의해 정의된 인덱스들에 더해서 새로운 인덱스들을 포함시키려면, 첫 번째 인자는 더해질 문자 리스트이고, 두 번째 인자는 IndexKind 이름이다.

```
In[*]:= SP`undefIndexKind[Capital];
SP`defIndexKind[Capital, {"A", "B"}];
GetIndices[Capital]
```

```
Out[*]=
```

```
{1A, 1B, uA, uB}
```

```
In[*]:= AddIndices[ToString /@ {C, D, E}, Capital]; GetIndices[Capital]
```

```
Out[*]=
```

```
{1A, 1B, 1C, 1D, 1E, uA, uB, uC, uD, uE}
```

다른 IndexKind에서 이미 사용 중인 인덱스를 포함시키면 안된다:

```
In[*]:= AddIndices[ToString /@ {a, b, F}, Capital]
```

Msg: {a, b, F} includes character(s) which are already defined.

동일한 IndexKind에 이미 사용 가능하게 등록된 인덱스들을 또 포함시키는 것은 아무런 효과가 없다:

```
In[*]:= AddIndices[ToString /@ {B, C, D}, Capital]; GetIndices[Capital]
```

```
Out[*]=
```

```
{1A, 1B, 1C, 1D, 1E, uA, uB, uC, uD, uE}
```

```
In[*]:= AddIndices[{"D"}, Capital]; GetIndices[Capital]
```

```
Out[*]=
```

```
{1A, 1B, 1C, 1D, 1E, uA, uB, uC, uD, uE}
```

포함시킬 인덱스가 많을 때에는 문자와 일련번호를 조합하여 새로운 인덱스들을 생성시킬 수 있다:

```
In[*]:= charL = "a" <> # & /@ ToString /@ Range[1, 10]
```

```
Out[*]=
```

```
{a1, a2, a3, a4, a5, a6, a7, a8, a9, a10}
```

```
In[*]:= AddIndices[charL, Latin]; GetIndices[Latin]
```

```
Out[*]=
```

```
{1a, 1b, 1c, 1d, 1e, 1f, 1g, 1h, 1i, 1j, 1k, 1l, 1m, 1n, 1o, 1p, 1q, 1r, 1s, 1t,
 1u, 1v, 1w, 1x, 1y, 1z, 1a1, 1a10, 1a2, 1a3, 1a4, 1a5, 1a6, 1a7, 1a8, 1a9,
 ua, ub, uc, ud, ue, uf, ug, uh, ui, uj, uk, ul, um, un, uo, up, uq, ur, us, ut,
 uu, uv, uw, ux, uy, uz, ua1, ua10, ua2, ua3, ua4, ua5, ua6, ua7, ua8, ua9}
```

```
In[*]:= ? 1a1
```

```
Out[*]=
```

Symbol

Global`1a1

UpValue Definitions

DnIndexQ[1a1] ^= True

IndexToKind[1a1] ^= Latin

RegularIndexQ[1a1] ^= True

UpIndexQ[1a1] ^= False

Attributes {Protected}

Full Name Global`1a1

^

기본 설정으로 복귀한다:

```
In[*]:= SetIndices[Alphabet[], Latin];
SP`undefIndexKind[Capital]
```

DropIndices

```
In[*]:= DropIndices::usage
```

```
Out[*]=
```

```
DropIndices[{"c1", "c2", ...}, ikind] removes
specified index characters from an existing index kind.
```

특정 IndexKind에 설정된 인덱스 중에서 몇 개를 다른 목적으로 사용하기 위해 제거한다:

In[*]:=	<code>DropIndices["t", Latin]</code>
In[*]:=	<code>SP`getCharacters[Latin]</code>
Out[*]=	<code>{a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, u, v, w, x, y, z}</code>
In[*]:=	<code>DropIndices[ToString /@ {x, y, z}, Latin]</code>
In[*]:=	<code>SP`getCharacters[Latin]</code>
Out[*]=	<code>{a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, u, v, w}</code>
In[*]:=	<code>DropIndices["m", Latin]</code>
In[*]:=	<code>SP`getCharacters[Latin]</code>
Out[*]=	<code>{a, b, c, d, e, f, g, h, i, j, k, l, n, o, p, q, r, s, u, v, w}</code>
In[*]:=	<code>DropIndices[{"p", "q", "r"}, Latin]</code>
In[*]:=	<code>SP`getCharacters[Latin]</code>
Out[*]=	<code>{a, b, c, d, e, f, g, h, i, j, k, l, n, o, s, u, v, w}</code>

기본 설정으로 복귀한다:

In[*]:=	<code>SetIndices[Alphabet[], Latin]</code>
---------	--

인덱스 함수

In[*]:=	<code>dnIndexL = {-1, la, lμ, lA}; upIndexL = {1, ua, uμ, uA}; indexL = Flatten[{dnIndexL, upIndexL}];</code>
In[*]:=	<code>SP`defIndexKind[Greek, Alphabet["Greek"]]; SP`defIndexKind[Capital, ToUpperCase /@ Alphabet[]];</code>

더미 인덱스 관련

NewDummy

In[*]:= **NewDummy::usage**

Out[*]=

NewDummy[ikind] generates a new unique dummy index of the specified index kind.

프로그램이 생성하는 더미 인덱스의 쌍을 리스트로 돌려 준다. 결과 값은 유일하고 {dn, up} 형태이다.

In[*]:= **dummyL = NewDummy /@ {Latin, Greek}**

Out[*]=

{{lLatin\$31913, uLatin\$31913}, {lGreek\$31915, uGreek\$31915}}

1D Kind의 DummyType은 없다:

In[*]:= **SP`defIndexKind[Zero, {"0"}];**

In[*]:= **NewDummy[Zero]**

Out[*]=

{l0, u0}

In[*]:= **SP`undefIndexKind[Zero]**

인덱스 올리고 내림

FlipIndex

In[*]:= **FlipIndex::usage**

Out[*]=

FlipIndex[index] changes an upper index to a lower index and vice-versa.

인덱스의 dn과 up을 뒤바꾼다.

In[*]:= **FlipIndex /@ dnIndexL**

Out[*]=

{1, ua, u μ , uA}

In[*]:= **FlipIndex /@ %**

Out[*]=

{-1, la, l μ , lA}

```
In[*]:= FlipIndex /@ indexL
Out[*]= {1, ua, u $\mu$ , uA, -1, la, l $\mu$ , lA}
```

ToDnIndex

```
In[*]:= ToDnIndex::usage
Out[*]= ToDnIndex[index] converts an upper index to its corresponding
lower index. It has no effect on lower or non-tensorial indices.
```

인덱스가 up이면 dn으로 바꾼다.

```
In[*]:= upIndexL
ToDnIndex /@ %
Out[*]= {1, ua, u $\mu$ , uA}
```

```
Out[*]= {-1, la, l $\mu$ , lA}
```

```
In[*]:= indexL
ToDnIndex /@ %
Out[*]= {-1, la, l $\mu$ , lA, 1, ua, u $\mu$ , uA}
Out[*]= {-1, la, l $\mu$ , lA, -1, la, l $\mu$ , lA}
```

ToUpIndex

```
In[*]:= ToUpIndex::usage
Out[*]= ToUpIndex[index] converts a lower index to its corresponding
upper index. It has no effect on upper or non-tensorial indices.
```

인덱스가 dn이면 up으로 바꾼다.

```
In[*]:= dnIndexL
ToUpIndex /@ %
Out[*]= {-1, la, l $\mu$ , lA}
```

```
Out[*]= {1, ua, u $\mu$ , uA}
```

```

In[*]:= indexL
ToUpIndex /@ %
Out[*]=
{-1, 1a, 1 $\mu$ , 1A, 1, ua, u $\mu$ , uA}
Out[*]=
{1, ua, u $\mu$ , uA, 1, ua, u $\mu$ , uA}

```

인덱스 정렬

IndexOrderedQ

```

In[*]:= IndexOrderedQ::usage
Out[*]=
IndexOrderedQ[indexList] returns True if the list
of indices is canonically sorted. IndexOrderedQ[list1, list2]
returns True if list1 is ordered before or is the same as list2.

In[*]:= dummyL = {NewDummy[Latin], NewDummy[Latin]}
Out[*]=
{{1Latin$31963, uLatin$31963}, {1Latin$31965, uLatin$31965}}

```

인덱스들이 순서대로 정렬되었는가를 판단하는 질의 함수이다.

```

In[*]:= IndexOrderedQ[{1b, dummyL[[1, 1]]}]
Out[*]=
True

In[*]:= IndexOrderedQ[{1a, 1b}]
Out[*]=
True

In[*]:= IndexOrderedQ[{1a, ua}]
Out[*]=
True

In[*]:= IndexOrderedQ[{1a, -1}]
Out[*]=
True

```

```
In[*]:= idxL = {la, ua, lb, ub} ~Join~ Flatten[dummyL] ~Join~ {-1, 1}
IndexOrderedQ[idxL]
```

```
Out[*]= {la, ua, lb, ub, lLatin$31963, uLatin$31963, lLatin$31965, uLatin$31965, -1, 1}
```

```
Out[*]= True
```

```
In[*]:= {dummyL[[1, 1], dummyL[[2, 1]]}
IndexOrderedQ[%]
```

```
Out[*]= {lLatin$31963, lLatin$31965}
```

```
Out[*]= True
```

인자가 두 개의 리스트인 경우, 그 두 리스트를 비교하여 정렬되었는가를 판단한다.

```
In[*]:= IndexOrderedQ[{la, lb, lc}, {la, lc}]
```

```
Out[*]= False
```

```
In[*]:= IndexOrderedQ[{la, lb, lc}, {la, lc, lb}]
```

```
Out[*]= True
```

IndexSort

```
In[*]:= IndexSort::usage
```

```
Out[*]= IndexSort[indexList] sorts a list of indices into a canonical order.
```

주어진 인덱스를 순서대로 정렬하는 함수이다. 우선 순위는 Tensorial/Component, Free/Dummy, Lexicographic, Lower/Upper에 따라 결정한다.

```
In[*]:= {la, ua, lb, ub, -1, 1} ~Join~ NewDummy[Latin]
idxL = RandomSample[%, Length@%]
```

```
Out[*]= {la, ua, lb, ub, -1, 1, lLatin$31997, uLatin$31997}
```

```
Out[*]= {ua, la, uLatin$31997, ub, -1, lb, lLatin$31997, 1}
```

IndexSort와 Sort의 비교:

```
In[*]:= pairIndexSort = SP`takePairs@idxL
```

```
Out[*]=
```

```
{ua, la, uLatin$31997, ub, lb, lLatin$31997}
```

```
In[*]:= lexIndexSort = Union[ToUpIndex /@ idxL]
```

```
Out[*]=
```

```
{1, ua, ub, uLatin$31997}
```

```
In[*]:= {SP`typeIndexSort[#, MemberQ[pairIndexSort, #],  
Position[lexIndexSort, ToUpIndex[#, {1}], UpIndexQ[#, #] & /@ idxL
```

```
Out[*]=
```

```
{{1.1, True, {{2}}, True, ua}, {1.1, True, {{2}}, False, la},  
{1.1, True, {{4}}, True, uLatin$31997}, {1.1, True, {{3}}, True, ub},  
{7, False, {{1}}, False, -1}, {1.1, True, {{3}}, False, lb},  
{1.1, True, {{4}}, False, lLatin$31997}, {7, False, {{1}}, True, 1}}
```

```
In[*]:= Sort[%]
```

```
Out[*]=
```

```
{{1.1, True, {{2}}, False, la}, {1.1, True, {{2}}, True, ua},  
{1.1, True, {{3}}, False, lb}, {1.1, True, {{3}}, True, ub},  
{1.1, True, {{4}}, False, lLatin$31997}, {1.1, True, {{4}}, True, uLatin$31997},  
{7, False, {{1}}, False, -1}, {7, False, {{1}}, True, 1}}
```

```
In[*]:= Last /@ %
```

```
Out[*]=
```

```
{la, ua, lb, ub, lLatin$31997, uLatin$31997, -1, 1}
```

```
In[*]:= % == IndexSort[idxL]
```

```
Out[*]=
```

```
True
```

Mathematica 자체의 Sort 함수와는 다른 결과를 준다:

```
In[*]:= Sort[idxL]
```

```
Out[*]=
```

```
{-1, 1, la, lb, lLatin$31997, ua, ub, uLatin$31997}
```

인덱스 쌍

PairIndexQ

In[*]:= PairIndexQ::usage

Out[*]=

PairIndexQ[index1, index2] returns True if index1 and index2 form a valid upper/lower pair. PairIndexQ[{{i1,j1},{i2,j2},...}] checks if all given pairs are valid index pairs.

인자의 인덱스들이 위/아래 쌍인가에 대한 질의 함수이다. 인자는 두 개의 인덱스이거나 {a,b} 형태인 임의의 개수의 인덱스 리스트들이다.

In[*]:= {PairIndexQ[1a, ua], PairIndexQ[1a, ub], PairIndexQ[1a, 1a]}

Out[*]=

{True, False, False}

In[*]:= {PairIndexQ[uA, 1A], PairIndexQ[1A, uB], PairIndexQ[1A, 1A]}

Out[*]=

{True, False, False}

1D Kind의 인덱스는 일차원 공간의 개념에 따라 인덱스 쌍이 될 수 없다. 또한 ComponentIndexQ를 만족하는 인덱스들도 인덱스 쌍이 될 수 없다.

In[*]:= SP`defIndexKind[Zero, {"0"}];

In[*]:= {PairIndexQ[-1, 1], PairIndexQ[10, u0]}

Out[*]=

{False, False}

In[*]:= SP`undefIndexKind[Zero]

인자가 여러 개의 인덱스 리스트인 경우, 인자 모두가 인덱스 쌍이어야 True 값을 준다.

In[*]:= PairIndexQ[{1a, ua}, {1μ, uμ}, {1A, uA}]

Out[*]=

True

In[*]:= PairIndexQ[{1a, ua}, {1μ, uv}]

Out[*]=

False

TakePairs

In[*]:= **TakePairs::usage**

Out[*]=

TakePairs[indexList] finds pairs
of identical upper and lower indices in 'indexList'.

인자로 주어진 인덱스 리스트에서 위/아래 쌍을 이루는 인덱스들을 돌려 준다. 결과 값은 {{dn,up},...} 형태이다. 옵션으로 IndexQs를 갖는다.

In[*]:= **indexL**

idxL = RandomSample[indexL, Length@indexL]

Out[*]=

{-1, la, l μ , lA, 1, ua, u μ , uA}

Out[*]=

{-1, u μ , uA, ua, la, 1, lA, l μ }

takePairs 함수는 인덱스 리스트에서 서로 쌍을 구성하는 인덱스들을 선택해서 **원래의 순서를 유지**한 결과를 돌려준다:

In[*]:= **SP`takePairs[idxL]**

Out[*]=

{u μ , uA, ua, la, lA, l μ }

In[*]:= **ToDnIndex /@ %**

Union@%

{#, ToUpIndex[#]} & /@ %

% == TakePairs[indexL]

Out[*]=

{l μ , lA, la, la, lA, l μ }

Out[*]=

{la, lA, l μ }

Out[*]=

{{la, ua}, {lA, uA}, {l μ , u μ }}

Out[*]=

True

Latin Kind의 인덱스들만 TakePairs 함수를 실행하려면 옵션 IndexQs를 사용하면 된다:

In[*]:= **TakePairs[idxL, IndexQs $\rightarrow$ {KindIndexQ[Latin]}]**

Out[*]=

{{la, ua}}

takePairsProper 함수는 인덱스 리스트에서 서로 쌍을 구성하는 인덱스들을 묶어서 **원래의 순서를 유지**한 결과를 돌려준다:

```
In[*]:= SP`takePairsProper[idxL]
Out[*]=
{{u $\mu$ , l $\mu$ }, {uA, lA}, {ua, la}}
```

기타

DuplicatedIndicesQ

```
In[*]:= DuplicatedIndicesQ::usage
Out[*]=
DuplicatedIndicesQ[indexL] returns True if there are any
duplicated tensorial indices in 'indexL', and False otherwise.
```

(성분 인덱스가 아닌) 텐서 인덱스가 중복되었는가를 묻는다:

```
In[*]:= DuplicatedIndicesQ[{-1, 1, 1}]
Out[*]=
False
```

```
In[*]:= DuplicatedIndicesQ[{la, lb, ua, la}]
Out[*]=
True
```

두 번째 인자가 True이면 텐서 인덱스가 중복되었을 경우 관련 메시지를 출력한다:

```
In[*]:= DuplicatedIndicesQ[{la, lb, ua, la}, True]
Out[*]=
Msg: duplicated indices: {la, lb, ua, la}
True
```

TensorialIndexQ

```
In[*]:= TensorialIndexQ::usage
Out[*]=
TensorialIndexQ[index] returns True if 'index' is a
symbolic index (regular or dummy), as opposed to a component index.
```

RegularType이거나 DummyType 인덱스이어서 ComponentIndexQ를 만족하지 않으면 텐서 인덱스이다.

```
In[*]:= dnIndexL
Out[*]=
{-1, la, l $\mu$ , lA}
```

```
In[*]:= SP`indexClass /@ dnIndexL
```

```
Out[*]= {ComponentIndexQ, Latin, Greek, Capital}
```

```
In[*]:= TensorialIndexQ /@ dnIndexL
```

```
Out[*]= {False, True, True, True}
```

```
In[*]:= TensorialIndexQ[#] && KindIndexQ[Capital] [#] & /@ dnIndexL
```

```
Out[*]= {False, False, False, True}
```

```
In[*]:= Select[dnIndexL, TensorialIndexQ]
```

```
Out[*]= {1a, 1 $\mu$ , 1A}
```

```
In[*]:= SP`takePairs[indexL]
```

```
Out[*]= {1a, 1 $\mu$ , 1A, ua, u $\mu$ , uA}
```

```
In[*]:= Select[indexL, TensorialIndexQ]
```

```
Out[*]= {1a, 1 $\mu$ , 1A, ua, u $\mu$ , uA}
```

UpupDndnIndexQ

```
In[*]:= UpupDndnIndexQ::usage
```

```
Out[*]= UpupDndnIndexQ[indexL] returns True if indices in indexL are all dn or all up.
```

인덱스 리스트의 원소가 모두 Lower 인덱스이거나 모두 Upper 인덱스인가를 묻는 질의 함수이다.

```
In[*]:= UpupDndnIndexQ /@ {{1a, 1b, 1c}, {1a, ub, 1c}, {ua, ub, 1c}, {ua, ub, ua}}
```

```
Out[*]= {True, False, False, True}
```

Local Functions

```
In[*]:= indexL
```

```
Out[*]= {-1, 1a, 1 $\mu$ , 1A, 1, ua, u $\mu$ , uA}
```

dnupState

```
In[*]:= SP`dnupState /@ indexL
Out[*]= {-1, -1, -1, -1, 1, 1, 1, 1}
```

takeTensorialFrees

텐서 인덱스 중에서 위/아래 쌍을 구성하지 않는 인덱스들을 돌려 준다:

```
In[*]:= SP`takeTensorialFrees[{1a, ua, 1c}]
Out[*]= {1c}
```

```
In[*]:= SP`takeTensorialFrees[indexL]
Out[*]= {}
```

dropPairs, takePairs, takePairsProper

인덱스 리스트에서 위/아래 쌍들을 제거한 리스트를 돌려 준다:

```
In[*]:= SP`dropPairs[indexL]
Out[*]= {-1, 1}
```

인덱스로 구성된 리스트에서 서로 쌍을 구성하는 인덱스들만 선택해서 ‘원래의 순서’를 유지한 채로 돌려준다:

```
In[*]:= SP`takePairs[indexL]
Out[*]= {1a, 1μ, 1A, ua, uμ, uA}
```

인덱스로 구성된 리스트에서 서로 쌍을 구성하는 인덱스들만 선택해서 ‘원래의 순서’를 유지하면서 {dn, up} 형태의 리스트로 돌려준다:

```
In[*]:= SP`takePairsProper[indexL]
Out[*]= {{1a, ua}, {1μ, uμ}, {1A, uA}}
```

‘원래의 순서’를 유지하지 않는 TakePairs의 결과와 비교하자:

```
In[*]:= TakePairs[indexL]
Out[*]= {{1a, ua}, {1A, uA}, {1μ, uμ}}
```

inPairQ

두 번째 인자인 어떤 인덱스와 서로 쌍을 구성하는 인덱스가 첫 번째 인자인 인덱스 리스트에 존재하는 가를 묻는 질의 함수이다.

```
In[*]:= indexL
SP`inPairQ[indexL, #] & /@ indexL

Out[*]=
{-1, la, lμ, lA, 1, ua, uμ, uA}

Out[*]=
{False, True, True, True, False, True, True, True}
```

기본 설정으로 복귀

```
In[*]:= SP`undefIndexKind[Greek];
SP`undefIndexKind[Capital];
```

Kind 설정

DefKind, UndefKind, DefinedKindQ

```
In[*]:= DefKind::usage
Out[*]=
DefKind[kind, {"a", "b", ...}] defines a new Kind with a set of indices.

In[*]:= UndefKind::usage
Out[*]=
UndefKind[kind] removes the definition of the specified kind.

In[*]:= DefinedKindQ::usage
Out[*]=
DefinedKindQ[kind] returns True if 'kind' is a defined kind.
```

Kind는 tangent bundle 개념으로 볼 수 있다. 따라서 Kind를 정의할 때 탄젠트 벡터들 (v^μ, ξ^a 등)이 사용할 IndexKind (μ, a 등)이 필요하다. 그리고 base manifold의 성질인 차원, 좌표계, Metric Signature 등을 설정할 수 있다. 예를 들면, 게이지 포텐셜 A_μ^a 의 경우 시공간에 대한 Kind는 ‘Greek’, 게이지 군의 Kind는 ‘Latin’이다.

Kind가 수학적으로는 tangent bundle과 완전히 동일하지는 않다. 왜냐하면, spin connection $\omega_{\mu a}^b$ 처럼, 하나의 tangent bundle을 (서로 다른 Kind들인) 좌표 기준에서의 ‘Greek’ Kind와 비좌표 기준에서의 ‘Latin’ Kind로 각각 묘사하기 때문이다.

Kind의 이름으로 base manifold의 이름을 사용하는 것이 개념적으로는 적당할 수도 있다. 그러나 텐서 대수를 다루는 본 프로그램의 입장에서는 텐서들이 사용할 인덱스의 종류가 더 중요하다.

(또는 수학적으로는 텐서가 아니지만 인덱스 표기법을 사용하는 경우에도 적용할 수 있도록 수학적 엄밀성을 요구하지 않을 수도 있다.) 그래서 Kind의 이름으로 base manifold의 이름이 아닌 (그 manifold의 탄젠트 공간 이름인) IndexKind의 이름을 차용할 것이다. (심볼의 이름 공간을 단순하게 할 목적도 있다.) 이를 위하여 DefKind[kind, ..] 함수는 첫 번째 인자인 Kind의 이름 'kind'로 local 함수인 defIndexKind[kind, ..] 함수를 호출한 후, 나중에 Kind (또는 base manifold)의 성질인 차원, 좌표계 등을 설정한다.

Kind와 IndexKind에 동일한 이름을 사용하기 때문에 혼란이 발생할 수 있다. 그러나 각각에 대한 구별은 상황에 따라 IndexKind인지 Kind인지를 구별할 수 있고, 특히 IndexKind 개념은 내부 구현에서만 사용될 것이기 때문에, 사용자 입장에서는 Kind 개념만 고려하면 된다.

주의: IndexKind는 defIndexKind[..] 함수로 정의하고, 정의를 취소하려면 UndefKind[..] 함수가 아니고 undefIndexKind[..] 함수를 사용해야 한다. 같은 의미로, Kind는 DefKind[..] 함수로 정의하고, 정의를 취소하려면 undefIndexKind[..] 함수가 아니고 UndefKind[..] 함수를 사용해야 한다.

그리스 문자($\mu, \nu, \rho, \dots$)를 IndexKind로 사용하는 'Greek' Kind를 정의한다:

In[]:=	UndefKind[Greek] DefKind[Greek, Alphabet["Greek"]]
In[]:=	DefinedKindQ[Greek]
Out[]:=	True
In[]:=	GetIndices::usage
Out[]:=	GetIndices[ikind] returns all defined indices of the specified index kind.
In[]:=	GetIndices[Greek]
Out[]:=	$\{1\alpha, 1\beta, 1\gamma, 1\delta, 1\epsilon, 1\zeta, 1\eta, 1\theta, 1\iota, 1\kappa, 1\lambda, 1\mu, 1\nu, 1\xi,$ $1\omicron, 1\pi, 1\rho, 1\sigma, 1\tau, 1\nu, 1\varphi, 1\chi, 1\psi, 1\omega, u\alpha, u\beta, u\gamma, u\delta, u\epsilon, u\zeta, u\eta,$ $u\theta, u\iota, u\kappa, u\lambda, u\mu, u\nu, u\xi, u\omicron, u\pi, u\rho, u\sigma, u\tau, u\nu, u\varphi, u\chi, u\psi, u\omega\}$
In[]:=	GetDimension::usage
Out[]:=	GetDimension[kind] returns the dimension of the specified kind.
	GetDimension[Greek] (* SetDimension[Greek]로 설정 필요 *)
Out[]:=	GetDimension[Greek]

In[*]:= **GetEpsilon::usage**

Out[*]=

GetEpsilon[kind] returns the Levi-Civita tensor (volume form) associated with the specified 'kind'.

In[*]:= **GetEpsilon[Greek]**

Out[*]=

EpsilonGreek

In[*]:= **? EpsilonGreek**

Out[*]=

Symbol

Global`EpsilonGreek

UpValue Definitions

mGRG`STensor`Private`getDnupL[EpsilonGreek] ^= {-1}

mGRG`STensor`Private`getGenSet[EpsilonGreek] ^= Antisymmetric

mGRG`STensor`Private`getKindL[EpsilonGreek] ^= {Greek}

mGRG`STensor`Private`getPrtStr[EpsilonGreek] ^= ∈ [Greek]

mGRG`STensor`Private`getRank[EpsilonGreek] ^= -1

mGRG`STensor`Private`getType[EpsilonGreek] ^= IndexedTensorQ

IndexedObjectQ[EpsilonGreek] ^= True

IndexedOperandQ[EpsilonGreek] ^= True

Format Definitions

EpsilonGreek /: MakeBoxes[EpsilonGreek[mGRG`STensor`Private`args\$____], StandardForm

Full Name Global`EpsilonGreek

^

이미 정의된 'Greek' 계열을 제거한다:

In[*]:= **UndefKind[Greek]**

In[*]:= **DefinedKindQ[Greek]**

Out[*]=

False

In[*]:= **GetDimension[Greek]**

Out[*]=

GetDimension[Greek]

In[*]:= **GetEpsilon[Greek]**

Out[*]=

GetEpsilon[Greek]

In[*]:= **? EpsilonGreek**

Out[*]=

Missing[UnknownSymbol, EpsilonGreek]

‘Latin’ Kind은 기본적으로 정의되었고, DefaultKind로 설정되었다:

In[*]:= **DefaultKind::usage**

Out[*]=

DefaultKind represents the default kind
(usually Latin) used when no kind is explicitly specified.

In[*]:= **DefinedKindQ[Latin]**

Out[*]=

True

DefaultKind로 설정된 Kind는 제거할 수 없다:

In[*]:= **UndefKind[Latin]**

Msg: Cannot remove the Latin kind.

CheckKind

CheckKind 함수는 DefinedKindQ 함수와 거의 동일하다. 차이는 단지 인자가 정의되지 않았을 경우에 오류 메시지를 출력하는 것 뿐이다.

In[*]:= **CheckKind::usage**

Out[*]=

CheckKind[kind] returns True if 'kind' is a defined kind, and returns False with an error message otherwise. It can also be applied to a list of kinds, returning True only if all kinds in the list are defined.

KindMatchQ

In[*]:= **KindMatchQ::usage**

Out[*]=

KindMatchQ[kind1, kind2] returns True if the two kinds are compatible, and False otherwise. The special kind 'All' is compatible with any other kind, while 'NonKind' is not compatible with any kind.

텐서 Kind가 서로 부합하는가를 묻는다. 특히 Kind의 이름이 All이면 모든 Kind와 부합한다.

In[*]:= {KindMatchQ[All, Latin], KindMatchQ[Capital, All], KindMatchQ[Latin, Capital]}

Out[*]=

{True, True, False}

Coordinates, Dimension, and Signature

Get, Set and Clear Coordinates

In[*]:= **GetCoordinates::usage**

Out[*]=

GetCoordinates[kind] returns the coordinates of the specified kind.

In[*]:= **SetCoordinates::usage**

Out[*]=

SetCoordinates[coSys, kind] sets the coordinates for the specified 'kind'.
Use SetCoordinates[coSys, basis, kind] for a non-coordinate basis.

In[*]:= **ClearCoordinates::usage**

Out[*]=

ClearCoordinates[kind] removes
the coordinate definitions for the specified 'kind'.

좌표계를 설정하거나 삭제한다. 옵션으로 Kind를 입력할 수 있다.

DefaultKind의 좌표계를 설정하고 삭제한다:

In[*]:= **coSys = {t, x, y, z};**
SetCoordinates[coSys]
GetCoordinates[]

Out[*]=

{t, x, y, z}

In[*]:= **ClearCoordinates[]**

비좌표 기준(Non-coordinate Basis)에서의 좌표계 설정과 삭제:

In[*]:= Off[CoordinateBasisFlag]

In[*]:= **coSys** = {t, x, y, z};
basisM = {{ $e^{-\rho[t,x]}$, 0, 0, 0}, {0, $e^{-\rho[t,x]}$, 0, 0}, {0, 0, 1, 0}, {0, 0, 0, 1}};
SetCoordinates[coSys, basisM];
GetCoordinates[]

Out[*]=
{t, x, y, z}

In[*]:= ClearCoordinates[]

좌표 기준으로 복귀한다:

In[*]:= On[CoordinateBasisFlag]

Get, Set, and Clear Dimension and Signature

In[*]:= GetDimension::usage

Out[*]=
GetDimension[kind] returns the dimension of the specified kind.

In[*]:= SetDimension::usage

Out[*]=
SetDimension[n, kind] sets the dimension for the specified 'kind' to 'n'.

In[*]:= ClearDimension::usage

Out[*]=
ClearDimension[kind] removes the dimension definition for the specified 'kind'.

DefaultKind의 차원 크기를 설정하고 조회한다:

In[*]:= SetDimension[4]
GetDimension[]

Out[*]=
4

In[*]:= GetSig::usage

Out[*]=
GetSig[kind] returns the number of
minus in metric's eigenvalues of the specified kind.

In[*]:= SetSig::usage

Out[*]=
SetSig[s, kind] sets the signature parameter 's' (number
of minus signs in the metric signature) for the specified 'kind'.

```
In[*]:= ClearSig::usage
```

```
Out[*]=
```

```
ClearSig[kind] removes the signature definition for the specified 'kind'.
```

DefaultKind의 Sig를 설정하고 조회한다. Sig는 계량 텐서를 대각선 단위 행렬 형태로 만들었을 때 음수의 갯수로 정의한다. (참고: Wald (1984), 88 페이지.) 한편, Signature는 대각선 요소의 합으로 정의한다. (참고: Hawking and Ellis (1973), 38 페이지)

```
In[*]:= SetSig[1]
```

```
GetSig[]
```

```
Out[*]=
```

```
1
```

DefaultKind의 차원 크기와 Sig를 삭제한다:

```
In[*]:= ClearDimension[]
```

```
GetDimension[]
```

```
Out[*]=
```

```
GetDimension[Latin]
```

```
In[*]:= ClearSig[]
```

```
GetSig[]
```

```
Out[*]=
```

```
GetSig[Latin]
```

특정 Kind의 차원 크기와 Sig를 설정, 조회, 삭제하려면 인자로 Kind의 이름을 사용한다:

```
In[*]:= DefKind[Capital, {"A", "B", "C"}]
```

```
In[*]:= SetDimension[3, Capital]
```

```
GetDimension[Capital]
```

```
Out[*]=
```

```
3
```

```
In[*]:= ClearDimension[Capital];
```

```
GetDimension[Capital]
```

```
Out[*]=
```

```
GetDimension[Capital]
```

```
In[*]:= SetSig[3, Capital]
```

```
GetSig[Capital]
```

```
Out[*]=
```

```
3
```

```
In[*]:= ClearSig[Capital];
         GetSig[Capital]
```

```
Out[*]:= GetSig[Capital]
```

```
In[*]:= UndefKind[Capital]
```

차원 크기나 Sig의 값을 숫자가 아닌 기호로 설정하려면 Get... 함수를 직접 사용한다:

```
In[*]:= GetDimension[Greek] = n;
         GetDimension[Greek]
```

```
Out[*]:= n
```

```
In[*]:= GetDimension[Greek] = .;
         GetDimension[Greek]
```

```
Out[*]:= GetDimension[Greek]
```

```
In[*]:= GetSig[Latin] = s;
         GetSig[Latin]
```

```
Out[*]:= s
```

```
In[*]:= GetSig[Latin] = .;
         GetSig[Latin]
```

```
Out[*]:= GetSig[Latin]
```

Show Kind's info

프로그램의 옵션과 전역 변수의 값을 표시해 준다. 옵션으로 Kind를 입력할 수 있다.

```
In[*]:= SetDimension[4]
Show[DefaultKind]
```

```
Out[*]//TableForm=
```

AutoFlag	False
MarkErrorFlag	True
ResetDummiesFlag	True
SyntaxCheckFlag	False
KdeltaFlag	True
MetricgFlag	True
InitCTensorFlag	False
TorsionFreeQ of CD	True

Kind	Latin
Dimension	4
Sig	Any
Coordinates	none
CoordinateBasisQ	True
EvaluateBDFlag	False

```
In[*]:= SetSig[1]
Show[DefaultKind]
```

```
Out[*]//TableForm=
```

AutoFlag	False
MarkErrorFlag	True
ResetDummiesFlag	True
SyntaxCheckFlag	False
KdeltaFlag	True
MetricgFlag	True
InitCTensorFlag	False
TorsionFreeQ of CD	True

Kind	Latin
Dimension	4
Sig	1
Coordinates	none
CoordinateBasisQ	True
EvaluateBDFlag	False

```
In[ ]:= ClearDimension[]
Show[DefaultKind]
```

```
Out[ ]//TableForm=
```

AutoFlag	False
MarkErrorFlag	True
ResetDummiesFlag	True
SyntaxCheckFlag	False
KdeltaFlag	True
MetricgFlag	True
InitCTensorFlag	False
TorsionFreeQ of CD	True

Kind	Latin
Dimension	Any
Sig	1
Coordinates	none
CoordinateBasisQ	True
EvaluateBDFlag	False

좌표계와 그에 따른 차원을 설정한다.

```
In[ ]:= SetCoordinates[{t, r,  $\theta$ ,  $\phi$ }]
Show[DefaultKind]
```

```
Out[ ]//TableForm=
```

AutoFlag	False
MarkErrorFlag	True
ResetDummiesFlag	True
SyntaxCheckFlag	False
KdeltaFlag	True
MetricgFlag	True
InitCTensorFlag	False
TorsionFreeQ of CD	True

Kind	Latin
Dimension	4
Sig	1
Coordinates	t r θ ϕ
CoordinateBasisQ	True
EvaluateBDFlag	False

설정된 좌표계와 차원의 값을 제거한다.

```
In[*]:= ClearCoordinates[]
Show[DefaultKind]
```

```
Out[*]//TableForm=
```

AutoFlag	False
MarkErrorFlag	True
ResetDummiesFlag	True
SyntaxCheckFlag	False
KdeltaFlag	True
MetricgFlag	True
InitCTensorFlag	False
TorsionFreeQ of CD	True

Kind	Latin
Dimension	Any
Sig	1
Coordinates	none
CoordinateBasisQ	True
EvaluateBDFlag	False

Kind에 따른 정보를 보여준다:

```
In[*]:= DefKind[Greek, {"μ", "ν"}]
```

```
In[*]:= Show[Greek]
```

```
Out[*]//TableForm=
```

Kind	Greek
Dimension	Any
Sig	Any
Coordinates	none
CoordinateBasisQ	False
EvaluateBDFlag	False

```
In[*]:= SetDimension[2, Greek]
Show[Greek]
```

```
Out[*]//TableForm=
```

Kind	Greek
Dimension	2
Sig	Any
Coordinates	none
CoordinateBasisQ	False
EvaluateBDFlag	False

```
In[*]:= ClearDimension[Greek]
Show[Greek]
```

```
Out[*]//TableForm=
```

Kind	Greek
Dimension	Any
Sig	Any
Coordinates	none
CoordinateBasisQ	False
EvaluateBDFlag	False

```
In[*]:= UndefKind[Greek]
```

Checking Indices

```
In[*]:= dnIndexL = {-1, la, lA};
upIndexL = {1, ua, uA};
notIndexL = {laa, uaa};
indexL = Flatten[{dnIndexL, upIndexL, notIndexL}];
```

ValidIndexQ

```
In[*]:= ValidIndexQ::usage
```

```
Out[*]=
```

ValidIndexQ[index, kind] returns True if 'index' is a valid index for the specified 'kind'. It checks for index kind compatibility for symbolic indices and dimension bounds for component indices.

인자가 텐서 인덱스로서 합당한가를 판단하는 질의 함수이다. 성분 인덱스도 합당한 인덱스로 간주한다.

```
In[*]:= DefKind[Capital, ToUpperCase /@ Alphabet[]];
```

Kind를 옵션으로 지정하지 않으면 DefaultKind의 인덱스로 간주한다:

```
In[*]:= DefaultKind
indexL
ValidIndexQ /@ indexL
```

```
Out[*]=
```

Latin

```
Out[*]=
```

{-1, la, lA, 1, ua, uA, laa, uaa}

```
Out[*]=
```

{True, True, False, True, True, False, False, False}

성분 인덱스 (0이 아닌 정수)에 대해서는, 두 번째 인자에 대한 Dimension 이 설정되어 있으면, 그 범위에 첫 번째 인자가 포함되어야 True 값을 돌려준다. Dimension 이 설정되어 있지 않으면, 첫 번째 인자의 값과 무관하게 True 값을 돌려준다.

```
In[*]:= ValidIndexQ[100, Latin]
```

```
Out[*]=
```

True

```
In[*]:= Select[indexL, ValidIndexQ[#, something] &]
```

```
Out[*]=
```

{-1, 1}

옵션으로 All을 사용하면 인덱스 Kind와는 무관하게 합당한 인덱스인가를 묻는다:

```
In[*]:= ValidIndexQ[#, All] & /@ indexL
```

```
Out[*]:= {True, True, True, True, True, True, False, False}
```

인자가 리스트이면 그 리스트의 모든 원소가 DefaultKind의 인덱스로 합당한가를 묻는다:

```
In[*]:= ValidIndexQ[indexL]
```

```
Out[*]:= False
```

인덱스 리스트에서 DefaultKind의 텐서 인덱스들만 추출할 수 있다:

```
In[*]:= Select[indexL, ValidIndexQ[#] && TensorialIndexQ[#] &]
```

```
Out[*]:= {1a, ua}
```

옵션으로 Kind가 주어지면 그 Kind에 합당한 인덱스인가를 판단한다. Kind가 옵션으로 주어지지 않은 경우는 Kind는 DefaultKind와 같다.

```
In[*]:= Select[indexL, ValidIndexQ]
```

```
Out[*]:= {-1, 1a, 1, ua}
```

```
In[*]:= Select[indexL, ValidIndexQ[#, Capital] &]
```

```
Out[*]:= {-1, 1A, 1, uA}
```

```
In[*]:= Select[indexL, ValidIndexQ[#, All] &]
```

```
Out[*]:= {-1, 1a, 1A, 1, ua, uA}
```

ValidIndicesQ

```
In[*]:= ValidIndicesQ::usage
```

```
Out[*]:= ValidIndicesQ[indexList, kind] returns True if all indices in 'indexList' are
valid for the specified 'kind' and there are no duplicated symbolic indices.
```

인덱스 리스트의 모든 원소가 서로서로 합당한 DefaultKind의 인덱스인가를 묻는다:

```
In[*]:= ValidIndicesQ[{1, 1a}]
```

```
Out[*]:= True
```

옵션으로 Kind가 주어지면 모든 인덱스가 그 Kind의 인덱스로 서로서로 합당한가를 묻는다:

In[*]:= **ValidIndicesQ**[{1a, 1A}, Latin]

Out[*]=
False

In[*]:= **ValidIndicesQ**[{1a, 1A}, All]

Out[*]=
True

동일한 인덱스가 있으면 서로 합당하지 않은 것으로 간주한다:

In[*]:= **{ValidIndicesQ**[{1a, ua}], **ValidIndicesQ**[{1a, 1a}]}

Out[*]=
{True, False}

초기 설정으로 복귀한다:

In[*]:= **UndefKind**[Capital]

IndexedObjects

In[*]:= **DefKind**[Greek, Alphabet["Greek"]];
DefKind[Capital, ToUpperCase /@ Alphabet[]];
DefKind[Bar, "b" <> # & /@ Alphabet[]];
DefKind[Dot, "d" <> # & /@ Alphabet[]];
DefKind[Hat, "h" <> # & /@ Alphabet[]];

Objects

Object는 IndexedObject와 ScalarFunction으로 나뉜다. 그리고 IndexedObject는 Operator와 Operand (Tensor, DiffForm 등)으로 나뉜다.

Object와 관련된 함수는 옵션으로 그것의 이름이 만족해야 할 질의 함수 리스트인 HeadQs를 가질 수 있다. 이 옵션이 존재하는 함수는 그 질의 함수들을 모두 만족하는 Object에 대해서만 작용한다. 질의 함수는 이름, 즉 Head를 인자로 하는 함수이어야 한다. 예를 들면, T[1a,1b]에서 질의 함수 someQ의 결과값은 someQ[T]이고, CD[1a,T[1b,1c]]에서는 someQ[T]의 값이 아니고 someQ[CD]의 값이다. 이것은 CD[1a,T[1b,1c]]의 Head가 CD이기 때문이다. 이 옵션을 사용하면 임의의 Object를 특정한 질의 함수를 만족하는 Object들만으로 분류할 수 있고, 연산도 그러한 종류에만 작용시킬 수 있다.

HeadQs의 기본 옵션은 IndexedObjectQ이다. 따라서 보통의 연산에서 ScalarFunction들은 제외된다. ScalarFunction들을 포함한 모든 Object에 연산을 적용하려면 HeadQs 옵션에 ObjectQ를 사용하면 된다.

ObjectQ는 인자로 주어진 이름이 Tensor, Operator 등의 IndexedObject이거나 ScalarFunction이면 True 값을 돌려주는 질의 함수이다.

In[*]:= **RemoveIndexedObject::usage**

Out[*]=

RemoveIndexedObject[oName] removes all definitions associated with the indexed object 'oName'. It does not work on reserved names.

RemoveIndexedObject 명령은 프로그램 사용 중 정의한 IndexedObject의 이름을 프로그램에서 제거한다. 인자는 이름이나 이름 리스트이다. 정의되지 않은 이름에는 아무런 변화가 없다. 프로그램에서 미리 정의된 IndexedObject의 이름은 제거할 수 없다.

IndexedObjectQ

In[*]:= **IndexedObjectQ::usage**

Out[*]=

IndexedObjectQ[name] returns True if 'name' is a defined indexed object (like a tensor or an operator), and False otherwise.

인자로 주어진 이름이 Tensor, Operator 등으로 정의되었으면 True 값을 돌려주는 질의 함수이다.

IndexedOperandQ

In[*]:= **IndexedOperandQ::usage**

Out[*]=

IndexedOperandQ[name] returns True if 'name' is an indexed operand (an object that takes indices, like a tensor or differential form), and False otherwise.

IndexedOperandQ는 인자로 주어진 이름이 Operand (텐서, 미분 형식 등)으로 정의되었으면 True 값을 돌려주는 질의 함수이다.

In[*]:= **IndexedTensorQ::usage**

Out[*]=

IndexedTensorQ[name] returns True if 'name' is a defined indexed tensor, and False otherwise.

IndexedTensorQ는 인자로 주어진 이름이 텐서이면 True 값을 돌려주는 질의 함수이다.

In[*]:= **DiffFormQ::usage**

Out[*]=

DiffFormQ[name] returns True if 'name' is a defined differential form, and False otherwise.

DiffFormQ는 인자로 주어진 이름이 미분 형식(differential form)이면 True 값을 돌려주는 질의 함수이다.

IndexedOperatorQ

```
In[*]:= IndexedOperatorQ::usage
Out[*]=
```

```
IndexedOperatorQ[name] returns True if 'name' is a
    defined indexed operator (like CD or LD), and False otherwise.
```

인자로 주어진 이름이 Operator로 정의되었으면 True 값을 돌려주는 질의 함수이다.

ScalarFunctionQ

인자로 주어진 이름이 ScalarFunction이면 True 값을 돌려주는 질의 함수이다.

```
In[*]:= ScalarFunctionQ::usage
Out[*]=
```

```
ScalarFunctionQ[name] returns True if 'name' is a scalar function
    (e.g., Sin, Cos, or Tscalar), which can take scalar arguments.
```

```
In[*]:= ScalarFunctionQ /@ {Tscalar, Power, Log, CD}
Out[*]=
```

```
{True, True, True, False}
```

Operands

Local Functions for defining IndexedObject

stringToImag

인덱스의 교환 대칭을 표현하는 대칭 문자열은 알파벳 문자로 이루어진 문자열이고, 대칭이면 '+', 반대칭이면 '-'가 첫 번째 문자이다.

문자열로 표시된 인덱스 대칭 문자열을 '부호있는 순열' 표기로 바꾼다:

```
In[*]:= SP`stringToImag[" abc "]
Out[*]=
```

```
Imag[1, 2, 3]
```

```
In[*]:= SP`stringToImag[" -bac"]
Out[*]=
```

```
-Imag[2, 1, 3]
```

```
In[*]:= SP`stringToImag[" + cba"]
```

```
Out[*]:=
Imag[3, 2, 1]
```

대칭 문자열의 실제 문자값은 중요하지 않다. 단지 그 문자들의 순서만 의미있다:

```
In[*]:= {SP`stringToImag["ab"], SP`stringToImag["cq"]}
```

```
Out[*]:=
{Imag[1, 2], Imag[1, 2]}
```

적절한 대칭 문자열이 아니면 오류 메시지가 출력된다:

```
In[*]:= SP`stringToImag[" aba"] (* Use "abc" *)
```

General: aba is not a valid perm.

```
In[*]:= SP`stringToImag["- abc"] (* Use "abc" *)
```

General: - abc is not a valid signed perm.

```
In[*]:= SP`stringToImag["aa"] (* Use "ab" *)
```

General: aa is not a valid perm.

```
In[*]:= SP`stringToImag["-cab"] (* Use "cab": Signature[{3,1,2}] == 1 *)
```

General: -cab is not a valid signed perm.

toRankAndGenSet

내부적으로 인덱스 대칭은 순열군을 사용한다. 따라서 대칭 문자열을 순열군의 Generating Set으로 변환하는 작업이 필요하다.

대칭 문자열을 {rank, symmetry}로 변환한다:

```
In[*]:= SP`toRankAndGenSet[""]
```

```
Out[*]:=
{0, GenSet[]}
```

```
In[*]:= SP`toRankAndGenSet["ab"]
```

```
Out[*]:=
{2, GenSet[]}
```

```
In[*]:= SP`toRankAndGenSet["-bac-acb"]
```

```
Out[*]:=
{3, GenSet[{Cycles[{1, 2}], -1}, {Cycles[{2, 3}], -1}]}
```

대칭 문자열 대신에 rank 값을 직접 입력할 수 있다:

In[*]:= **SP`toRankAndGenSet**["10"]

Out[*]=

{10, Nosymmetric}

대칭 문자열 대신에 rank 값을 입력하면서 완전 대칭 또는 완전 반대칭을 지정할 수 있다:

In[*]:= **SP`toRankAndGenSet**["10+"]

Out[*]=

{10, Symmetric}

In[*]:= **SP`toRankAndGenSet**["10-"]

Out[*]=

{10, Antisymmetric}

임의의 rank 값을 갖는 경우도 다를 수 있다:

In[*]:= **SP`toRankAndGenSet**["*"] (* '-1'은 임의 크기의 랭크를 의미 *)

Out[*]=

{-1, Nosymmetric}

In[*]:= **SP`toRankAndGenSet**["*+"]

Out[*]=

{-1, Symmetric}

In[*]:= **SP`toRankAndGenSet**["*-"]

Out[*]=

{-1, Antisymmetric}

대칭 문자열에 오류가 있으면 {-1, "Error"}을 돌려준다:

In[*]:= **SP`toRankAndGenSet**["-abc"]

Out[*]=

General: -abc is not a valid signed perm.

{-1, Error}

In[*]:= **SP`toRankAndGenSet**["-cab"]

Out[*]=

General: -cab is not a valid signed perm.

{-1, Error}

```
In[*]:= SP`toRankAndGenSet["ab+abc"]
```

... General: ab+abc is not a valid perm; it has inconsistent rank.

```
Out[*]=
```

```
{-1, Error}
```

```
In[*]:= SP`toRankAndGenSet["abc-ba"]
```

... General: abc-ba is not a valid perm; it has inconsistent rank.

```
Out[*]=
```

```
{-1, Error}
```

```
In[*]:= SP`toRankAndGenSet["-bac+acb"]
```

... General: -bac+acb is not a valid perm; it has inconsistent weights.

```
Out[*]=
```

```
{-1, Error}
```

```
In[*]:= SP`toRankAndGenSet["acb-bac"]
```

... General: acb-bac is not a valid perm; it has inconsistent weights.

```
Out[*]=
```

```
{-1, Error}
```

```
In[*]:= SP`toRankAndGenSet["bac-bac"]
```

... General: bac-bac is not a valid perm; it has inconsistent weights.

```
Out[*]=
```

```
{-1, Error}
```

대칭 문자열의 실제 문자값은 중요하지 않다. 단지 그 문자들의 순서로만 순열 표현을 얻는다:

```
In[*]:= SP`stringToImag["bcf"] (* "bcf" is ordered: 'b' < 'c' < 'f' *)
```

```
Out[*]=
```

```
Imag[1, 2, 3]
```

```
In[*]:= SP`toRankAndGenSet["bcf"]
```

```
Out[*]=
```

```
{3, GenSet[]}
```

```
In[*]:= SP`toRankAndGenSet["abc+bcf"] (* It's OK! *)
```

```
Out[*]=
```

```
{3, GenSet[]}
```

```
In[*]:= SP`toRankAndGenSet["aa"]
```

General: +aa is not a valid perm.

```
Out[*]=
```

```
{-1, Error}
```

```
In[*]:= SP`toRankAndGenSet["-ab"]
```

General: -ab is not a valid signed perm.

```
Out[*]=
```

```
{-1, Error}
```

```
In[*]:= SP`toRankAndGenSet["ac"] (* OK: 'a' < 'c' *)
```

```
Out[*]=
```

```
{2, GenSet[]}
```

checkSymKindDnup

텐서를 **정의할 때** 도입하는 인덱스 대칭은 동일한 Kind 사이에서만 가능하다. 마찬가지로 (위 또는 아래의) 동일한 첨자 사이에서만 가능하다. 이러한 사실을 확인하여 모순이 없으면 0, Kind에 모순이 있으면 -1, 첨자에 모순이 있으면 -2를 돌려 준다.

$F_{ab} = -F_{ba}$ 인 경우:

```
In[*]:= {nRank, GS} = SP`toRankAndGenSet["-ba"]
```

```
Out[*]=
```

```
{2, GenSet[{Cycles[{{1, 2}}], -1]}}
```

```
In[*]:= SP`checkSymKindDnup[GS, nRank, {Latin, Latin}, {-1, -1}]
```

```
Out[*]=
```

```
0
```

$F_{aB} = -F_{Ba}$ 인 경우:

```
In[*]:= SP`checkSymKindDnup[GS, nRank, {Latin, Capital}, {-1, -1}]
```

```
Out[*]=
```

```
-1
```

$F_a^b = -F_a^b$ 인 경우:

```
In[*]:= SP`checkSymKindDnup[GS, nRank, {Latin, Latin}, {-1, +1}]
```

```
Out[*]=
```

```
-2
```

More examples:

```
In[*]:= kindL = {Greek, Greek, Latin, Latin, Latin}; dnupL = {-1, -1, 1, 1, 1};
```

```
In[*]:= sameL = Select[GatherBy[Range@Length[kindL], kindL[[#]] &], Length[#] > 1 &]
```

```
Out[*]=
```

```
{ {1, 2}, {3, 4, 5} }
```

For a permS = "-bacde"

```
In[*]:= {rank, gs} = SP`toRankAndGenSet["-bacde"]
```

```
Out[*]=
```

```
{5, GenSet[{Cycles[{ {1, 2} }], -1}]}
```

```
In[*]:= Orbits[#, gs, rank] & /@ sameL
```

```
Out[*]=
```

```
{{ {1, 2} }, {{3}, {4}, {5} }}
```

```
In[*]:= Union[(Flatten@Orbits[#, gs, rank])] & /@ sameL
```

```
Out[*]=
```

```
{ {1, 2}, {3, 4, 5} }
```

```
In[*]:= (Union[Flatten@Orbits[#, gs, rank]] === Union[#]) & /@ sameL
```

```
Out[*]=
```

```
{True, True}
```

```
In[*]:= SP`checkSymKindDnup[gs, rank, kindL, dnupL] (*consistent*)
```

```
Out[*]=
```

```
0
```

For a permS = "-acbde"

```
In[*]:= {rank, gs} = SP`toRankAndGenSet["-acbde"]
```

```
Out[*]=
```

```
{5, GenSet[{Cycles[{ {2, 3} }], -1}]}
```

```
In[*]:= Orbits[#, gs, rank] & /@ sameL
```

```
Out[*]=
```

```
{{ {1}, {2, 3} }, {{2, 3}, {4}, {5} }}
```

```
In[*]:= Union[(Flatten@Orbits[#, gs, rank])] & /@ sameL
```

```
Out[*]=
```

```
{ {1, 2, 3}, {2, 3, 4, 5} }
```

```
In[*]:= (Union[Flatten@Orbits[#, gs, rank]] === Union[#]) & /@ sameL
```

```
Out[*]=
```

```
{False, False}
```

```
In[*]:= SP`checkSymKindDnup[gs, rank, kindL, dnupL] (*inconsistent*)
```

```
Out[*]=  
-1
```

For a permS = "-abcd"

```
In[*]:= {rank, gs} = SP`toRankAndGenSet["-abcd"]
```

```
Out[*]=  
{5, GenSet[{Cycles[{{4, 5}}], -1]}}
```

```
In[*]:= Orbits[#, gs, rank] & /@ sameL
```

```
Out[*]=  
{{{1}, {2}}, {{3}, {4, 5}}}
```

```
In[*]:= Union[(Flatten@Orbits[#, gs, rank])] & /@ sameL
```

```
Out[*]=  
{{1, 2}, {3, 4, 5}}
```

```
In[*]:= (Union[Flatten@Orbits[#, gs, rank]] == Union[#]) & /@ sameL
```

```
Out[*]=  
{True, True}
```

```
In[*]:= SP`checkSymKindDnup[gs, rank, kindL, dnupL] (*consistent*)
```

```
Out[*]=  
0
```

indexCharSpace

IndexedObject의 첨자 출력을 위해 사용한다.

첨자가 성분인 경우 정수로 표현되고, 값이 10이상이면 그 값에 underline을 표시한다:

```
In[*]:= SP`indexCharSpace[1]
```

```
Out[*]=  
{1, StyleBox[1, ShowContents → False, StripOnInput → False]}
```

```
In[*]:= SP`indexCharSpace[10]
```

```
Out[*]=  
{UnderscriptBox[10, _], StyleBox[11, ShowContents → False, StripOnInput → False]}
```

```
In[*]:= SP`indexCharSpace[100]
```

```
Out[*]=  
{UnderscriptBox[100, _], StyleBox[111, ShowContents → False, StripOnInput → False]}
```

RegularType의 Latin Kind의 인덱스:

```
In[*]:= SP`indexCharSpace[1a]
Out[*]= {a, StyleBox[1, ShowContents → False, StripOnInput → False]}
```

DummyType의 Latin Kind의 인덱스:

```
In[*]:= dummyL = NewDummy[Latin]
SP`indexCharSpace[dummyL[[1]]]
Out[*]= {lLatin$17331, uLatin$17331}
Out[*]= {Latin$17331, StyleBox[1, ShowContents → False, StripOnInput → False]}
```

RegularType의 Bar Kind의 인덱스:

```
In[*]:= SP`indexCharSpace[1ba]
Out[*]= {OverscriptBox[a, _], StyleBox[1, ShowContents → False, StripOnInput → False]}
```

RegularType의 Dot Kind의 인덱스:

```
In[*]:= SP`indexCharSpace[1da]
Out[*]= {OverscriptBox[a, .], StyleBox[1, ShowContents → False, StripOnInput → False]}
```

RegularType의 Hat Kind의 인덱스:

```
In[*]:= SP`indexCharSpace[1ha]
Out[*]= {OverscriptBox[a, ^], StyleBox[1, ShowContents → False, StripOnInput → False]}
```

인덱스가 아닌 경우:

```
In[*]:= SP`indexCharSpace[NonIndex]
Out[*]= {StyleBox[NonIndex, FontColor → ■],
StyleBox[2, ShowContents → False, StripOnInput → False]}
```

symToGenSet

완전 대칭이나 완전 반대칭, 또는 무대칭 표시를 순열군의 Generating Set 표현으로 바꾼다:

```
In[*]:= SP`symToGenSet["Symmetric", 4]
Out[*]= GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{2, 3}}], 1}, {Cycles[{{3, 4}}], 1}]
```

```
In[*]:= SP`symToGenSet["Antisymmetric", 3]
Out[*]:= GenSet[{Cycles[{{1, 2}}], -1}, {Cycles[{{2, 3}}], -1}]
```

```
In[*]:= SP`symToGenSet["Nosymmetric", 5]
Out[*]:= GenSet[]
```

```
In[*]:= SP`symToGenSet[GenSet[], 3]
Out[*]:= GenSet[]
```

DefTensor, Tdefine, and UndefTensor

```
In[*]:= IndexedTensorQ::usage
Out[*]:= IndexedTensorQ[name] returns True if
          'name' is a defined indexed tensor, and False otherwise.
```

```
In[*]:= DefTensor::usage
Out[*]:= DefTensor[tensor[indices], "symmetryString"]
          defines a new tensor with specified index kinds and symmetry.
```

```
In[*]:= Tdefine::usage
Out[*]:= Alias of DefTensor
```

```
In[*]:= UndefTensor::usage
Out[*]:= UndefTensor[tensor] removes all definitions associated with a tensor.
```

defineOperand 명령은 새로운 IndexedOperand를 정의하기 위해 내부적으로 사용한다. 인자로는 IndexedOperand의 이름 oName, 대칭 문자열 permS, Kind 리스트 kindL, dnup 리스트 dnupL, IndexedOperand의 타입을 나타내는 oType이 있다. 옵션으로 출력을 위한 문자열을 지정하기 위한 PrintAs가 있다. 출력을 위한 문자열을 지정하는 PrintAs 옵션이 없으면 IndexedOperand의 이름을 출력의 이름으로 사용한다. IndexedOperand의 rank는 입력한 대칭 문자열로부터 자동적으로 계산된다:

```
defineOperand[oName, permS, kindL, dnupL, oType, PrintAs -> "prtStr"]
```

새로운 텐서를 정의하려면 DefTensor 또는 Tdefine 명령을 사용한다. Tdefine은 DefTensor의 별칭이다. IndexedOperand의 이름과 대칭 문자열만이 인자로 있을 경우:

```
Tdefine[oName_Symbol, permS_String, PrintAs -> "prtStr"]
```

옵션으로 Kind를 입력한다. 기본적으로는 DefaultKind이다. 이 함수에 의해 정의된 텐서는 질의 함수 IndexedTensorQ를 만족한다. 또한, dnup 리스트는 모두 {-1}로 설정된다. 따라서 이 방법으로 텐서를 정의하면 그 텐서는 아래 첨자만 갖는 것으로 정의된다. IndexedOperand를 정의할 때 입력한 Kind 리스트가 그 IndexedOperand의 Kind로 정의됨을 기억하자. (예: Kind of guage field $F_{\mu\nu}^a$ is {Greek, Greek, Latin}.)

계량 텐서가 있는 경우는 아래 첨자를 위 첨자로 변경시킬 수 있으므로 아래 첨자만을 갖는 텐서를 정의한다고 해서 문제가 생기지는 않는다. 그러나 (계량 텐서의 존재 여부와 무관하게) Gauge 이론에서의 텐서와 같이 다양한 Kind와 특정한 위, 아래 첨자를 갖도록 텐서를 정의하려면 아래의 방법을 사용한다:

```
Tdefine[oName_[shapes], permS_String, PrintAs -> "prtStr"]
```

여기서 인덱스의 shape은 Kind와 첨자의 위치(위 또는 아래)로 결정된다. 예를 들면, 'la'는 (Latin Kind, 아래 인덱스)이고, 'uA'는 (Capital Kind, 위 인덱스)이다.

zero rank

이름이 s, 출력 문자열이 s, rank 0인 텐서를 정의한다:

```
In[*]:= DefTensor[s[]];
Tdefine[s[]];
Tdefine[s, ""];
Tdefine[s];
SP`defineOperand[s, "", {Latin}, {}, IndexedTensorQ];
s[]

Out[*]=
s
```

이름이 s, 출력 문자열이 f, rank 0인 텐서를 정의한다:

```
In[*]:= DefTensor[s[], PrintAs -> "f"];
Tdefine[s[], PrintAs -> "f"];
Tdefine[s, "", PrintAs -> "f"];
Tdefine[s, PrintAs -> "f"];
SP`defineOperand[s, "", {Latin}, {}, IndexedTensorQ, PrintAs -> "f"];
s[]

Out[*]=
f
```

In[]:=

? s

Out[]:=

Symbol
Global's
UpValue Definitions
mGRG`STensor`Private`getDnupL[s] ^= {}
mGRG`STensor`Private`getGenSet[s] ^= GenSet[]
mGRG`STensor`Private`getKindL[s] ^= {Latin}
mGRG`STensor`Private`getPrtStr[s] ^= f
mGRG`STensor`Private`getRank[s] ^= 0
mGRG`STensor`Private`getType[s] ^= IndexedTensorQ
IndexedObjectQ[s] ^= True
IndexedOperandQ[s] ^= True
Format Definitions
s /: MakeBoxes[s[mGRG`STensor`Private`args\$____], StandardForm] := mGRG`STensor`Pri
Full Name Global's
^

스칼라의 Kind는 지정하지 않으면 DefaultKind이다. 만일 다른 Kind의 스칼라가 필요하다면 그 스칼라를 정의할 때 Kind를 옵션으로 지정할 수 있다.

이름이 λ , 출력 문자열도 λ , rank 0, Kind가 Greek인 텐서를 정의한다:

In[]:=

```
DefTensor[ $\lambda$ [], Greek];
Tdefine[ $\lambda$ [], Greek];
Tdefine[ $\lambda$ , Greek];
SP`defineOperand[ $\lambda$ , "", {Greek}, {}, IndexedTensorQ];
 $\lambda$ []
```

Out[]:=

 λ

In[]:=

?λ

Out[]:=

Symbol
Global`λ
UpValue Definitions
mGRG`STensor`Private`getDnupL[λ] ^= { }
mGRG`STensor`Private`getGenSet[λ] ^= GenSet[]
mGRG`STensor`Private`getKindL[λ] ^= {Greek}
mGRG`STensor`Private`getPrtStr[λ] ^= λ
mGRG`STensor`Private`getRank[λ] ^= 0
mGRG`STensor`Private`getType[λ] ^= IndexedTensorQ
IndexedObjectQ[λ] ^= True
IndexedOperandQ[λ] ^= True
Format Definitions
λ /: MakeBoxes[λ[mGRG`STensor`Private`args\$____], StandardForm] := mGRG`STensor`Pri
Full Name Global`λ
^

이름이 Lambda, 출력 문자열이 Λ , rank 0, Kind가 Greek인 텐서를 정의한다:

In[]:=

```
DefTensor[Lambda[], Greek, PrintAs → "Λ"];
Tdefine[Lambda[], Greek, PrintAs → "Λ"];
Tdefine[Lambda, Greek, PrintAs → "Λ"];
SP`defineOperand[Lambda, "", {Greek}, {}, IndexedTensorQ, PrintAs → "Λ"];
Lambda[]
```

Out[]:=

Λ

In[]:=

? Lambda

Out[]:=

Symbol

Global`Lambda

UpValue Definitions

mGRG`STensor`Private`getDnupL[Lambda] ^= {}

mGRG`STensor`Private`getGenSet[Lambda] ^= GenSet[]

mGRG`STensor`Private`getKindL[Lambda] ^= {Greek}

mGRG`STensor`Private`getPrtStr[Lambda] ^= Δ

mGRG`STensor`Private`getRank[Lambda] ^= 0

mGRG`STensor`Private`getType[Lambda] ^= IndexedTensorQ

IndexedObjectQ[Lambda] ^= True

IndexedOperandQ[Lambda] ^= True

Format Definitions

Lambda /: MakeBoxes[Lambda[mGRG`STensor`Private`args\$\_\_\_\_], StandardForm] := mGRG`S

Full Name Global`Lambda

^

이름이 scaA, 출력 문자열이 A, rank 0, Kind가 Capital인 텐서를 정의한다:

In[]:=

```
DefTensor[scaA[], Capital, PrintAs -> "A"];
DefTensor[scaA, Capital, PrintAs -> "A"];
Tdefine[scaA, Capital, PrintAs -> "A"];
SP`defineOperand[scaA, "", {Capital}, {}, IndexedTensorQ, PrintAs -> "A"];
scaA[]
```

Out[]:=

A

In[*]:=

? scaA

Out[*]=

Symbol

Global`scaA

UpValue Definitions

mGRG`STensor`Private`getDnupL[scaA] ^= {}

mGRG`STensor`Private`getGenSet[scaA] ^= GenSet[]

mGRG`STensor`Private`getKindL[scaA] ^= {Capital}

mGRG`STensor`Private`getPrtStr[scaA] ^= A

mGRG`STensor`Private`getRank[scaA] ^= 0

mGRG`STensor`Private`getType[scaA] ^= IndexedTensorQ

IndexedObjectQ[scaA] ^= True

IndexedOperandQ[scaA] ^= True

Format Definitions

scaA /: MakeBoxes[scaA[mGRG`STensor`Private`args\$\_\_\_\_], StandardForm] := mGRG`STens

Full Name Global`scaA

^

finite rank and all the same index shape

이름이 tens, 출력 문자열이 tens, rank 5, 인덱스 대칭이 없고, 인덱스 모두 (DefaultKind, 아래 첨자)인 텐서를 정의한다:

In[*]:=

```
DefTensor[tens, 5];
Tdefine[tens, 5];
SP`defineOperand[tens, "5", {DefaultKind}, {-1}, IndexedTensorQ];
tens[la, ub, lc, ld, ue]
```

Out[*]=

$$\text{tens}_{a \quad c d}^{b \quad e}$$

이름이 tens, 출력 문자열이 T, rank 3, 인덱스 대칭이 없고, 인덱스 모두 (DefaultKind, 아래 첨자)인 텐서를 정의한다:

```
In[*]:= DefTensor[tens, 3, PrintAs → "T"];
Tdefine[tens, 3, PrintAs → "T"];
SP`defineOperand[tens, "3", {DefaultKind}, {-1}, IndexedTensorQ, PrintAs → "T"];
tens[la, ub, lc]
```

Out[*]=

$$T_{a \ c}^b$$

이름이 tens, 출력 문자열이 tens, rank 4, 완전 대칭 인덱스, 인덱스 모두 (DefaultKind, 아래 첨자)인 텐서를 정의한다:

```
In[*]:= DefTensor[tens, "4+"];
Tdefine[tens, "4+"];
SP`defineOperand[tens, "4+", {DefaultKind}, {-1}, IndexedTensorQ];
tens[la, lb, uc, ld]
```

Out[*]=

$$tens_{ab \ d}^c$$

이름이 tens, 출력 문자열이 T, rank 3, 완전 반대칭 인덱스, 인덱스 모두 (DefaultKind, 아래 첨자)인 텐서를 정의한다:

```
In[*]:= DefTensor[tens, "3-", PrintAs → "T"];
Tdefine[tens, "3-", PrintAs → "T"];
SP`defineOperand[tens, "3-", {DefaultKind}, {-1}, IndexedTensorQ, PrintAs → "T"];
tens[la, lb, lc]
```

Out[*]=

$$T_{abc}$$

In[*]:=

? tens

Out[*]=

Symbol
Global` tens
UpValue Definitions
<code>mGRG`STensor`Private`getDnupL[tens] ^= {-1}</code>
<code>mGRG`STensor`Private`getGenSet[tens] ^= Antisymmetric</code>
<code>mGRG`STensor`Private`getKindL[tens] ^= {Latin}</code>
<code>mGRG`STensor`Private`getPrtStr[tens] ^= T</code>
<code>mGRG`STensor`Private`getRank[tens] ^= 3</code>
<code>mGRG`STensor`Private`getType[tens] ^= IndexedTensorQ</code>
<code>IndexedObjectQ[tens] ^= True</code>
<code>IndexedOperandQ[tens] ^= True</code>
Format Definitions
<code>tens /: MakeBoxes[tens[mGRG`STensor`Private`args\$____], StandardForm] := mGRG`STens</code>
Full Name Global` tens
^

이름이 **tens**, 출력 문자열이 **T**, rank 4, 완전 반대칭 인덱스, 인덱스 모두 (Capital Kind, 아래 첨자)인 텐서를 정의한다:

In[*]:=

```
DefTensor[tens, "4-", Capital, PrintAs -> "T"];
Tdefine[tens, "4-", Capital, PrintAs -> "T"];
SP`defineOperand[tens, "4-", {Capital}, {-1}, PrintAs -> "T"];
tens[1A, 1B, 1C, 1D]
```

Out[*]=

$$T_{ABCD}$$

In[]:=

? tens

Out[]:=

Symbol

Global`tens

UpValue Definitions

mGRG`STensor`Private`getDnupL[tens] ^= {-1}

mGRG`STensor`Private`getGenSet[tens] ^= Antisymmetric

mGRG`STensor`Private`getKindL[tens] ^= {Capital}

mGRG`STensor`Private`getPrtStr[tens] ^= tens

mGRG`STensor`Private`getRank[tens] ^= 4

mGRG`STensor`Private`getType[tens] ^= PrintAs → T

IndexedObjectQ[tens] ^= True

IndexedOperandQ[tens] ^= True

Format Definitions

tens /: MakeBoxes[tens[mGRG`STensor`Private`args\$\_\_\_\_], StandardForm] := mGRG`STens

Full Name Global`tens

^

arbitrary rank and all the same index shape

이름이 tens, 출력 문자열이 tens, 임의의 rank, 인덱스 대칭이 없고, 인덱스 모두 (DefaultKind, 아래 첨자)인 텐서를 정의한다:

In[]:=

```
DefTensor[tens, "*"];
Tdefine[tens, "*"];
SP`defineOperand[tens, "*", {DefaultKind}, {-1}, IndexedTensorQ];
{tens[], tens[1a], tens[1a, ub], tens[1a, ub, 1c]}
```

Out[]:=

$$\{tens, tens_a, tens_a^b, tens_a^b_c\}$$

In[*]:=

? tens

Out[*]=

Symbol
Global` tens
UpValue Definitions
<code>mGRG`STensor`Private`getDnupL[tens] ^= {-1}</code>
<code>mGRG`STensor`Private`getGenSet[tens] ^= Nosymmetric</code>
<code>mGRG`STensor`Private`getKindL[tens] ^= {Latin}</code>
<code>mGRG`STensor`Private`getPrtStr[tens] ^= tens</code>
<code>mGRG`STensor`Private`getRank[tens] ^= -1</code>
<code>mGRG`STensor`Private`getType[tens] ^= IndexedTensorQ</code>
<code>IndexedObjectQ[tens] ^= True</code>
<code>IndexedOperandQ[tens] ^= True</code>
Format Definitions
<code>tens /: MakeBoxes[tens[mGRG`STensor`Private`args\$____], StandardForm] := mGRG`STens</code>
Full Name Global` tens
^

이름이 **tens**, 출력 문자열이 **T**, 임의의 rank, 완전 대칭 인덱스, 인덱스 모두 (DefaultKind, 아래 첨자)인 텐서를 정의한다:

In[*]:=

```
DefTensor[tens, "+", PrintAs -> "T"];
Tdefine[tens, "+", PrintAs -> "T"];
SP`defineOperand[tens, "+", {Latin}, {-1}, PrintAs -> "T"];
{tens[], tens[la], tens[la, ub], tens[la, ub, lc], tens[la, lb, uc, ld]}
```

Out[*]=

$$\{T, T_a, T_a^b, T_{a\ c}^b, T_{ab\ d}^c\}$$

In[*]:=

? tens

Out[*]=

Symbol

Global`**tens**

UpValue Definitions

mGRG`STensor`Private`getDnupL**[**tens**] ^= {-1}**mGRG`STensor`Private`getGenSet[**tens**] ^= **Symmetric****mGRG`STensor`Private`getKindL**[**tens**] ^= {**Latin**}**mGRG`STensor`Private`getPrtStr**[**tens**] ^= **tens****mGRG`STensor`Private`getRank**[**tens**] ^= -1**mGRG`STensor`Private`getType**[**tens**] ^= **PrintAs** → **T****IndexedObjectQ**[**tens**] ^= **True****IndexedOperandQ**[**tens**] ^= **True**

Format Definitions

tens /: **MakeBoxes**[**tens**[**mGRG`STensor`Private`args\$\_\_\_\_**], **StandardForm**] := **mGRG`STensor`Private`makeBoxes**[**tens**, **args**]Full Name Global`**tens**

^

이름이 **tens**, 출력 문자열이 **T**, 임의의 rank, 완전 반대칭 인덱스, 인덱스 모두 (DefaultKind, 아래 첨자)인 텐서를 정의한다:

In[*]:=

```
DefTensor[tens, "*-", PrintAs → "T"];
Tdefine[tens, "*-", PrintAs → "T"];
SP`defineOperand[tens, "*-", {Latin}, {-1}, PrintAs → "T"];
{tens[], tens[la], tens[la, ub], tens[la, ub, lc], tens[la, lb, uc, ld] }
```

Out[*]=

$$\{T, T_a, T_a^b, T_{a\ c}^b, T_{ab\ d}^c\}$$

In[*]:=

? tens

Out[*]=

Symbol
Global` tens
UpValue Definitions
<code>mGRG`STensor`Private`getDnupL[tens] ^= {-1}</code>
<code>mGRG`STensor`Private`getGenSet[tens] ^= Antisymmetric</code>
<code>mGRG`STensor`Private`getKindL[tens] ^= {Latin}</code>
<code>mGRG`STensor`Private`getPrtStr[tens] ^= tens</code>
<code>mGRG`STensor`Private`getRank[tens] ^= -1</code>
<code>mGRG`STensor`Private`getType[tens] ^= PrintAs → T</code>
<code>IndexedObjectQ[tens] ^= True</code>
<code>IndexedOperandQ[tens] ^= True</code>
Format Definitions
<code>tens /: MakeBoxes[tens[mGRG`STensor`Private`args\$____], StandardForm] := mGRG`STens</code>
Full Name Global` tens
^

이름이 **tens**, 출력 문자열이 **T**, 임의의 rank, 완전 반대칭 인덱스, 인덱스 모두 (Capital Kind, 위 첨자)인 텐서를 정의한다:

In[*]:=

```
SP`defineOperand[tens, "*-", {Capital}, {+1}, PrintAs → "T"];
{tens[], tens[uA], tens[lA, uB], tens[lA, uB, uC], tens[lA, uB, uC, lD]}
```

Out[*]=

$$\{T, T^A, T_A^B, T_A^{BC}, T_A^{BC}_D\}$$

In[*]:=

? tens

Out[*]=

Symbol

Global`*tens*

UpValue Definitions

*mGRG`STensor`Private`getDnupL[tens] ^= {1}**mGRG`STensor`Private`getGenSet[tens] ^= Antisymmetric**mGRG`STensor`Private`getKindL[tens] ^= {Capital}**mGRG`STensor`Private`getPrtStr[tens] ^= tens**mGRG`STensor`Private`getRank[tens] ^= -1**mGRG`STensor`Private`getType[tens] ^= PrintAs → T**IndexedObjectQ[tens] ^= True**IndexedOperandQ[tens] ^= True*

Format Definitions

tens */:* *MakeBoxes[tens[mGRG`STensor`Private`args\$\_\_\_\_], StandardForm] := mGRG`STens*Full Name Global`*tens*

^

finite rank and various index shapes

이름이 *tens*, 출력 문자열이 *tens*, rank 2, 첫 번째 인덱스가 (Latin Kind, 아래 첨자)이고, 두 번째 인덱스가 (Capital Kind, 위 첨자)인 텐서를 정의한다:

In[*]:=

```
DefTensor[tens[la, uA]];
Tdefine[tens[la, uA]];
SP`defineOperand[tens, "ab", {Latin, Capital}, {-1, +1}, IndexedTensorQ];
tens[la, uB]
```

Out[*]=

 tens_a^B

이름이 *tens*, 출력 문자열이 *T*, rank 3, 첫 번째와 두 번째 인덱스가 반대칭, 첫 번째와 두 번째 인덱스가 (Latin Kind, 아래 첨자)이고, 세 번째 인덱스가 (Capital Kind, 위 첨자)인 텐서를 정의한다:

```
In[*]:= DefTensor[tens[la, lb, uA], "-bac", PrintAs -> "T"];
Tdefine[tens[la, lb, uA], "-bac", PrintAs -> "T"];
SP`defineOperand[tens, "-bac",
  {Latin, Latin, Capital}, {-1, -1, +1}, PrintAs -> "T"];
tens[la, lb, uC]
```

Out[*]=

$$T_{ab}^c$$

양-밀스 게이지 이론을 위해 새로운 인덱스 Kind를 정의하고, 기본 Kind로 설정하자:

```
In[*]:= UndefKind[Greek];
DefKind[Greek, Alphabet["Greek"]];
```

임시적인 방법으로 DefaultKind를 변경하는 방법 (나중에 올바른 방법을 설명할 것임):

```
In[*]:= Unprotect[DefaultKind]; DefaultKind = Greek; Protect[DefaultKind];
```

양-밀스 게이지장 A_μ^a 와 $F_{\mu\nu}^a$, 구조 상수 ϵ_{abc} 를 정의하자:

```
In[*]:= Tdefine[YMA[lμ, ua], PrintAs -> "A"];
YMA[lμ, ua]
```

Out[*]=

$$A_\mu^a$$

```
In[*]:= Tdefine[YMF[lμ, lν, ua], "-bac", PrintAs -> "F"];
YMF[lμ, lν, ua]
```

Out[*]=

$$F_{\mu\nu}^a$$

```
In[*]:= Tdefine[YMeps[la, lb, lc], "3-", PrintAs -> "ε"];
YMeps[la, lb, lc]
```

Out[*]=

$$\epsilon_{abc}$$

In[]:=

? YMeps

Out[]:=

Symbol

Global`YMeps

UpValue Definitions

mGRG`STensor`Private`getDnupL[YMeps] ^= {-1}

mGRG`STensor`Private`getGenSet[YMeps] ^= Antisymmetric

mGRG`STensor`Private`getKindL[YMeps] ^= {Latin}

mGRG`STensor`Private`getPrtStr[YMeps] ^= ∈

mGRG`STensor`Private`getRank[YMeps] ^= 3

mGRG`STensor`Private`getType[YMeps] ^= IndexedTensorQ

IndexedObjectQ[YMeps] ^= True

IndexedOperandQ[YMeps] ^= True

Format Definitions

YMeps /: MakeBoxes[YMeps[mGRG`STensor`Private`args\$\_\_\_\_], StandardForm] := mGRG`STe

Full Name Global`YMeps

^

기본 Kind인 Latin으로 복귀하자:

In[]:=

Unprotect[DefaultKind]; DefaultKind = Latin; Protect[DefaultKind];

More examples

Rank 2인 DefaultKind의 대칭 텐서를 정의한다:

In[]:=

Tdefine[R[1a, 1b], "ba"]; R[1a, 1b]

Out[]:=

 R_{ab}

출력을 위한 문자열은 출력할 때만 사용된다.

In[]:=

Tdefine[affineG[1a, 1b, uc], "bac", PrintAs → "Γ"]; affineG[1b, 1a, uc]

Out[]:=

 Γ_{ba}^c

반대칭 텐서를 정의한다:

```
In[*]:= Tdefine[F[la, lb], "-ba"]; {F[la, ub], F[-1, 1], F[-1, -1], F[la, ua]}
```

```
Out[*]=
```

$$\{F_a^b, F_1^1, F_{11}, F_a^a\}$$

```
In[*]:= Tdefine[cuvR[la, lb, lc, ud], "-bacd", PrintAs -> "R"]; cuvR[la, lb, lc, ud]
```

```
Out[*]=
```

$$R_{abc}^d$$

인덱스 대칭이 없는 경우 대칭 문자열 대신 rank에 해당하는 숫자를 입력할 수 있다:

```
In[*]:= Tdefine[T, 10]; (* Tdefine[T,"abcdefghij"] *)
```

완전 대칭 또는 반대칭인 경우 대칭 문자열 대신 rank와 + 또는 -를 조합한 문자열을 입력한다:

```
In[*]:= Tdefine[T, "10+"] (* rank 10 and totally symmetric *)
```

```
In[*]:= Tdefine[T, "10-"] (* rank 10 and totally anti-symmetric *)
```

임의의 rank를 갖고 무대칭, 완전 대칭, 완전 반대칭인 텐서를 각각 정의할 수 있다:

```
In[*]:= Tdefine[T, "*"]; {T[la], T[la, ub], T[la, lb, lc, ld]}
```

```
Out[*]=
```

$$\{T_a, T_a^b, T_{abcd}\}$$

```
In[*]:= Tdefine[S, "**"]; {S[la], S[la, ub], S[ld, la, lb, lc]}
```

```
Out[*]=
```

$$\{S_a, S_a^b, S_{dabc}\}$$

```
In[*]:= Tdefine[Eps, "*-", PrintAs -> "ε"]; {Eps[la, lb], Eps[ld, la, lb, lc]}
```

```
Out[*]=
```

$$\{\epsilon_{ab}, \epsilon_{dabc}\}$$

인덱스의 shape은 Kind와 위/아래 첨자로 결정된다. 텐서를 정의할 때 각각의 인덱스에 대해서 다른 shape을 지정할 수 있다. 예를 들면 두 개의 (Greek Kind, 아래 첨자)인 인덱스와 마지막의 (Latin Kind, 위 첨자)인 인덱스로 구성된 양-밀스 게이지장 텐서를 정의할 수 있다:

```
In[*]:= Tdefine[F[lμ, lν, ua], "-bac"]; F[lμ, lν, ua]
```

```
Out[*]=
```

$$F_{\mu\nu}^a$$

```
In[*]:= Tdefine[F[lμ, lν, ua], "-bac", PrintAs -> "ℱ"]; F[lμ, lν, ua]
```

```
Out[*]=
```

$$\mathcal{F}_{\mu\nu}^a$$

모든 인덱스들이 동일한 shape이면 한 개의 shape만 입력한다.

Rank 3이고 Greek Kind인 텐서 Eps를 정의한다:

```
In[*]:= Tdefine[Eps[lμ], 3, PrintAs → "ε"]; Eps[lμ, lν, lρ]
```

```
Out[*]=
```

$$\epsilon_{\mu\nu\rho}$$

임의의 rank를 갖는 텐서는 모든 인덱스가 같은 shape을 갖는다고 가정한다.

```
In[*]:= Tdefine[Eps[uμ], "*-", PrintAs → "ε"]; {Eps[uμ, uν], Eps[uμ, uν, uρ, uσ]}
```

```
Out[*]=
```

$$\{\epsilon^{\mu\nu}, \epsilon^{\mu\nu\rho\sigma}\}$$

Pretty printing indices

위/아래 첨자 출력:

```
In[*]:= Tdefine[R[la, lb], "ba"]; R[la, ub]
```

```
Out[*]=
```

$$R_a^b$$

```
In[*]:= R_a^b // FullForm (* copy of a previous output *)
```

```
Out[*]//FullForm=
```

$$R[la, ub]$$

여러 가지 Kind의 혼합:

```
In[*]:= Tdefine[T, "*"]
```

```
In[*]:= T[la, ua, lμ, uμ, lA, uA]
```

```
Out[*]=
```

$$T_a^{\mu A}$$

Bar, Dot, Hat 인덱스의 출력:

```
In[*]:= T[lba, uba, lda, uda, lha, uha]
```

```
Out[*]=
```

$$T_{\bar{a}}^{\hat{a} \cdot \hat{a}}$$

10 이상의 성분 인덱스 출력:

```
In[*]:= T[-1, 42, -2, 3, 567, -8]
```

```
Out[*]=
```

$$T_1^{\frac{42}{2} \frac{3567}{8}}$$

적합한 인덱스가 아닐 때의 출력:

```
In[*]:= T[labcd, ua, uabcd, la, ua]
```

```
Out[*]=
```

```
Ta alabcd uabcda
```

Check

텐서의 이름이 인덱스 이름과 같아서는 안된다.

```
In[*]:= SP`checkName[ll]
```

```
Msg: ll is used as an index name!
```

```
Out[*]=
```

```
False
```

```
In[*]:= Tdefine[ll, "a"]
```

```
Msg: ll is used as an index name!
```

```
Out[*]=
```

```
$Failed
```

텐서의 이름이 시스템에서 사용하는 이름과 같아서는 안된다.

```
In[*]:= Tdefine[C, "a"]
```

```
Msg: C is reserved!
```

```
Out[*]=
```

```
$Failed
```

대칭 문자열이 잘못된 경우 그 텐서는 정의되지 않는다:

```
In[*]:= SP`stringToImag["-ab"]
```

```
General: -ab is not a valid signed perm.
```

```
In[*]:= Tdefine[errorT, "-ab"]
```

```
General: -ab is not a valid signed perm.
```

```
Out[*]=
```

```
$Failed
```

```
In[*]:= ?errorT
```

```
Out[*]=
```

```
Missing[UnknownSymbol, errorT]
```

```
In[*]:= SP`checkSymKindDnup[GenSet[{Cycles[{1, 2}], -1}],  
3, {Latin, Latin, Latin}, {-1, 1, -1}]
```

```
Out[*]=
```

```
-2
```

```
In[*]:= Tdefine[errorT[1a, ub, 1c], "-bac"] (* from checkSymKindDnup[...] *)
```

Msg: incompatible between -bac and {-1, 1, -1}

Out[*]=

\$Failed

```
In[*]:= Tdefine[errorT[1a, 1A, 1c], "-bac"]
```

Msg: incompatible between -bac and {Latin, Capital, Latin}

Out[*]=

\$Failed

```
In[*]:= Tdefine[errorT[1a, 1b, uc], "-cba"]
```

Msg: incompatible between -cba and {-1, -1, 1}

Out[*]=

\$Failed

```
In[*]:= Tdefine[errorT[1a, 1b, 1A], "-cba"]
```

Msg: incompatible between -cba and {Latin, Latin, Capital}

Out[*]=

\$Failed

인덱스의 개수는 텐서를 정의할 때의 rank와 같아야 한다:

```
In[*]:= Tdefine[f];
f[1a] // SyntaxCheck
```

Msg: invalid number of indices for f: {1a}

Out[*]=

f_a

```
In[*]:= Tdefine[T, "a"];
T[] // SyntaxCheck
```

Msg: invalid number of indices for T

Out[*]=

T

```
In[*]:= T[1a, 1b] // SyntaxCheck
```

Msg: invalid number of indices for T: {1a, 1b}

Out[*]=

T_{ab}

UndefTensor

DefTensor 또는 Tdefine으로 정의한 텐서를 취소한다:

In[*]:= **DefTensor**[**tens**[**la**, **uA**], 2, **PrintAs** → "T"]; **tens**[**la**, **uB**]

Out[*]=

T_a^B

In[*]:= **? tens**

Out[*]=

Symbol

Global`**tens**

UpValue Definitions

mGRG`STensor`Private`getDnupL[**tens**] ^= {-1, 1}

mGRG`STensor`Private`getGenSet[**tens**] ^= **Nosymmetric**

mGRG`STensor`Private`getKindL[**tens**] ^= {**Latin**, **Capital**}

mGRG`STensor`Private`getPrtStr[**tens**] ^= **T**

mGRG`STensor`Private`getRank[**tens**] ^= 2

mGRG`STensor`Private`getType[**tens**] ^= **IndexedTensorQ**

IndexedObjectQ[**tens**] ^= **True**

IndexedOperandQ[**tens**] ^= **True**

Format Definitions

tens /: **MakeBoxes**[**tens**[**mGRG`STensor`Private`args\$\_\_\_\_**], **StandardForm**] := **mGRG`STens**

Full Name Global`**tens**

^

In[*]:= **UndefTensor**[**tens**]; **tens**[**la**, **uB**]

Out[*]=

tens[**la**, **uB**]

In[*]:= **? tens**

Out[*]=

Symbol

Global`**tens**

Full Name Global`**tens**

^

텐서 정의를 취소한 후에 다시 같은 이름의 텐서가 필요하면 다시 정의한다:

```
In[*]:= DefTensor[tens[1a, 1b], 2, PrintAs → "againT"]; tens[1a, 1b]
```

```
Out[*]= againTab
```

```
In[*]:= UndefTensor[tens]
```

DnupAt

```
In[*]:= DnupAt::usage
```

```
Out[*]= DnupAt[name, pos] returns the up/down state (-1 for down, +1 for
up) of the index at position 'pos' for the specified indexed object.
```

텐서를 정의할 때 각 인덱스의 위/아래 첨자 값(+1/-1)은 결정된다. DnupAt 함수는 두 번째 인자의 위치에 있는 인덱스의 위/아래 첨자 값을 돌려 준다. (만일 두 번째 인자의 값이 텐서의 rank보다 크면 마지막 인덱스로 간주한다.)

```
In[*]:= Tdefine[F[1μ, 1ν, ua], "-bac", PrintAs → "ℱ"]; F[1μ, 1ν, ua]
```

```
Out[*]= ℱμνa
```

```
In[*]:= {DnupAt[F, 1], DnupAt[F, 2], DnupAt[F, 3], DnupAt[F, 5]}
```

```
Out[*]= {-1, -1, 1, 1}
```

```
In[*]:= DnupAt[F, #] & /@ Range[GetRank[F]]
```

```
Out[*]= {-1, -1, 1}
```

GetRank, get*

```
In[*]:= GetRank::usage
```

```
Out[*]= GetRank[tensor] returns the rank of a defined tensor.
```

```
In[*]:= Tdefine[F[1μ, 1ν, ua], "-bac", PrintAs → "ℱ"]; GetRank[F]
```

```
Out[*]= 3
```

```
In[*]:= Through[
  {SP`getType, SP`getDnupL, SP`getGenSet, SP`getKindL, SP`getPrtStr, SP`getRank}[F]]
Out[*]=
{IndexedTensorQ, {-1, -1, 1},
  GenSet[{Cycles[{{1, 2}}], -1}], {Greek, Greek, Latin},  $\mathcal{F}$ , 3}
```

AllPermutations, GStoString

```
In[*]:= AllPermutations::usage
Out[*]=
AllPermutations[permS] generates a string representing all
permutations and their weights implied by the symmetry string
permS. It is useful for visualizing the full symmetry of a tensor.
```

```
In[*]:= GStoString::usage
Out[*]=
GStoString[gs, len] converts a generator set gs into
its string representation for a tensor of rank len. If len
is omitted, the maximum rank from the generator set is used.
```

함수 toRankAndGenSet은 대칭 문자열로부터 rank와 순열군의 Generating Set을 돌려 준다:

```
In[*]:= {rank, GS} = SP`toRankAndGenSet["-bacd-abdc+cdab"]
Out[*]=
{4, GenSet[{Cycles[{{1, 2}}], -1},
  {Cycles[{{3, 4}}], -1}, {Cycles[{{1, 3}, {2, 4}}], 1}]}
```

반대로, rank와 순열군의 Generating Set으로부터 대칭 문자열을 얻는다:

```
In[*]:= List@@(SP`toOnePermWeightStr[#, rank] & /@ GS)
Out[*]=
{-bacd, -abdc, +cdab}
```

```
In[*]:= SP`toPermWeightStr[List@@GS, rank]
Out[*]=
-bacd-abdc+cdab
```

```
In[*]:= GStoString[GS, rank]
Out[*]=
-bacd-abdc+cdab
```

AllPermutations 함수는 주어진 대칭 문자열로부터 (순열군의 원소가 되는) 모든 가능한 대칭 문자열을 돌려 준다.

```
In[*]:= {rank, GS} = SP`toRankAndGenSet["-bacd-abdc+cdab"]
Out[*]= {4, GenSet[{Cycles[{{1, 2}}], -1},
  {Cycles[{{3, 4}}], -1}, {Cycles[{{1, 3}, {2, 4}}], 1}]}
```

```
In[*]:= MakePermGroup::usage
Out[*]= MakePermGroup[gs, len] constructs a PermutationGroup object from a GenSet
  'gs' acting on 'len' elements. It returns all elements of the group.
```

```
In[*]:= MakePermGroup[GS, rank]
Out[*]= {{Cycles[{}], 1}, {Cycles[{{3, 4}}], -1},
  {Cycles[{{1, 2}}], -1}, {Cycles[{{1, 2}, {3, 4}}], 1},
  {Cycles[{{1, 3}, {2, 4}}], 1}, {Cycles[{{1, 3, 2, 4}}], -1},
  {Cycles[{{1, 4, 2, 3}}], -1}, {Cycles[{{1, 4}, {2, 3}}], 1}}
```

```
In[*]:= SP`toPermWeightStr[%, rank]
Out[*]= +abcd-abdc-bacd+badc+cdab-cdba-dcab+dcba
```

```
In[*]:= % == AllPermutations["-bacd-abdc+cdab"]
Out[*]= True
```

```
In[*]:= AllPermutations[""] // FullForm
Out[*]//FullForm= ""
```

```
In[*]:= AllPermutations["c"]
Out[*]= a
```

```
In[*]:= AllPermutations["cb"]
Out[*]= +ab+ba
```

```
In[*]:= AllPermutations["agb"]
Out[*]= +abc+acb
```

GetSymmetry, SetSymmetry, getGenSetOf

In[*]:= **GetSymmetry::usage**

Out[*]=

GetSymmetry[tensor] returns the symmetry generator set of a defined tensor.

In[*]:= **SetSymmetry::usage**

Out[*]=

SetSymmetry[tensor, "symmetryString"]
sets the index symmetry for a defined tensor.

정의된 텐서의 대칭을 돌려주거나 설정한다. 인자는 텐서의 이름이다.

In[*]:= **Tdefine[T, "abc"]; GetSymmetry[T]**

Out[*]=

GenSet[]

In[*]:= **SetSymmetry[T, "acb"]; GetSymmetry[T]**

Out[*]=

GenSet[{Cycles[{2, 3}], 1}]

In[*]:= **Tdefine[Eps, "*"]; GetSymmetry[Eps]**

Out[*]=

Nosymmetric

In[*]:= **SetSymmetry[Eps, "*-"]; GetSymmetry[Eps]**

Out[*]=

Antisymmetric

In[*]:= **SetSymmetry[Eps, "*+"]; GetSymmetry[Eps]**

Out[*]=

Symmetric

getGenSetOf 함수는 정의된 텐서의 대칭을 입력된 인덱스를 기준으로 생성하여 돌려 준다:

In[*]:= **Tdefine[T, "badc"]; GetSymmetry[T]**

Out[*]=

GenSet[{Cycles[{1, 2}, {3, 4}], 1}]

```
In[*]:= SP`getGenSetOf /@ {T[1a], T[1a, 1b], T[1a, 1b, 1c],  
T[1a, 1b, 1c, 1d], T[1a, 1b, 1c, 1d, 1e]} // ColumnForm
```

```
Out[*]=
```

```
GenSet[]  
GenSet[{Cycles[{{1, 2}}], 1}]  
GenSet[{Cycles[{{1, 2}}], 1}]  
GenSet[{Cycles[{{1, 2}, {3, 4}}], 1}]  
GenSet[{Cycles[{{1, 2}, {3, 4}}], 1}]
```

```
In[*]:= Tdefine[T, "4+"]; GetSymmetry[T]
```

```
Out[*]=
```

```
Symmetric
```

```
In[*]:= SP`getGenSetOf /@ {T[1a], T[1a, 1b], T[1a, 1b, 1c],  
T[1a, 1b, 1c, 1d], T[1a, 1b, 1c, 1d, 1e]} // ColumnForm
```

```
Out[*]=
```

```
GenSet[]  
GenSet[{Cycles[{{1, 2}}], 1}]  
GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{2, 3}}], 1}]  
GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{2, 3}}], 1}, {Cycles[{{3, 4}}], 1}]  
GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{2, 3}}], 1}, {Cycles[{{3, 4}}], 1}]
```

```
In[*]:= Tdefine[T, "*+"]; GetSymmetry[T]
```

```
Out[*]=
```

```
Symmetric
```

```
In[*]:= SP`getGenSetOf /@ {T[1a], T[1a, 1b], T[1a, 1b, 1c],  
T[1a, 1b, 1c, 1d], T[1a, 1b, 1c, 1d, 1e]} // ColumnForm
```

```
Out[*]=
```

```
GenSet[]  
GenSet[{Cycles[{{1, 2}}], 1}]  
GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{2, 3}}], 1}]  
GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{2, 3}}], 1}, {Cycles[{{3, 4}}], 1}]  
GenSet[{Cycles[{{1, 2}}], 1}, {Cycles[{{2, 3}}], 1}, {Cycles[{{3, 4}}], 1}, {Cycles[{{
```

```
In[*]:= Tdefine[T, "*"]; GetSymmetry[T]
```

```
Out[*]=
```

```
Nosymmetric
```

```
In[*]:= SP`getGenSetOf /@ {T[1a], T[1a, 1b], T[1a, 1b, 1c],  
T[1a, 1b, 1c, 1d], T[1a, 1b, 1c, 1d, 1e]} // ColumnForm
```

```
Out[*]=
```

```
GenSet[]  
GenSet[]  
GenSet[]  
GenSet[]  
GenSet[]
```

Kind of IndexedObject

```
In[*]:= KindOf::usage
```

```
Out[*]=
```

KindOf[obj, pos] returns the kind of the object 'obj' at a given index position 'pos'. For an indexed object expression like $T[a,b]$, KindOf[T[a,b], a] returns the kind of index 'a'.

Kind of Tensors

텐서의 Kind는 사용하는 인덱스의 Kind로부터 유추한다. 참고로 인덱스가 없는 스칼라의 경우는 지정하지 않으면 DefaultKind이고, 다른 Kind의 스칼라가 필요하다면 그 스칼라를 정의할 때 옵션으로 지정할 수 있다.

```
In[*]:= Tdefine[f[]]; SP`getKindL[f]
```

```
Out[*]=
```

{Latin}

```
In[*]:= Tdefine[someF[], Capital]; SP`getKindL[someF]
```

```
Out[*]=
```

{Capital}

```
In[*]:= Tdefine[F[l $\mu$ , l $\nu$ , ua], "-bac", PrintAs  $\rightarrow$  "ƒ"]; SP`getKindL[F]
```

```
Out[*]=
```

{Greek, Greek, Latin}

인자 없이 텐서의 이름만 사용하면 첫 번째 인덱스의 Kind를 돌려 준다:

```
In[*]:= KindOf[F]
```

```
Out[*]=
```

Greek

두 번째 인자가 숫자이면 그 숫자에 해당하는 번호의 인덱스 Kind를 돌려 준다:

```
In[*]:= {KindOf[F, 1], KindOf[F, 2], KindOf[F, 3]}
```

```
Out[*]=
```

{Greek, Greek, Latin}

만일 두 번째 인자의 숫자 값이 그 텐서의 rank보다 크면 마지막 인덱스의 Kind를 돌려 준다:

```
In[*]:= KindOf[F, 10]
```

```
Out[*]=
```

Latin

첫 번째 인자가 텐서 이름이 아니고 텐서 입력 형태이면 두 번째 인자에 해당하는 인덱스의 Kind를 돌려 준다:

$In[*]:=$	<code>{KindOf[F[1μ, 1ν, ua], 1μ], KindOf[F[1μ, 1ν, ua], 1ν], KindOf[F[1μ, 1ν, ua], ua]}</code>
$Out[*]=$	<code>{Greek, Greek, Latin}</code>

두 번째 인자가 첫 번째 인자에 없으면 그 텐서의 첫 번째 인덱스 Kind를 돌려 준다:

$In[*]:=$	<code>KindOf[F[1μ, 1ν, ua], 1b]</code>
$Out[*]=$	<code>Greek</code>

Kdelta는 모든 Kind에서 사용 가능한 특별한 텐서이다:

$In[*]:=$	<code>{KindOf[Kdelta[1a, ub], 1a], KindOf[Kdelta[1A, uB], uB], KindOf[Kdelta[1a, ub], 1c], KindOf[Kdelta]}</code>
$Out[*]=$	<code>{Latin, Capital, All, All}</code>

$In[*]:=$	<code>{KindOf[Kdelta, 1a], KindOf[Kdelta, 1μ]}</code>
$Out[*]=$	<code>{Latin, Greek}</code>

특별한 텐서 Epsilon, Metricg, Torsion은 항상 DefaultKind이다.

$In[*]:=$	<code>{KindOf[Epsilon[1a, 1b], any], KindOf[Metricg[1A, 1B], 1A], KindOf[Torsion[1μ, 1ν, uρ], 1ν]}</code>
$Out[*]=$	<code>{Latin, Latin, Latin}</code>

$In[*]:=$	<code>{KindOf[Epsilon, any], KindOf[Metricg, any], KindOf[Torsion, any]}</code>
$Out[*]=$	<code>{Latin, Latin, Latin}</code>

Kind of Operators

연산자의 Kind를 돌려 준다. LD Type의 연산자는 첫 번째 인자의 Kind와 동일하고, 나머지 연산자는 정의할 때 설정된 (기본적으로는 DefaultKind) Kind이다. 그리고, BD, CD와같은 특별한 연산자의 Kind는 따로 정한다.

$In[*]:=$	<code>{KindOf[CovD, any], KindOf[CovD]}</code>
$Out[*]=$	<code>{Latin, Latin}</code>

$In[*]:=$	<code>SP`defineOperator[LieD, "L", LD]; Tdefine[v[1a], 1]</code>
-----------	--

```
In[*]:= {KindOf[v], KindOf[LieD, v]}
```

```
Out[*]= {Latin, Latin}
```

```
In[*]:= Tdefine[v[lμ], 1]; {KindOf[v], KindOf[LieD, v]}
```

```
Out[*]= {Greek, Greek}
```

특별한 연산자 CD는 항상 DefaultKind에 속한다:

```
In[*]:= {KindOf[CD, la], KindOf[CD, lμ], KindOf[CD, non]}
```

```
Out[*]= {Latin, NonKind, Latin}
```

또 다른 특별한 연산자 BD는 모든 Kind에서 사용한다. 따라서 BD의 Kind는 사용하는 인자에 따라 결정된다. 보통은 두 번째 인자의 인덱스 Kind로 결정한다.

```
In[*]:= {KindOf[BD, la], KindOf[BD, lμ], KindOf[BD, non]}
```

```
Out[*]= {Latin, Greek, All}
```

Operators

```
In[*]:= Tdefine[f[]]; Tdefine[T, "ba"]; Tdefine[V, 1, PrintAs → "ξ"]
```

defineOperator

새로운 연산자를 도입하는 내부 함수다. 이 함수는 연산자의 출력 모양과 기타 설정을 한다. 그러나 각각의 연산자가 IndexedObject에 작용하는 실제 역할은 따로 정의해야 한다. 인자로써 연산자의 이름, 출력을 위한 문자열, 연산자 Type, 옵션으로 그 연산자가 속할 Kind가 있다. 연산자 Type은 CD, LD, XD, XP가 있다.

CD Type Operator

공변 도함수(Covariant Derivative) 형태의 연산자이다. 연산자의 인자로 하나의 인덱스와 피연산자가 있는 경우이다.

```
In[*]:= SP`defineOperator[CovD, "∇", CD]
```

```
In[*]:= {CovD[-1, T[ua, ub]], CovD[1, T[ua, ub]],  
CovD[la, T[ua, ub]], CovD[ua, T[la, ub]], CovD[ua, nonT[ub, uc]]}
```

```
Out[*]= {∇1Tab, ∇1Tab, ∇aTab, ∇aTab, ∇anonT[ub, uc]}
```

$In[*]:=$ **? CovD** $Out[*]=$

Symbol

Global`CovD

UpValue Definitions

 $\text{mGRG`STensor`Private`getKindL}[\text{CovD}] \wedge = \{\text{Latin}\}$  $\text{mGRG`STensor`Private`getPrtStr}[\text{CovD}] \wedge = \nabla$ $\text{mGRG`STensor`Private`getType}[\text{CovD}] \wedge = \text{CD}$ $\text{IndexedObjectQ}[\text{CovD}] \wedge = \text{True}$ $\text{IndexedOperatorQ}[\text{CovD}] \wedge = \text{True}$

Format Definitions

 $\text{CovD} /: \text{MakeBoxes}[\text{CovD}[\text{mGRG`STensor`Private`arg}\_\_, \text{mGRG`STensor`Private`expr}____],$

Full Name Global`CovD

^

 $In[*]:=$ **SP`defineOperator[OD, "∂", CD];** $In[*]:=$ $\{\text{OD}[-1, \text{T}[\text{ua}, \text{ub}]], \text{OD}[1, \text{T}[\text{ua}, \text{ub}]],$
 $\text{OD}[1\mu, \text{T}[\text{ua}, \text{ub}]], \text{OD}[1a, \text{T}[\text{ua}, \text{ub}]], \text{OD}[1a, \text{nonT}[\text{ua}, \text{ub}]]\}$ $Out[*]=$ $\{\partial_1 T^{ab}, \partial^1 T^{ab}, \partial_\mu T^{ab}, \partial_a T^{ab}, \partial_a \text{nonT}[\text{ua}, \text{ub}]\}$

In[*]:=

? OD

Out[*]=

Symbol

Global`OD

UpValue Definitions

 $\text{mGRG`STensor`Private`getKindL}[\text{OD}] \wedge = \{\text{Latin}\}$  $\text{mGRG`STensor`Private`getPrtStr}[\text{OD}] \wedge = \partial$ $\text{mGRG`STensor`Private`getType}[\text{OD}] \wedge = \text{CD}$ $\text{IndexedObjectQ}[\text{OD}] \wedge = \text{True}$ $\text{IndexedOperatorQ}[\text{OD}] \wedge = \text{True}$

Format Definitions

 $\text{OD} /: \text{MakeBoxes}[\text{OD}[\text{mGRG`STensor`Private`arg}\_\_, \text{mGRG`STensor`Private`expr}____], \text{Sta}$

Full Name Global`OD

^

연산자 Kind를 마지막 인자로 지정할 수 있다:

In[*]:=

SP`defineOperator[BasisD, "D", CD, Greek]

In[*]:=

 $\{\text{BasisD}[-1, \text{T}[\text{ua}, \text{ub}]], \text{BasisD}[1, \text{T}[\text{ua}, \text{ub}]],$
 $\text{BasisD}[1\mu, \text{T}[\text{ua}, \text{ub}]], \text{BasisD}[1\mu, \text{nonT}[\text{ua}, \text{ub}]]\}$

Out[*]=

 $\{D_1 T^{ab}, D^1 T^{ab}, D_\mu T^{ab}, D_\mu \text{nonT}[\text{ua}, \text{ub}]\}$

In[]:=

?BasisD

Out[]:=

Symbol

Global`BasisD

UpValue Definitions

mGRG`STensor`Private`getKindL[BasisD] ^= {Greek}

mGRG`STensor`Private`getPrtStr[BasisD] ^= D

mGRG`STensor`Private`getType[BasisD] ^= CD

IndexedObjectQ[BasisD] ^= True

IndexedOperatorQ[BasisD] ^= True

Format Definitions

BasisD /: MakeBoxes[BasisD[mGRG`STensor`Private`arg\$\_, mGRG`STensor`Private`expr\$_

Full Name Global`BasisD

^

In[]:=

SP`defineOperator[CCovD, "D", CD, Capital];

SP`defineOperand[CT, "ab", {Capital}, {-1}, IndexedTensorQ, PrintAs → "T"]

In[]:=

CCovD[1A, CT[uB, uC]]

Out[]:=

 $D_A T^{BC}$

LD Type Operator

Lie Derivative 형태의 연산자이다. 연산자의 인자로 하나의 기호와 피연산자가 있는 경우이다.

In[]:=

SP`defineOperator[LieD, "L", LD]

In[]:=

{LieD[someV, T[ua, ub]], LieD[someV, nonT[ua, ub]]}

Out[]:=

 $\{\mathcal{L}_{\text{someV}} T^{ab}, \mathcal{L}_{\text{someV}} \text{nonT}[ua, ub]\}$

<code>In[*]:=</code>	? LieD
<code>Out[*]=</code>	Symbol Global`LieD</div> <div>           UpValue Definitions  <code>mGRG`STensor`Private`getKindL[LieD] ^= {Latin}</code> <code>mGRG`STensor`Private`getPrtStr[LieD] ^= <math>\mathcal{L}</math></code>  <code>mGRG`STensor`Private`getType[LieD] ^= LD</code> <code>IndexedObjectQ[LieD] ^= True</code> <code>IndexedOperatorQ[LieD] ^= True</code> Format Definitions <code>LieD /: MakeBoxes[LieD[mGRG`STensor`Private`arg\$, mGRG`STensor`Private`expr\$____],</code> Full Name Global`LieD ^
<code>In[*]:=</code>	V[ua]
<code>Out[*]=</code>	ξ^a
<code>In[*]:=</code>	LieD[V, T[ua, ub]] (* NB: V[ua]의 출력은 ξ^a *)
<code>Out[*]=</code>	$\mathcal{L}_V T^{ab}$
<code>In[*]:=</code>	"$\mathcal{L}_V T^{ab}$" // FullForm (* copy of a previous output *)
<code>Out[*]//FullForm=</code>	<code>LieD[V, T[ua, ub]]</code>

In[*]:= % // ToBoxes

Out[*]=

```
InterpretationBox[StyleBox[
  TemplateBox[{SubscriptBox[" $\mathcal{L}$ ", V], InterpretationBox[StyleBox[SubsuperscriptBox[
    T, TemplateBox[{StyleBox[1, ShowContents → False, StripOnInput → False],
      StyleBox[1, ShowContents → False, StripOnInput → False]}, RowDefault],
    TemplateBox[{a, b}, RowDefault]}, AutoSpacing → False,
    ShowAutoStyles → False], T@@{ua, ub}, Editable → False]}, RowDefault],
  AutoSpacing → False, ShowAutoStyles → False],  $\mathcal{L}_V$ 
  Tab,
  Editable →
  False]
```

XD Type Operator

Exterior Derivative 형태의 연산자이다. 연산자의 인자로 피연산자만 있는 경우이다.

In[*]:= SP`defineOperator[BOX, "□", XD]

In[*]:= {BOX[T[ua, ub]], BOX[nonT[ua, ub]]}

Out[*]=

```
{□Tab, □nonT[ua, ub]}
```

In[*]:= ? BOX

Out[*]=

Symbol

Global`BOX

UpValue Definitions

mGRG`STensor`Private`getKindL[BOX] ^= {Latin}

mGRG`STensor`Private`getPrtStr[BOX] ^= □

mGRG`STensor`Private`getType[BOX] ^= XD

IndexedObjectQ[BOX] ^= True

IndexedOperatorQ[BOX] ^= True

Format Definitions

BOX /: MakeBoxes[BOX[mGRG`STensor`Private`arg\$, mGRG`STensor`Private`expr\$____], S

Full Name Global`BOX

^

In[*]:= SP`defineOperator[Lap, "∂²", XD]

In[*]:= **Lap**[T[ua, ub]]

Out[*]=

$$\partial^2 T^{ab}$$

In[*]:= **? Lap**

Out[*]=

Symbol

Global`Lap

UpValue Definitions

mGRG`STensor`Private`getKindL[Lap] ^= {Latin}

mGRG`STensor`Private`getPrtStr[Lap] ^= ∂^2

mGRG`STensor`Private`getType[Lap] ^= XD

IndexedObjectQ[Lap] ^= True

IndexedOperatorQ[Lap] ^= True

Format Definitions

Lap /: MakeBoxes[Lap[mGRG`STensor`Private`arg\$\_, mGRG`STensor`Private`expr\$____], S

Full Name Global`Lap

^

XP Type Operator

Exterior Product 형태의 연산자이다. 연산자의 인자로 여러 개의 피연산자가 있는 경우이다. 이 경우는 실제 출력의 형태를 지정하기 위해 정의하려는 연산자의 MakeBoxes 함수를 따로 정의하여야 한다.

In[*]:= **SP`defineOperator**[ExtP, "^", XP]

In[*]:= **ExtP**[T[ua, ub], nonT[ua, ub]]

Out[*]=

$$\text{ExtP}[T^{ab}, \text{nonT}[ua, ub]]$$

In[*]:= **ExtP** /: MakeBoxes[ExtP[args__], StandardForm] := MakeBoxes[Wedge[args]]

In[*]:= **ExtP**[T[ua, ub], nonT[ua, ub]]

Out[*]=

$$T^{ab} \wedge \text{nonT}[ua, ub]$$

get*

```

In[*]:= SP`defineOperator[CovD, "∇", CD]

In[*]:= Through[{SP`getType, SP`getPrtStr, SP`getKindL}[CovD]]
Out[*]=
{CD, ∇, {Latin}}

```

Scalar Functions

```

In[*]:= Tdefine[f[]]; Tdefine[T, "ba"]

```

ScalarFunctionQ

```

In[*]:= ScalarFunctionQ::usage
Out[*]=
ScalarFunctionQ[name] returns True if 'name' is a scalar function
(e.g., Sin, Cos, or Tscalar), which can take scalar arguments.

In[*]:= ScalarFunctionQ/@{Tscalar, Power, Log, Sin, Cos, Tanh, someF}
Out[*]=
{True, True, True, True, True, True, False}

```

Wrapper of Scalars: Tscalar

```

In[*]:= Tscalar::usage
Out[*]=
A wrapper for scalar quantities to protect them from tensor operations.

```

IndexedObject로 구성된 표현 전체를 스칼라로 다루기 위해 Tscalar로 그 표현을 둘러 쓴다:

```

In[*]:= {Sinh[f[]], Sinh[Tscalar[f[]]], Sinh[Tscalar[T[la, lb] × T[ua, ub]]]}
Out[*]=
{Sinh[f], Sinh[(f)], Sinh[(Tab Tab)]}

```

```

In[*]:= % // FullForm
Out[*]//FullForm=
List[Sinh[f[]], Sinh[Tscalar[f[]]], Sinh[Tscalar[Times[T[la, lb], T[ua, ub]]]]]

```

상수 기호와 ScalarFunctionQ를 만족하는 것들은 자동적으로 재조정된다:

```
In[*]:= {Tscalar[a Sin[f[]] Log[f[]]], Tscalar[c f[]], Tscalar[c]}
```

```
Out[*]= {a Log[f] Sin[f], c (f), c}
```

```
In[*]:= % // FullForm
```

```
Out[*]//FullForm= List[Times[a, Log[f[]], Sin[f[]]], Times[c, Tscalar[f[]]], c]
```

중첩된 Tscalar는 자동적으로 재조정된다:

```
In[*]:= Tscalar[Sinh[x] T[la, lb] × T[ua, ub] × Tscalar[T[la, ub] × T[ua, lb]] a Tscalar[f[]]
```

```
Out[*]= a Sinh[x] (f) (Tab Tba) (Tab Tab)
```

```
In[*]:= % // FullForm
```

```
Out[*]//FullForm= Times[a, Sinh[x], Tscalar[f[]],  
Tscalar[Times[T[la, ub], T[ua, lb]]], Tscalar[Times[T[la, lb], T[ua, ub]]]]
```

```
In[*]:= Tscalar[Tscalar[T[la, lb] × T[ua, ub]]]
```

```
Out[*]= (Tab Tab)
```

```
In[*]:= % // FullForm
```

```
Out[*]//FullForm= Tscalar[Times[T[la, lb], T[ua, ub]]]
```

In[*]:= %% // ToBoxes

Out[*]=

```
InterpretationBox[
  StyleBox[TemplateBox[{StyleBox[, FontColor → ■], RowBox[{InterpretationBox[
    StyleBox[SubsuperscriptBox[T, TemplateBox[{a, b}, RowDefault],
      TemplateBox[{StyleBox[1, ShowContents → False, StripOnInput → False],
        StyleBox[1, ShowContents → False, StripOnInput → False]}, RowDefault]],
    AutoSpacing → False, ShowAutoStyles → False], T@@{la, lb},
    Editable → False], , InterpretationBox[StyleBox[SubsuperscriptBox[T,
      TemplateBox[{StyleBox[1, ShowContents → False, StripOnInput → False],
        StyleBox[1, ShowContents → False, StripOnInput → False]}, RowDefault],
      TemplateBox[{a, b}, RowDefault]], AutoSpacing → False,
      ShowAutoStyles → False], T@@{ua, ub}, Editable → False]]],
    StyleBox[, FontColor → ■]], RowDefault],
  AutoSpacing → False, ShowAutoStyles →
    False], (Ta
    Tab),
  Editable → False]
```

스칼라 표현의 Log:

In[*]:= Log@Tscalar[a f[] × T[la] × T[ua]]

Out[*]=

Log[a (f T_a T^a)]

In[*]:= FullForm[%]

Out[*]//FullForm=

Log[Times[a, Tscalar[Times[f[], T[la], T[ua]]]]]

스칼라 표현의 Power:

In[*]:= Tscalar[a T[la] × T[ua]]²

Out[*]=

a² (T_a T^a)²

In[*]:= % // FullForm

Out[*]//FullForm=

Times[Power[a, 2], Power[Tscalar[Times[T[la], T[ua]]], 2]]

In[*]:= {Tscalar[T[la] × T[ua]]², Tscalar[Tscalar[T[la] × T[ua]]²}

Out[*]=

{(T_a T^a)², (T_a T^a)²}

인덱스를 갖는 텐서에 Power 연산을 하려면 (비록 실제로는 스칼라 표현이 아닐 수 있어도) 먼저 Tscalar로 둘러 싸야 한다:

```
In[*]:= {(T[la, lb] × T[uc, ud])2, Tscalar[T[la, lb] × T[uc, ud]]2}
Out[*]= {Tab2 Tcd2, (Tab Tcd)2}
```

```
In[*]:= FullForm[%]
Out[*]//FullForm= List[Times[Power[T[la, lb], 2], Power[T[uc, ud], 2]],
  Power[Tscalar[Times[T[la, lb], T[uc, ud]]], 2]]
```

Wrapper for Invalid Expressions: ErrorT

```
In[*]:= ErrorT::usage
Out[*]= ErrorT[expr] is a wrapper for expressions
  with errors, causing expr to be displayed in red.
```

프로그램 동작 중 적절하지 않은 표현은 ErrorT로 둘러 싸서 더 이상의 연산을 멈추게 한다.

```
In[*]:= Through[{ObjectQ, ScalarFunctionQ}@ErrorT]
Out[*]= {False, False}
```

```
In[*]:= ErrorT[A]
Out[*]= A
```

```
In[*]:= % // ToBoxes
Out[*]= InterpretationBox[StyleBox[StyleBox[A, FontColor → ■],
  AutoSpacing → False, ShowAutoStyles → False], A, Editable → False]
```

```
In[*]:= {T[1A, lb], T[la, ub]}
Out[*]= {TAb, Tab}
```

```
In[*]:= % // SyntaxCheck
Out[*]= ... General: 1A is not a valid index.
  {TAb, Tab}
```

Utilities for Indexed Objects

```
In[*]:=
Tdefine[scalarR[], PrintAs → "R"];
Tdefine[R, "ba"];
Tdefine[F, "-ba"];
SP`defineOperator[CovD, "∇", CD];
Tdefine[f[]];
Tdefine[v, "a"];
```

ExpandObject

```
In[*]:= ExpandObject::usage
```

```
Out[*]=
```

ExpandObject[expr] expands expressions containing indexed objects, distributing products over sums.

임의의 표현에서 IndexedObject를 전개한다. 옵션으로 HeadQs가 있다.

```
In[*]:=
expr =
(a + b) (CovD[1c, R[1a, 1b]] + CovD[1c, F[1a, 1b]]) (R[ua, ub] + F[ua, ub]) escalarR[]
ExpandObject[expr]
```

```
Out[*]=
```

$$(a + b) e^R (\nabla_c F_{ab} + \nabla_c R_{ab}) (F^{ab} + R^{ab})$$

```
Out[*]=
```

$$(a + b) e^R \nabla_c F_{ab} F^{ab} + (a + b) e^R \nabla_c R_{ab} F^{ab} + (a + b) e^R \nabla_c F_{ab} R^{ab} + (a + b) e^R \nabla_c R_{ab} R^{ab}$$

IndexedObject 중에서 연산자 표현만 전개하도록 옵션으로 제한할 수 있다:

```
In[*]:= ExpandObject[expr, HeadQs → {IndexedOperatorQ}]
```

```
Out[*]=
```

$$(a + b) e^R \nabla_c F_{ab} (F^{ab} + R^{ab}) + (a + b) e^R \nabla_c R_{ab} (F^{ab} + R^{ab})$$

FreeObjectQ

```
In[*]:= FreeObjectQ::usage
```

```
Out[*]=
```

FreeObjectQ[expr] returns True if 'expr' is free of indexed objects, and False otherwise.

임의의 표현에서 IndexedObject의 존재 여부를 판단하는 질의 함수이다. 옵션으로 HeadQs가 있다.

```
In[*]:= expr = {scalarR[]  $\times$  R[la, lb] + a F[la, lb], a * b}  
expr // FreeObjectQ
```

```
Out[*]=  
{a Fab + Rab R, a b}
```

```
Out[*]=  
{False, True}
```

IndexedObject 중에서 연산자 표현의 존재 여부만 질의할 수 있다:

```
In[*]:= FreeObjectQ[expr, HeadQs  $\rightarrow$  {IndexedOperatorQ}]
```

```
Out[*]=  
{True, True}
```

ScalarFunction도 IndexedObject로 간주하려면 HeadQs 옵션으로 ObjectQ를 사용한다:

```
In[*]:= FreeObjectQ[Tscalar[a R[la, lb]  $\times$  R[ua, ub]]]
```

```
Out[*]=  
True
```

```
In[*]:= FreeObjectQ[Tscalar[a R[la, lb]  $\times$  R[ua, ub]], HeadQs  $\rightarrow$  {ObjectQ}]
```

```
Out[*]=  
False
```

ForEachTerm

```
In[*]:= ForEachTerm::usage
```

```
Out[*]=  
ForEachTerm[expr, f, args...] applies the  
function 'f' to each term of a sum or equation in 'expr'.
```

Visit each terms in an expression and then act 'func' on them with 'args'.

```
In[*]:= ForEachTerm[c == a d + b e, func, arg1, arg2]
```

```
Out[*]=  
func[c, arg1, arg2] == func[a d, arg1, arg2] + func[b e, arg1, arg2]
```

ForEachObject

```
In[*]:= ForEachObject::usage
```

```
Out[*]=  
ForEachObject[expr, hOptL, f, args...] applies 'f' to each  
indexed object in 'expr' that satisfies the head options 'hOptL'.
```

Visit each objects in an expression and then act 'func' on them with 'hOptL' and 'args'. 여기서

'hOptL'은 HeadQs 옵션 리스트이다.

```
In[*]:= ForEachObject[f[] == v[la] × v[ua] + a R[lb, ub], {some1 → some2}, func, arg1, arg2]
Out[*]=
func[f, arg1, arg2] ==
a func[Rbb, arg1, arg2] + func[va, arg1, arg2] × func[va, arg1, arg2]
```

SplitTerm

```
In[*]:= SplitTerm::usage
Out[*]=
SplitTerm[term, hOptL] splits a term into {scalarPart, tensorPart},
separating indexed objects defined by 'hOptL' from other factors.
```

IndexedObject인 것과 아닌 것들의 곱으로 구성된 항(Term)에서 그것들을 분리한다:

```
In[*]:= SplitTerm[a R[la, lb]]
Out[*]=
{a, Rab}
```

두 번째 인자는 HeadQs 옵션 리스트이다 (기본은 IndexedObjectQ).

```
In[*]:= SplitTerm[a R[la, lb] × CovD[lc, scalarR[]]]
Out[*]=
{a, ∇c Rab}
```

IndexedObject 중에서 연산자와 연산자가 아닌 것들을 분리한다:

```
In[*]:= SplitTerm[a R[la, lb] × CovD[lc, scalarR[]], {HeadQs → {IndexedOperatorQ}}]
Out[*]=
{a Rab, ∇c R}
```

ScalarFunction은 IndexedObject가 아니다:

```
In[*]:= SplitTerm[a Tscalar[R[la, lb] × R[ua, ub]] Log[scalarR[]] CovD[lc, scalarR[]]]
Out[*]=
{a Log[R] (Rab Rab), ∇c R}
```

ScalarFunction도 IndexedObject에 포함시키려면 HeadQs 옵션을 조정한다:

```
In[*]:= expr = a Tscalar[R[la, lb] × R[ua, ub]] Log[scalarR[]] CovD[lc, scalarR[]]
SplitTerm[expr, {HeadQs → {ObjectQ}}]
Out[*]=
a ∇c R Log[R] (Rab Rab)
Out[*]=
{a, ∇c R Log[R] (Rab Rab)}
```

wrapObject

첫 번째 인자에 있는 IndexedObject들 각각을 두 번째 인자로 둘러 쓴다. 옵션으로 HeadQs가 있다.

```
In[ ]:= expr = a F[ua, lb] × CovD[la, scalarR[]] + b R[ua, lb]
SP`wrapObject[expr, W]
```

```
Out[ ]:= a ∇aR Fab + b Rab
```

```
Out[ ]:= a W[∇aR] × W[Fab] + b W[Rab]
```

IndexedObject 중에서 연산자 표현만 둘러 쓴다:

```
In[ ]:= SP`wrapObject[expr, W, HeadQs → {IndexedOperatorQ}]
```

```
Out[ ]:= b Rab + a Fab W[∇aR]
```

ScalarFunction은 IndexedObject로 간주하지 않는다:

```
In[ ]:= expr = {xn Log[R[la, lb] × R[ua, ub]], ex Tscalar[R[la, lb] × R[ua, ub]]2}
SP`wrapObject[expr, W]
```

```
Out[ ]:= {xn Log[Rab Rab], ex (Rab Rab)2}
```

```
Out[ ]:= {xn Log[Rab Rab], ex (Rab Rab)2}
```

ScalarFunction도 IndexedObject로 간주하려면 HeadQs 옵션으로 ObjectQ를 사용한다:

```
In[ ]:= SP`wrapObject[expr, W, HeadQs → {ObjectQ}]
```

```
Out[ ]:= {W[xn] × W[Log[Rab Rab]], W[ex] × W[(Rab Rab)2]}
```

NB: x^n is a ScalarFunction because Head[x^n] == Power:

```
In[ ]:= ScalarFunctionQ[Head[xn]]
```

```
Out[ ]:= True
```

Power 표현만 둘러 싸도록 HeadQs 옵션을 설정한다:

```
In[ ]:= expr = Log[R[la, lb] × R[ua, ub]] ex Tscalar[R[la, lb] × R[ua, ub]]2
SP`wrapObject[expr, W, HeadQs → {# == Power &}]
```

```
Out[ ]:= ex Log[Rab Rab] (Rab Rab)2
```

```
Out[ ]:= Log[Rab Rab] W[ex] × W[(Rab Rab)2]
```

Operations

```
In[*]:= DefKind[Greek, Alphabet["Greek"]];
DefKind[Capital, ToUpperCase /@ Alphabet[]];
DefKind[Bar, "b" <> # & /@ Alphabet[]];
DefKind[Dot, "d" <> # & /@ Alphabet[]];
DefKind[Hat, "h" <> # & /@ Alphabet[]];
```

Find Indices of Indexed Expressions

```
In[*]:= Tdefine[R, "ba"]; Tdefine[F, "-ba"]; SP`defineOperator[CovD, "∇", CD]
```

FindIndices

```
In[*]:= FindIndices::usage
```

```
Out[*]=
```

FindIndices[expr] returns a list of all indices present in 'expr'.

IndexedObject로 구성된 임의의 항에서 (인덱스 Kind에 합당한, 즉 ValidIndexQ를 만족하는) 모든 인덱스를 (정렬하지 않고) 리스트에 담는다. 옵션으로 IndexQs와 HeadQs가 있다.

```
In[*]:= FindIndices[R[-1, uc]]
```

```
Out[*]=
```

{-1, uc}

Latin Kind에 부합하는 인덱스들을 텐서 표현에 나타나는 순서대로 돌려 준다:

```
In[*]:= expr = (a + b) CovD[1a, F[ua, 1b]] × R[-1, uc]
FindIndices[expr]
```

```
Out[*]=
```

$(a + b) \nabla_a F^a_b R_1^c$

```
Out[*]=
```

{1a, ua, 1b, -1}

텐서 인덱스만 돌려 주도록 옵션 IndexQs를 사용한다:

```
In[*]:= FindIndices[expr, IndexQs → {TensorialIndexQ}]
```

```
Out[*]=
```

{1a, ua, 1b}

특별한 표현의 인덱스만 돌려 주도록 옵션 HeadQs 를 사용한다 (CovD는 연산자이다):

```
In[*]:= FindIndices[expr, HeadQs → {IndexedOperandQ}]
```

```
Out[*]:= {-1}
```

같은 인덱스가 여러 개일지라도 (적절한 텐서 표현은 아니지만) 그대로 돌려 준다:

```
In[*]:= FindIndices[R[1a, 1b] × R[1a, ub]]
```

```
Out[*]:= {1a, 1b, 1a, ub}
```

IndexedObject가 아닌 것의 인덱스는 고려되지 않는다.

```
In[*]:= FindIndices[nonT[1a, ub]]
```

```
Out[*]:= {}
```

ScalarFunction은 IndexedObject가 아니다:

```
In[*]:= {Log[R[1a, 1b] × R[1a, ub]], Tscalar[R[1a, 1b] × R[1c, ub]]2}  
FindIndices /@ %
```

```
Out[*]:= {Log[Rab Rab], (Rab Rcb)2}
```

```
Out[*]:= {{}, {}}
```

ScalarFunction의 인덱스들도 처리하려면 HeadQs의 옵션으로 ObjectQ를 이용한다:

```
In[*]:= {Log[R[1a, 1b] × R[1a, ub]], Tscalar[R[1a, 1b] × R[1c, ub]]2, eR[1d, ud]}  
FindIndices[#, HeadQs → {ObjectQ}] & /@ %
```

```
Out[*]:= {Log[Rab Rab], (Rab Rcb)2, eRdd}
```

```
Out[*]:= {{1a, 1b, 1a, ub}, {1a, 1b, 1c, ub}, {1d}}
```

인덱스 Kind가 고려된다. 아래 표현은 DefaultKind의 텐서이므로 그 Kind의 인덱스만 돌려 준다:

```
In[*]:= (a + b) CovD[1A, F[ua, 1b]] × R[1c, 1B]  
FindIndices[%]
```

```
Out[*]:= (a + b) ∇A Fab RcB
```

```
Out[*]:= {ua, 1b, 1c}
```

FindFreeTensorialIndices

In[*]:= **FindFreeTensorialIndices::usage**

Out[*]=

FindFreeTensorialIndices[expr] returns a
list of free (uncontracted) tensorial indices in 'expr'.

IndexedObject로 구성된 임의의 항에서 (인덱스 Kind에 합당한) free 텐서 인덱스를 돌려 준다.
옵션으로 IndexQs, HeadQs가 있다.

In[*]:= **expr = (a + b) CovD[1a, F[ua, 1b]] × R[-1, uc]**
FindFreeTensorialIndices[expr]

Out[*]=

$(a + b) \nabla_a F^a_b R_1^c$

Out[*]=

{1b, uc}

위 첨자 인덱스만 돌려 주도록 제한한다:

In[*]:= **FindFreeTensorialIndices[expr, IndexQs → {UpIndexQ}]**

Out[*]=

{uc}

같은 인덱스가 여러 개일지라도 (비록 정당한 텐서 표현은 아니지만) 그대로 돌려 준다:

In[*]:= **R[1a, 1b] × R[1a, ub]**
FindFreeTensorialIndices[%]

Out[*]=

$R_{ab} R_a^b$

Out[*]=

{1a, 1a}

IndexedObject가 아닌 것의 인덱스는 고려되지 않는다.

In[*]:= **FindFreeTensorialIndices[nonT[1a, uB]]**

Out[*]=

{}

ScalarFunction은 IndexedObject가 아니다:

In[*]:= **expr = {Tscalar[Log[R[1a, 1b] × R[1c, ub]]], Tscalar[R[1a, 1b] × R[1c, ub]]²}**
FindFreeTensorialIndices /@ expr

Out[*]=

{Log[R_{ab} R_c^b], (R_{ab} R_c^b)²}

Out[*]=

{{}, {}}

ScalarFunction의 인덱스들도 처리하려면 HeadQs의 옵션으로 ObjectQ를 이용한다:

```
In[*]:= FindFreeTensorialIndices[#, HeadQs → {ObjectQ}] & /@ expr
Out[*]= {{1a, 1c}, {1a, 1c}}
```

```
In[*]:= Tscalar[R[1a, 1b] × R[1c, ub]]2
FindFreeTensorialIndices[#, HeadQs → {ObjectQ}] & @ %
Out[*]= (Rab Rcb)2
Out[*]= {1a, 1c}
```

인덱스 Kind를 고려한다. 아래 표현은 DefaultKind의 텐서이므로 그 Kind의 free 인덱스만 돌려 준다:

```
In[*]:= (a + b) CovD[1a, R[ua, 1b]] × R[1c, 1B]
FindFreeTensorialIndices[%]
Out[*]= (a + b) ∇aRab RcB
Out[*]= {1b, 1c}
```

FindIndicesAll, FindFreeTensorialIndicesAll

```
In[*]:= FindIndicesAll::usage
Out[*]= FindIndicesAll[expr] returns a list of all
indices in 'expr', without checking for index validity.
```

```
In[*]:= FindFreeTensorialIndicesAll::usage
Out[*]= FindFreeTensorialIndicesAll[expr] returns a list of all free
tensorial indices in 'expr', without checking for index validity.
```

IndexedObject로 구성된 임의의 항에서 모든 Kind의 모든 인덱스를 돌려 준다.

```
In[*]:= expr = (a + b) CovD[1A, F[ua, 1B]] × R[-1, uA] eR[1a, uD]
Through@{FindIndicesAll, FindFreeTensorialIndicesAll}[expr]
Out[*]= (a + b) eRaD ∇AFaB R1A
Out[*]= {{1A, ua, 1B, -1, uA}, {ua, 1B}}
```

같은 인덱스가 여러 개일지라도 그 순서대로 돌려 준다:

```
In[*]:= Through@{FindIndicesAll, FindFreeTensorialIndicesAll}[R[la, lb] × R[la, ub]]
Out[*]:= {{la, lb, la, ub}, {la}}
```

IndexedObject가 아닌 것의 인덱스는 고려되지 않는다.

```
In[*]:= FindIndicesAll[nonT[la, ub]]
Out[*]:= {}
```

ScalarFunction은 IndexedObject가 아니다.

```
In[*]:= expr = {Log[R[la, lb] × R[la, ub]], Tscalar[R[la, lb] × R[lc, ub]]^2}
FindIndicesAll /@ expr
Out[*]:= {{Log[Rab Rab], (Rab Rcb)^2}}
Out[*]:= {{}, {}}
```

ScalarFunction의 인덱스들도 처리하려면 HeadQs의 옵션으로 ObjectQ를 이용한다:

```
In[*]:= FindIndicesAll[#, HeadQs → {ObjectQ}] & /@ expr
Out[*]:= {{la, lb, la, ub}, {la, lb, lc, ub}}
```

```
In[*]:= FindFreeTensorialIndicesAll[#, HeadQs → {ObjectQ}] & /@ expr
Out[*]:= {{la}, {la, lc}}
```

Utilities for Indexed Objects

```
In[*]:= Tdefine[f[]];
Tdefine[v, "a"];
Tdefine[scalarR[], PrintAs → "R"];
Tdefine[R, "ba"];
Tdefine[F, "-ba"];
SP`defineOperator[CovD, "∇", CD]
```

ScalarFunctionQ and NoIndexQ

In[]:= **ScalarFunctionQ::usage**

Out[]:=

ScalarFunctionQ[name] returns True if 'name' is a scalar function (e.g., Sin, Cos, or Tscalar), which can take scalar arguments.

ScalarFunctionQ는 인자로 주어진 이름이 ScalarFunction이면 True 값을 돌려주는 질의 함수이다.

In[]:= **ScalarFunctionQ** /@ {Tscalar, Power, Log, Exp, f}

Out[]:=

{True, True, True, True, False}

In[]:= **NoIndexQ::usage**

Out[]:=

NoIndexQ[expr] returns True if 'expr' is a scalar or contains no free tensorial indices.

Free Index가 없는 IndexedObject나 ScalarFunction들은 NoIndexQ를 만족한다.

In[]:= **NoIndexQ** /@ {scalarR[], R[1a, 1b] × R[ua, ub]}

Out[]:=

{True, True}

In[]:= **NoIndexQ** /@ {Tscalar[R[1a, 1b]], Log[R[1a, ub]]}

Out[]:=

{True, True}

In[]:= **NoIndexQ** /@ {R[-1, 2], nonSF[x]}

Out[]:=

{True, True}

In[]:= **NoIndexQ** /@ {Sinh[f[]], Tscalar[f[]]}

Out[]:=

{True, True}

스칼라가 아닌 인덱스를 갖는 텐서임에도 불구하고 인덱스 표현이 잘못되어 NoIndexQ를 만족할 수 있음을 **주의**하자. 예를 들면, 텐서 R은 Latin Kind이므로 그 Kind가 아닌 인덱스는 합당한 인덱스로 간주되지 않는다:

In[]:= **NoIndexQ** /@ {R[1 α , 1 β], R[1A, 1B]}

Out[]:=

{True, True}

In[*]:= **NoIndexQ** /@ {**R**[**la**, **lb**], **R**[**la**, **lb**] }

Out[*]=
{False, False}

In[*]:= **NoIndexQ** /@ {**R**[**la**, **lb**] × **R**[**lμ**, **lv**], **R**[**lμ**, **lv**] × **R**[**lρ**, **lσ**] }

Out[*]=
{False, True}

AntisymmetrizeIndices and SymmetrizeIndices

In[*]:= **AntisymmetrizeIndices::usage**

Out[*]=
AntisymmetrizeIndices[**expr**, {**i1**, **i2**, ...}] returns the expression antisymmetrized with respect to the list of indices.

두 번째 인자로 주어진 인덱스 리스트로 첫 번째 인자에 있는 IndexedObject의 인덱스를 대칭 또는 반대칭이 되도록 한다. 옵션으로 HeadQs가 있다. (내부적으로 모든 가능한 순열을 생성하므로 대칭 또는 반대칭이 되는 인덱스의 수가 너무 많으면 연산 시간과 메모리 자원을 과도하게 사용하므로 주의가 필요하다.)

In[*]:= **AntisymmetrizeIndices** [**F**[**la**, **lb**], {**la**, **lb**}]

Out[*]=
$$\frac{1}{2} (F_{ab} - F_{ba})$$

In[*]:= **CovD**[**uc**, **CovD**[**la**, **F**[**ub**, **lc**]]]

Out[*]=
AntisymmetrizeIndices [% , {**la**, **ub**, **lc**}]

Out[*]=
$$\nabla^c \nabla_a F^b_c$$

Out[*]=
$$\frac{1}{6} (-\nabla^c \nabla_a F^b_c + \nabla^c \nabla_a F^b_c + \nabla^c \nabla_c F^b_a - \nabla^c \nabla_c F^b_a - \nabla^c \nabla^b F_{ac} + \nabla^c \nabla^b F_{ca})$$

In[*]:= **expr** = **a scalarR**[] × **CovD**[**la**, **R**[**lb**, **lc**]] × **F**[**ld**, **lf**]

Out[*]=
AntisymmetrizeIndices [**expr**, {**la**, **lb**, **ld**}]

Out[*]=
$$a \nabla_a R_{bc} F_{df} R$$

Out[*]=
$$\frac{1}{6} a (\nabla_b R_{dc} F_{af} R - \nabla_d R_{bc} F_{af} R - \nabla_a R_{dc} F_{bf} R + \nabla_d R_{ac} F_{bf} R + \nabla_a R_{bc} F_{df} R - \nabla_b R_{ac} F_{df} R)$$

두 번째 인자의 인덱스들이 HeadQs를 만족하는 IndexedObject에 존재해야 한다.

```
In[*]:= AntisymmetrizeIndices[expr, {la, lb, ld}, HeadQs -> {IndexedOperatorQ}]
```

```
Msg: {la, lb, ld} is not a subset of {la, lb, lc}
```

```
Out[*]=
```

$$a \nabla_a R_{bc} F_{df} R$$

```
In[*]:= SplitTerm[expr, {HeadQs -> {IndexedOperatorQ}}]
```

```
Out[*]=
```

$$\{a F_{df} R, \nabla_a R_{bc}\}$$

ScalarFunction은 IndexedObject로 간주하지 않는다:

```
In[*]:= SymmetrizeIndices[Log[R[la, lb] × R[ua, ub]], {la, ub}]
```

```
Msg: {la, ub} is not a subset of {}
```

```
Out[*]=
```

$$\text{Log}[R_{ab} R^{ab}]$$

ScalarFunction의 인덱스들도 처리하려면 HeadQs의 옵션으로 ObjectQ를 이용한다:

```
In[*]:= SymmetrizeIndices[a Tscalar[R[la, lb] × R[ua, ub]]^2, {la, ub}, HeadQs -> {ObjectQ}]
```

```
Out[*]=
```

$$\frac{1}{2} a \left((R_{ab} R^{ab})^2 + (R^a_a R^b_b)^2 \right)$$

scalarNameQ, vectorNameQ

인자가 스칼라의 이름인지 또는 벡터의 이름인지를 묻는다:

```
In[*]:= SP`scalarNameQ /@ {f, v}
```

```
Out[*]=
```

```
{True, False}
```

```
In[*]:= SP`vectorNameQ /@ {f, v}
```

```
Out[*]=
```

```
{False, True}
```

Functions for Dummy Indices

```
In[*]:=
Tdefine[R, "ba"];
Tdefine[CR[1A, 1B]];
Tdefine[F, "-ba"];
Tdefine[CurvR, "-bacd-abdc+cdab", PrintAs -> "R"];
Tdefine[g, "ba"];
Tdefine[Cg[1A, 1B], "ba"];
SP`defineOperator[CovD, "∇", CD];
SP`defineOperator[CCovD, "D", CD, Capital]
```

Dum

```
In[*]:= Dum::usage
```

```
Out[*]=
```

Dum[expr] contracts all possible dummy index pairs in 'expr'.

임의의 표현에서 (Kind에 부합하는) 위/아래 첨자 쌍이 더미 인덱스가 아니면 더미 인덱스로 바꾼다. 옵션으로 IndexQs와 HeadQs가 있다. (DefaultKind는 Latin이다.)

```
In[*]:=
expr = A R[1a, ua] × CurvR[1c, 1d, uc, ud] +
      a CovD[1A, CurvR[1a, 1b, uc, ud]] × CovD[uA, CurvR[ua, ub, 1c, 1d]]
Dum[expr]
```

```
Out[*]=
```

$$a \nabla_A R_{ab}^{cd} \nabla^A R_{cd}^{ab} + A R_{cd}^{cd} R_a^a$$

```
Out[*]=
```

$$a \nabla_{R_{\text{Latin}\$17171\text{Latin}\$17173}}^{\text{Latin}\$17175\text{Latin}\$17177} \nabla^A R_{\text{Latin}\$17175\text{Latin}\$17177}^{\text{Latin}\$17171\text{Latin}\$17173} + A R_{\text{Latin}\$17182\text{Latin}\$17184}^{\text{Latin}\$17182\text{Latin}\$17184} R_{\text{Latin}\$17180}^{\text{Latin}\$17180}$$

IndexedObject 중에서 연산자 표현에만 Dum 연산을 하도록 HeadQs 옵션을 조정할 수 있다:

```
In[*]:= Dum[expr, HeadQs -> {IndexedOperatorQ}]
```

```
Out[*]=
```

$$a \nabla_{R_{\text{Latin}\$17190\text{Latin}\$17192}}^{\text{Latin}\$17194\text{Latin}\$17196} \nabla^A R_{\text{Latin}\$17194\text{Latin}\$17196}^{\text{Latin}\$17190\text{Latin}\$17192} + A R_{cd}^{cd} R_a^a$$

Kind가 Capital인 인덱스에 대해서만 Dum 연산을 하도록 HeadQs 옵션을 조정할 수 있다:

```
In[*]:= Dum[CR[1A, uA] × R[1a, ua], IndexQs -> {KindIndexQ[Capital]}]
```

```
Out[*]=
```

$$CR_{\text{Capital}\$17203}^{\text{Capital}\$17203} R_a^a$$

ScalarFunction은 IndexedObject가 아니다:

```
In[*]:= {Log[R[lc, ld] × R[uc, ud]], Tscalar[R[lc, ld] × R[uc, ud]]^2}
Dum /@ %
```

```
Out[*]= {Log[Rcd Rcd], (Rcd Rcd)^2}
```

```
Out[*]= {Log[Rcd Rcd], (Rcd Rcd)^2}
```

ScalarFunction의 인덱스들도 고려하려면 HeadQs의 옵션으로 ObjectQ를 사용한다:

```
In[*]:= Dum[#, HeadQs → {ObjectQ}] & /@ %
```

```
Out[*]= {Log[RLatin$17214Latin$17216 RLatin$17214Latin$17216], (RLatin$17219Latin$17221 RLatin$17219Latin$17221)^2}
```

합당한 텐서 표현이 아니어도 있는 그대로 바꾸므로 주의해야 한다. 입력이 잘못되면 출력도 잘못된다.

```
In[*]:= CurvR[la, lb, ua, uc] × CurvR[la, lb, ua, ud]
Dum [%]
```

```
Out[*]= Rabac Rabad
```

```
Out[*]= RLatin$17227bLatin$17227c RLatin$17227bLatin$17227d
```

DumFresh

```
In[*]:= DumFresh::usage
```

```
Out[*]= DumFresh[expr] renames all dummy indices in 'expr' to new unique indices.
```

Dum은 더미 인덱스가 아닌 인덱스 쌍을 더미 인덱스로 바꾼다. 따라서 이미 더미 인덱스인 경우에는 아무런 효과가 없다:

```
In[*]:= Dum[expr]
% // Dum
```

```
Out[*]= a ∇ARLatin$17236Latin$17238Latin$17240Latin$17242 ∇ARLatin$17240Latin$17242Latin$17236Latin$17238 + A RLatin$17247Latin$17249Latin$17247Latin$17249 RLatin$17245Latin$17245
```

```
Out[*]= a ∇ARLatin$17236Latin$17238Latin$17240Latin$17242 ∇ARLatin$17240Latin$17242Latin$17236Latin$17238 + A RLatin$17247Latin$17249Latin$17247Latin$17249 RLatin$17245Latin$17245
```

DumFresh는 임의의 표현에서 (Kind에 부합하는) 모든 인덱스 쌍을 새로운 더미 인덱스로 바꾼다. 따라서 이미 더미 인덱스인 경우에도 새로운 더미 인덱스로 바꾼다.

In[*]:= **Dum[expr]**
DumFresh[%]

Out[*]=

$$a \nabla_{\text{R}_{\text{Latin\$17257Latin\$17259}}}^{\text{Latin\$17261Latin\$17263}} \nabla_{\text{R}_{\text{Latin\$17261Latin\$17263}}}^{\text{Latin\$17257Latin\$17259}} + A \text{R}_{\text{Latin\$17268Latin\$17270}}^{\text{Latin\$17268Latin\$17270}} \text{R}_{\text{Latin\$17266}}^{\text{Latin\$17266}}$$

Out[*]=

$$a \nabla_{\text{R}_{\text{Latin\$17273Latin\$17275}}}^{\text{Latin\$17277Latin\$17279}} \nabla_{\text{R}_{\text{Latin\$17273Latin\$17275}}}^{\text{Latin\$17277Latin\$17279}} + A \text{R}_{\text{Latin\$17284Latin\$17286}}^{\text{Latin\$17284Latin\$17286}} \text{R}_{\text{Latin\$17282}}^{\text{Latin\$17282}}$$

IndexedObject 중에서 연산자 표현에만 DumFresh 연산을 하도록 HeadQs 옵션을 조정할 수 있다:

In[*]:= **DumFresh[expr, HeadQs → {IndexedOperatorQ}]**

Out[*]=

$$a \nabla_{\text{R}_{\text{Latin\$17292Latin\$17294}}}^{\text{Latin\$17296Latin\$17298}} \nabla_{\text{R}_{\text{Latin\$17292Latin\$17294}}}^{\text{Latin\$17296Latin\$17298}} + A \text{R}_{\text{cd}}^{\text{cd}} \text{R}_a^a$$

인덱스 Kind가 Capital인 인덱스에 대해서 DumFresh 연산을 하도록 HeadQs 옵션을 조정할 수 있다:

In[*]:= **CR[1A, uA] × R[1a, ua]**
DumFresh[%, IndexQs → {KindIndexQ[Capital]}]

Out[*]=

$$\text{CR}_A^A \text{R}_a^a$$

Out[*]=

$$\text{CR}_{\text{Capital\$17305}}^{\text{Capital\$17305}} \text{R}_a^a$$

ScalarFunction은 IndexedObject가 아니다. 그러나 ScalarFunction의 인덱스들도 고려하려면 HeadQs의 옵션으로 ObjectQ를 사용한다:

In[*]:= **{Log[R[1c, 1d] × R[uc, ud]], Tscalar[R[1c, 1d] × R[uc, ud]]^2}**
DumFresh[#, HeadQs → {ObjectQ}] & /@ %

Out[*]=

$$\left\{ \text{Log}[\text{R}_{\text{cd}} \text{R}^{\text{cd}}], (\text{R}_{\text{cd}} \text{R}^{\text{cd}})^2 \right\}$$

Out[*]=

$$\left\{ \text{Log}[\text{R}_{\text{Latin\$17311Latin\$17313}}^{\text{Latin\$17311Latin\$17313}} \text{R}^{\text{Latin\$17311Latin\$17313}}], (\text{R}_{\text{Latin\$17316Latin\$17318}}^{\text{Latin\$17316Latin\$17318}} \text{R}^{\text{Latin\$17316Latin\$17318}})^2 \right\}$$

ResetDummies

In[*]:= **ResetDummies::usage**

Out[*]=

ResetDummies[expr, opts] renames dummy indices in 'expr' to a canonical form (e.g., 1a, 1b, ...), avoiding conflicts with existing free indices. Options like HeadQs and IndexQs can be used to control which parts of the expression are processed.

임의의 표현에 있는 더미 인덱스와 인덱스 쌍들을 순차적으로 재조정하여 정규 인덱스로 바꾼다. **주의:** 프로그램이 생성한 더미 인덱스는 프로그램을 사용하는 동안 유일한 값을

가지므로 재조정된 표현과 다른 표현을 섞어서 사용하면 잘못된 결과가 생길 수도 있다.

옵션으로는 IndexQs와 HeadQs가 있다. 그리고 프로그램이 생성한 더미 인덱스만 재조정하여 정규 인덱스로 바꾸려면 옵션으로 All -> False를 입력하거나 Options[ResetDummies]를 조정한다. 기본적으로는 All -> True이므로 더미 인덱스 뿐만 아니라 모든 인덱스 쌍들이 재조정된다.

인덱스 Kind를 고려하여 인덱스 쌍을 더미 인덱스로 바꾼다:

```
In[*]:= A R[1a, ua] × CurvR[1c, 1d, uc, ud] +
a CovD[1B, CurvR[1a, 1b, uc, ud]] × CovD[uB, CurvR[ua, ub, 1c, 1d]]
expr = Dum[%]
```

$$a \nabla_B R_{ab}^{cd} \nabla^B R_{cd}^{ab} + A R_{cd}^{cd} R_a^a$$

$$a \nabla_B R_{\text{Latin}\$17327\text{Latin}\$17329}^{\text{Latin}\$17331\text{Latin}\$17333} \nabla^B R_{\text{Latin}\$17331\text{Latin}\$17333}^{\text{Latin}\$17327\text{Latin}\$17329} + A R_{\text{Latin}\$17338\text{Latin}\$17340}^{\text{Latin}\$17338\text{Latin}\$17340} R_{\text{Latin}\$17336}^{\text{Latin}\$17336}$$

기본적으로는 더미 인덱스 뿐만 아니라 (Kind와 무관하게) 모든 인덱스 쌍들을 재조정하여 정규 인덱스로 바꾼다:

```
In[*]:= ResetDummies[expr]
```

$$a \nabla_A R_{ab}^{cd} \nabla^A R_{cd}^{ab} + A R_{ab}^{ab} R_c^c$$

프로그램이 생성한 더미 인덱스들만 재조정하여 정규 인덱스로 바꾸려면 옵션으로 조정할 수 있다:

```
In[*]:= ResetDummies[expr, All -> False]
```

$$a \nabla_B R_{ab}^{cd} \nabla^B R_{cd}^{ab} + A R_{ab}^{ab} R_c^c$$

IndexedObject 중에서 연산자 표현만 재조정할 수 있다:

```
In[*]:= ResetDummies[expr, HeadQs -> {IndexedOperatorQ}]
```

$$a \nabla_A R_{ab}^{cd} \nabla^A R_{cd}^{ab} + A R_{\text{Latin}\$17338\text{Latin}\$17340}^{\text{Latin}\$17338\text{Latin}\$17340} R_{\text{Latin}\$17336}^{\text{Latin}\$17336}$$

특정 Kind의 인덱스만 재조정하도록 옵션을 사용할 수 있다:

```
In[*]:= expr = expr + R[1p, up]
ResetDummies[expr, IndexQs -> {KindIndexQ[Capital]}]
ResetDummies[expr]
```

$$a \nabla_B R_{\text{Latin}\$17327\text{Latin}\$17329}^{\text{Latin}\$17331\text{Latin}\$17333} \nabla^B R_{\text{Latin}\$17331\text{Latin}\$17333}^{\text{Latin}\$17327\text{Latin}\$17329} + A R_{\text{Latin}\$17338\text{Latin}\$17340}^{\text{Latin}\$17338\text{Latin}\$17340} R_{\text{Latin}\$17336}^{\text{Latin}\$17336} + R_p^p$$

$$a \nabla_A R_{\text{Latin}\$17382\text{Latin}\$17384}^{\text{Latin}\$17386\text{Latin}\$17388} \nabla^A R_{\text{Latin}\$17386\text{Latin}\$17388}^{\text{Latin}\$17382\text{Latin}\$17384} + A R_{\text{Latin}\$17338\text{Latin}\$17340}^{\text{Latin}\$17338\text{Latin}\$17340} R_{\text{Latin}\$17336}^{\text{Latin}\$17336} + R_p^p$$

$$a \nabla_A R_{ab}^{cd} \nabla^A R_{cd}^{ab} + R_a^a + A R_{ab}^{ab} R_c^c$$

ScalarFunction의 인덱스들도 고려하려면 HeadQs의 옵션으로 ObjectQ를 사용한다:

```
In[*]:= {Log[R[lc, ld] × R[uc, ud]], Tscalar[R[lc, ld] × R[uc, ud]]^2}
Dum[#, HeadQs → {ObjectQ}] & /@ %

Out[*]= {Log[Rcd Rcd], (Rcd Rcd)2}
```

```
In[*]:= % // ResetDummies
% // ResetDummies[#, HeadQs → {ObjectQ}] &

Out[*]= {Log[RLatin$17420Latin$17422 RLatin$17420Latin$17422], (RLatin$17425Latin$17427 RLatin$17425Latin$17427)2}
```

```
In[*]:= % // ResetDummies
% // ResetDummies[#, HeadQs → {ObjectQ}] &

Out[*]= {Log[Rab Rab], (Rab Rab)2}
```

SplitIndices

```
In[*]:= SplitIndices::usage

Out[*]= SplitIndices[expr, {i, i1, i2, ...}, {j, j1, j2, ...}, ...] acts as a symbolic
version of Table, creating a list of expressions by replacing index 'i'
with 'i1', 'i2', etc., index 'j' with 'j1', 'j2', etc., and so on. It is
useful for generating lists of tensor components or related expressions.
```

Sub-Kind가 성분 인덱스를 제외하고 두 개 이상일 때, 임의 표현의 main-인덱스를 sub-인덱스들로 대체한 Table을 생성한다. 첫 번째 인자는 IndexedObject를 포함한 표현이고, 두 번째 인자는 Table 생성을 위한 main-Kind와 sub-Kind의 리스트이다. 옵션으로 HeadQs가 있다. (적합한 텐서 표현이 아니어도 인덱스가 대체된 표현의 Table을 생성함을 **주의**하자.) Sub-Kind에 어떤 인덱스들을 포함시키냐는 것은 main-Kind와 (문법적으로도 또는 수학적으로도) 아무런 상관이 없고, 사용 시의 필요에 따라 결정한다.

Kaluza-Klein mode

```
In[*]:= SplitIndices[Cg[1A, 1B], {1A, 1α, 1β}]

Out[*]= {CgαB, CgβB}
```

Sub-Kind 리스트에 원래의 main-Kind 인덱스가 포함될 수 있다:

```
In[*]:= SplitIndices[Cg[1A, 1B], {1A, 1A, 1α, 1β}]

Out[*]= {CgAB, CgαB, CgβB}
```

In[*]:= SplitIndices[Cg[1A, 1B], {1A, 1 α , 1a}]

Out[*]=

$$\{Cg_{\alpha B}, Cg_{aB}\}$$

In[*]:= SplitIndices[Cg[1A, 1B], {1A, 1 α , 1a, 1ba}]

Out[*]=

$$\{Cg_{\alpha B}, Cg_{aB}, Cg_{\bar{a}B}\}$$

In[*]:= SplitIndices[Cg[1A, 1B], {1A, 1 α , 1a, 1ba, 1da}]

Out[*]=

$$\{Cg_{\alpha B}, Cg_{aB}, Cg_{\bar{a}B}, Cg_{\dot{a}B}\}$$

In[*]:= SplitIndices[Cg[1A, 1B], {1A, 1 α , 1a, 1ba, 1da, 1ha}]

Out[*]=

$$\{Cg_{\alpha B}, Cg_{aB}, Cg_{\bar{a}B}, Cg_{\dot{a}B}, Cg_{\hat{a}B}\}$$

In[*]:= SplitIndices[Cg[1A, 1B], {1A, 1 α , 1a}, {1B, 1 β , 1b}]

Out[*]=

$$\{\{Cg_{\alpha\beta}, Cg_{\alpha b}\}, \{Cg_{a\beta}, Cg_{ab}\}\}$$

In[*]:= SplitIndices[Cg[1A, 1B], {1A, 1 α , 1a, 1ha}, {1B, 1 β , 1b, 1hb}]

Out[*]=

$$\{\{Cg_{\alpha\beta}, Cg_{\alpha b}, Cg_{\alpha\hat{b}}\}, \{Cg_{a\beta}, Cg_{ab}, Cg_{a\hat{b}}\}, \{Cg_{\hat{a}\beta}, Cg_{\hat{a}b}, Cg_{\hat{a}\hat{b}}\}\}$$

In[*]:= SplitIndices[Cg[1A, 1B] $\times$ CR[uC, 1D], {1A, 1 α , 1a}, {1B, 1 β , 1b}, {uC, u γ , uc}]

Out[*]=

$$\{\{Cg_{\alpha\beta} CR_D^\gamma, Cg_{\alpha\beta} CR_D^c\}, \{Cg_{\alpha b} CR_D^\gamma, Cg_{\alpha b} CR_D^c\}\}, \\ \{\{Cg_{a\beta} CR_D^\gamma, Cg_{a\beta} CR_D^c\}, \{Cg_{ab} CR_D^\gamma, Cg_{ab} CR_D^c\}\}\}$$

$$In[*] := \text{SplitIndices}[Cg[1A, 1B] \times CR[uC, 1D], \{1A, 1\alpha, 1a, 1da\}, \\ \{1B, 1\beta, 1b, 1db\}, \{uC, u\gamma, uc, udc\}, \{1D, 1\delta, 1d, 1dd\}]$$
[illegible]

두 번째 인자의 인덱스가 첫 번째 인자의 표현에 없으면 출력 결과가 적절하지 않다.

```
In[•]:= SplitIndices[Cg[lA, lB], {lC, lα}]
```

$$\{C_{g_{AB}}\}$$
$$\text{In}[\bullet]:= \text{SplitIndices}[\text{Cg}[\text{1A}, \text{1B}], \{\text{1C}, \text{1}\alpha, \text{1a}\}]$$
$$\{Cg_{AB}, Cg_{AB}\}$$

텐서 표현으로 적합치 않음에도 인덱스를 변환시키므로 주의하자:

$$In[*] := \text{SplitIndices}[\text{Cg}[1A, 1B] + \text{Cg}[1C, uC], \{1C, 1\alpha, 1\beta\}]$$
$$\{Cg_{AB} + Cg_{\alpha}^C, Cg_{AB} + Cg_{\beta}^C\}$$

ScalarFunction:

$$In[*]:= \text{SplitIndices}[a \text{Log}[\text{CR}[lA, lB] \times \text{CR}[uB, uC]], \{lA, l\alpha, l\alpha\}]$$
$$\{a \text{ Log} [CR_{\alpha B} CR^{BC}], a \text{ Log} [CR_{aB} CR^{BC}]\}$$

(3+1)-decomposition

```
In[*]:= Tdefine[g[lμ, lν], "ba"]; Tdefine[R[lμ, lν], "ba"] (* Greek kind *)
```

```
In[*]:= DefKind[Zero, {"0"}]
```

```
In[*]:= SplitIndices[g[lμ, lν], {lμ, l0, li}, {lν, l0, lj}]
```

```
Out[*]=
```

```
{ {g00, g0j}, {gi0, gij} }
```

```
In[*]:= SplitIndices[g[lμ, lν] × R[uρ, lγ],  
  {lμ, l0, li}, {lν, l0, lj}, {uρ, u0, uk}, {lγ, l0, ll}]
```

```
Out[*]=
```

```
{ { { {g00 R00, g00 R01}, {g00 R0k, g00 R01k} }, { {g0j R00, g0j R01}, {g0j R0k, g0j R01k} },  
  { { {g00 R00, g00 R01}, {g00 R0k, g00 R01k} }, { {g0j R00, g0j R01}, {g0j R0k, g0j R01k} },  
  { { {g00 R00, g00 R01}, {g00 R0k, g00 R01k} }, { {g0j R00, g0j R01}, {g0j R0k, g0j R01k} } }
```

원래의 설정으로 복귀한다:

```
In[*]:= UndefKind[Zero]
```

```
In[*]:= Tdefine[#, "ba"] & /@ {g, R}; (* to DefaultKind *)
```

SumDum

```
In[*]:= SumDum::usage
```

```
Out[*]=
```

```
SumDum[expr, {i1, i2}, kind] performs a numeric summation  
  over all dummy index pairs of the specified 'kind' from i1 to i2.  
SumDum[expr, {i, i1, i2, ...}, ...] performs a symbolic summation by  
  replacing index 'i' with 'i1', 'i2', etc., and summing the results.  
SumDum[expr, {kind1, kind2, ...}] performs a Kaluza-Klein style decomposition,  
  summing over indices of 'kind1' by splitting them into indices of 'kind2', etc.
```

인자로 주어진 인덱스들을 이용하여 인덱스 쌍들을 다른 인덱스들로 각각 대체하고 그 결과를 합한다.

Numeric Summation

두 번째 인자로 주어진 인덱스 리스트들이 모두 숫자인 경우, 첫 번째 인자에 있는 표현의 인덱스 쌍들을 첫 번째 숫자부터 나중 숫자까지 1씩 증가시키면서 순차적으로 대체한 후에 합한다. 옵션으로는 HeadQs가 있다.

기본적으로는 DefaultKind의 인덱스 쌍만 고려한다:

In[*]:= **expr = a R[1a, ua] × CovD[1A, R[1μ, uA]] × CR[1B, uB]**
SumDum[expr, {1, 3}]

Out[*]=

$$a \nabla_A R_\mu^A CR_B^B R_a^a$$

Out[*]=

$$a \left(\nabla_A R_\mu^A CR_B^B R_3^3 + \nabla_A R_\mu^A CR_B^B R_2^2 + \nabla_A R_\mu^A CR_B^B R_1^1 \right)$$

IndexedObject 중에서 피연산자 표현만 고려하도록 옵션을 조정할 수 있다:

In[*]:= **SumDum[expr, {1, 3}, HeadQs → {IndexedOperandQ}]**

Out[*]=

$$a \nabla_A R_\mu^A \left(CR_B^B R_3^3 + CR_B^B R_2^2 + CR_B^B R_1^1 \right)$$

세 번째 인자로 인덱스 Kind를 입력하면 그 Kind에 속한 IndexedObject의 인덱스 쌍만 고려된다:

In[*]:= **SumDum[expr, {1, 3}, Capital]**

Out[*]=

$$a \left(\nabla_A R_\mu^A CR_3^3 R_a^a + \nabla_A R_\mu^A CR_2^2 R_a^a + \nabla_A R_\mu^A CR_1^1 R_a^a \right)$$

Capital Kind의 피연산자 표현만 고려하도록 옵션으로 지정할 수 있다:

In[*]:= **SumDum[expr, {1, 3}, Capital, HeadQs → {IndexedOperandQ}]**

Out[*]=

$$a \nabla_A R_\mu^A \left(CR_3^3 R_a^a + CR_2^2 R_a^a + CR_1^1 R_a^a \right)$$

ScalarFunction에 대한 HeadQs 옵션을 조정하여도, 즉 옵션으로 HeadQs → {ObjectQ}를 입력하여도, (현재까지의 구현 방법 때문에) SumDum 연산이 동작하지 않는다:

In[*]:= **SumDum[a Log[R[1A, 1b] × R[uA, ub]], {1, 2}]**
SumDum[a Log[R[1A, 1b] × R[uA, ub]], {1, 2}, HeadQs → {ObjectQ}]

Out[*]=

$$a \text{Log} \left[R_{Ab} R^{Ab} \right]$$

Out[*]=

$$a \text{Log} \left[R_{Ab} R^{Ab} \right]$$

잘못된 텐서 표현도 있는 그대로 바꾸므로 **주의**해야 한다.

In[*]:= **R[1a, ua] × g[1a, 1b]**
SumDum[%, {1, 2}]

Out[*]=

$$g_{ab} R_a^a$$

Out[*]=

$$g_{2b} R_2^2 + g_{1b} R_1^1$$

Component mode

공간의 차원 값이 정수일 때 인덱스 쌍들을 성분 인덱스로 대체하고 각각을 합한다. 두 번째 인자로 인덱스 계열이 입력되지 않는한 DefaultKind의 인덱스만 고려된다. 옵션으로는 HeadQs가 있다.

```
In[*]:= GetDimension[DefaultKind] = 2;
```

```
In[*]:= SumDum[g[1a, ua]]
```

```
Out[*]=
```

$$g_2^2 + g_1^1$$

IndexedObject 중에서 피 연산자 표현만 고려하도록 옵션을 지정할 수 있다:

```
In[*]:= expr = {CovD[1a, R[1b, ua]], g[1a, ua]}
SumDum[#, HeadQs → {IndexedOperandQ}] & /@ expr
```

```
Out[*]=
```

$$\{\nabla_a R_b^a, g_a^a\}$$

```
Out[*]=
```

$$\{\nabla_a R_b^a, g_2^2 + g_1^1\}$$

IndexedObject 중에서 연산자 표현만 고려하도록 옵션을 지정할 수 있다:

```
In[*]:= SumDum[#, HeadQs → {IndexedOperatorQ}] & /@ expr
```

```
Out[*]=
```

$$\{\nabla_2 R_b^2 + \nabla_1 R_b^1, g_a^a\}$$

두 번째 인자로 인덱스 Kind를 사용하면 그 Kind의 IndexedObject만 고려한다:

```
In[*]:= GetDimension[Capital] = 3;
```

```
In[*]:= SumDum[g[1a, ua] × CR[1A, uA]]
SumDum[g[1a, ua] × CR[1A, uA], Capital]
```

```
Out[*]=
```

$$CR_A^A g_2^2 + CR_A^A g_1^1$$

```
Out[*]=
```

$$CR_3^3 g_a^a + CR_2^2 g_a^a + CR_1^1 g_a^a$$

ScalarFunction에 대한 HeadQs 옵션을 조정하여도 SumDum 연산이 동작하지 않는다:

```
In[*]:= {Log[R[1a, 1b] × R[ua, ub]], Tscalar[R[1a, 1b] × R[ua, ub]]^2}
SumDum[#, HeadQs → {ObjectQ}] & /@ %
```

```
Out[*]=
```

$$\{\text{Log}[R_{ab} R^{ab}], (R_{ab} R^{ab})^2\}$$

```
Out[*]=
```

$$\{\text{Log}[R_{ab} R^{ab}], (R_{ab} R^{ab})^2\}$$

SumDum의 성분 모드는 공간 차원이 정해져야 한다:

In[*]:= **GetDimension[Latin] = .;**

In[*]:= **SumDum[g[1a, ua]]**

... **Msg:** Component mode of SumDum requires Dimension for Latin . Use SetDimension.

Out[*]=

g_a^a

In[*]:= **GetDimension[Capital] = .; (*default*)**

Symbolic Summation

두 번째 인자 리스트(lower 인덱스들)의 처음 요소에 해당하는 인덱스 쌍들을 다음부터 마지막까지에 해당하는 인덱스들로 각각 대치하고 그 결과를 합한다. 옵션으로는 HeadQs가 있다.

SumDum의 기호 모드에서는 IndexedObject의 **Kind가 고려되지 않는다**. 즉 IndexedObject의 정의에 사용된 인덱스의 **Kind**는 무시되고, 모든(All) 인덱스 쌍이 고려된다. 따라서 텐서 표현의 인덱스에 대한 타당성은 사용자의 몫이다.

In[*]:= **expr = a R[1a, ua] × CovD[1A, R[1μ, uA]] × CR[1B, uB]**
SumDum[%, {1a, 1i, 1j}]

Out[*]=

$a \nabla_A R_\mu^A CR_B^B R_a^a$

Out[*]=

$a \left(\nabla_A R_\mu^A CR_B^B R_i^i + \nabla_A R_\mu^A CR_B^B R_j^j \right)$

In[*]:= **SumDum[expr, {1a, -1, -2, 1i, 1j}]**

Out[*]=

$a \left(\nabla_A R_\mu^A CR_B^B R_2^2 + \nabla_A R_\mu^A CR_B^B R_1^1 + \nabla_A R_\mu^A CR_B^B R_i^i + \nabla_A R_\mu^A CR_B^B R_j^j \right)$

In[*]:= **SumDum[expr, {1a, -1, -2, 1i, 1j}, HeadQs → {IndexedOperandQ}]**

Out[*]=

$a \nabla_A R_\mu^A \left(CR_B^B R_2^2 + CR_B^B R_1^1 + CR_B^B R_i^i + CR_B^B R_j^j \right)$

In[*]:= **SumDum[CurvR[1i, 1j, 1k, 1l] × CurvR[ui, uj, uk, ul],
{1i, 1μ, 1a}, {1j, 1ν, 1b}, {1k, 1ρ, 1c}, {1l, 1σ, 1d}]**

Out[*]=

$R_{abcd} R^{abcd} + R_{abco} R^{abc\sigma} + R_{abod} R^{ab\sigma d} + R_{ab\sigma o} R^{ab\sigma\sigma} + R_{avcd} R^{avcd} +$
 $R_{avco} R^{avc\sigma} + R_{avod} R^{av\sigma d} + R_{av\sigma o} R^{av\sigma\sigma} + R_{\mu bcd} R^{\mu bcd} + R_{\mu bc\sigma} R^{\mu bc\sigma} +$
 $R_{\mu b\sigma d} R^{\mu b\sigma d} + R_{\mu b\sigma\sigma} R^{\mu b\sigma\sigma} + R_{\nu cd} R^{\nu cd} + R_{\nu c\sigma} R^{\nu c\sigma} + R_{\nu\sigma d} R^{\nu\sigma d} + R_{\nu\sigma\sigma} R^{\nu\sigma\sigma}$

SumDum의 기호 모드에서는 인덱스 Kind가 무시되므로, 비록 CR의 Kind가 Capital일지라도 다른 Kind의 인덱스 쌍으로 대치될 수 있다:

In[*]:= SumDum[CR[uA, lA], {lA, lμ, li}]

Out[*]= $CR^i_i + CR^\mu_\mu$

In[*]:= SumDum[R[lA, lB] × R[uA, uB], {lA, lμ, li}, {lB, lv, lj}]
(* NB: The kind of R is Latin *)

Out[*]= $R_{ij} R^{ij} + R_{i\nu} R^{i\nu} + R_{\mu j} R^{\mu j} + R_{\mu\nu} R^{\mu\nu}$

In[*]:= SumDum[R[la, lb] × R[ua, ub], {la, lμ, li}, {lb, lv, lj}]

Out[*]= $R_{ij} R^{ij} + R_{i\nu} R^{i\nu} + R_{\mu j} R^{\mu j} + R_{\mu\nu} R^{\mu\nu}$

주의: 대치되는 것들이 합당하지 않은 인덱스가 있을 때:

In[*]:= SumDum[R[ua, la] × R[ub, lb], {la, uμ, li}]

... Msg: invalid input in {la, uμ, li} . It should be a list of lower indices.

Out[*]= $R^a_a R^b_b$

인자로 주어진 인덱스가 operand에 없으면 경고 메시지가 발생된다.

In[*]:= SumDum[R[uA, lA], {la, lμ, li}, {lA, lv, lj}]

... Msg: no compatible index: la in R^A_A

Out[*]= $R^j_j + R^v_v$

ScalarFunction에 대한 HeadQs 옵션을 조정하여도 SumDum 연산이 동작하지 않는다:

In[*]:= SumDum[a Tscalar[Log[R[lA, lB] × R[uA, uB]]], {la, li, lj}]

... Msg: no compatible index: la in 1

Out[*]= $a \text{Log}[R_{ab} R^{ab}]$

In[*]:= SumDum[a Log[R[lA, lB] × R[uA, uB]], {la, li, lj}, HeadQs → {ObjectQ}]

... Msg: no compatible index: la in 1

Out[*]= $a \text{Log}[R_{ab} R^{ab}]$

Symbolic Summation in Kaluza-Klein Mode

Kaluza-Klein 모드는 두 개 이상의 sub-Kind가 필요하다. 그런데 원래의 텐서가 가질 수 있는 인덱스에 sub-Kind가 속하지 않으면 SyntaxCheck 오류이다. 이것을 방지하려면 Off[AutoFlag]하거나, 최소한 Off[SyntaxCheck]을 해야 한다.

Kaluza-Klein 방식에서 sub-Kind가 Component 인덱스를 제외하고 두 개 이상일 때, main-인덱스 쌍들을 sub-인덱스로 대체하고 각각을 합한다.

In[*]:=

On[AutoFlag]
On[SyntaxCheckFlag]

In[*]:=

SumDum[a CR[1A, uA], {1A, 1a, 1μ}]

... General: 1a is not a valid index.

Out[*]=

$a CR_{\mu}^{\mu} + CR_a^a$

In[*]:=

SumDum[a CR[1A, uB] × CR[1B, uA], {1A, 1a, 1μ}]

... General: 1a is not a valid index.

... General: ua is not a valid index.

Out[*]=

$a CR_B^{\mu} CR_{\mu}^B + CR_a^B CR_B^a$

In[*]:=

Off[SyntaxCheckFlag]

In[*]:=

a CR[1A, uA]
SumDum[%, {1A, 1a, 1μ}]

Out[*]=

$a CR_A^A$

Out[*]=

$a CR_a^a + a CR_{\alpha}^{\alpha}$

In[*]:=

a CR[1A, uB] × CR[1B, uA]
SumDum[%, {1A, 1a, 1μ}]

Out[*]=

$a CR_A^B CR_B^A$

Out[*]=

$a CR_a^A CR_A^a + a CR_A^{\alpha} CR_{\alpha}^A$

In[*]:=

a CR[1A, uA] × CR[1B, 1C]
SumDum[%, {1A, 1a, 1μ}]

Out[*]=

$a CR_A^A CR_{BC}$

Out[*]=

$a CR_a^a CR_{BC} + a CR_{BC} CR_{\alpha}^{\alpha}$

In[*]:= **a** CR[1A, 1B] × g[uA, uB]
SumDum[%, {1A, 1a, 1μ}, {1B, 1b, 1ν}]

Out[*]=

$$a \text{CR}_{AB} g^{AB}$$

Out[*]=

$$a \text{CR}_{ab} g^{ab} + a \text{CR}_{a\alpha} g^{a\alpha} + a \text{CR}_{\alpha a} g^{\alpha a} + a \text{CR}_{\alpha\beta} g^{\alpha\beta}$$

In[*]:= **a** CovD[1A, CR[1a, uA]] × g[uB, 1B]
SumDum[%, {1A, 1a, 1μ}, {1B, 1b, 1ν}]

Out[*]=

$$a \nabla_A \text{CR}_a^A g_B^B$$

Out[*]=

$$a \nabla_a \text{CR}_a^a g_b^b + a \nabla_\alpha \text{CR}_a^\alpha g_b^b + a \nabla_a \text{CR}_a^a g_\alpha^\alpha + a \nabla_\alpha \text{CR}_a^\alpha g_\beta^\beta$$

In[*]:= **a** CovD[1A, CR[1a, uA]] × g[uB, 1B]
SumDum[%, {1A, 1a, 1μ}, {1B, 1b, 1ν}, HeadQs → {IndexedOperatorQ}]

Out[*]=

$$a \nabla_A \text{CR}_a^A g_B^B$$

Msg: no compatible index: 1B in $\nabla_A \text{CR}_a^A$

Out[*]=

$$a \nabla_a \text{CR}_a^a g_A^A + a \nabla_\alpha \text{CR}_a^\alpha g_A^A$$

ScalarFunction:

In[*]:= **a** Log[CR[1A, 1B] × R[uA, uB]]
SumDum[%, {1A, 1a, 1μ}, {1B, 1b, 1ν}]

Out[*]=

$$a \text{Log}[\text{CR}_{AB} R^{AB}]$$

Msg: no compatible index: 1A in 1

Msg: no compatible index: 1B in 1

Out[*]=

$$a \text{Log}[\text{CR}_{AB} R^{AB}]$$

Option HeadQs→{ObjectQ} is suppressed to avoid mathematically illegal operations:

In[*]:= **a** Log[CR[1A, 1B] × R[uA, uB]]
SumDum[% , {1A, 1a, 1μ}, {1B, 1b, 1ν}, HeadQs → {ObjectQ}]

Out[*]=

a Log[CR_{AB} R^{AB}]

...

Msg: no compatible index: 1A in 1

Msg: no compatible index: 1B in 1

Out[*]=

a Log[CR_{AB} R^{AB}]

In[*]:= **Off**[SyntaxCheckFlag]
Off[AutoFlag]

Another usage of Kaluza-Klein SumDum

Kaluza-Klein 방식에서 sub-Kind가 Component 인덱스를 제외하고 두 개일 때, main-인덱스 쌍들을 sub-인덱스로 대체하고 각각을 합한다. 이 경우에는 main-인덱스만 고려되므로 IndexQs에 대한 옵션이 없고, 두 번째 인자가 Kind 리스트이다.

In[*]:= **On**[AutoFlag]

In[*]:= **SumDum**[a R[1A, uA] × R[1a, ua], {Capital, Latin, Greek}] (* R is Latin *)

Out[*]=

a R_a^a R_A^A

In[*]:= **SumDum**[a CR[1A, uA] × R[1a, ua], {Capital, Latin, Greek}]

Out[*]=

a CR_α^α R_a^a + a CR_a^a R_b^b

In[*]:= **SumDum**[a CR[1A, uA] × R[1a, ua], {Capital, Latin, Greek}] // SyntaxCheck

...

General: !Greek\$10553 is not a valid index.

Out[*]=

a CR_a^a R_b^b + R_a^a CR_{Greek\$10553}^{Greek\$10553}

In[*]:= **Off**[MarkErrorFlag]

In[*]:= **SumDum**[a CR[1A, uA] × R[1a, ua], {Capital, Latin, Greek}]

Out[*]=

a CR_α^α R_a^a + a CR_a^a R_b^b

In[*]:= SumDum[a CR[1A, uA] × R[1a, ua], {Capital, Latin, Greek}] // SyntaxCheck

... General: !Greek\$10606 is not a valid index.

Out[*]=

$$CR_{\alpha}^{\alpha} R_a^a + a CR_a^a R_b^b$$

In[*]:= On[MarkErrorFlag]

In[*]:= Off[AutoFlag]

In[*]:= SumDum[a CR[1A, uA] × R[1a, ua], {Capital, Latin, Greek}] // SyntaxCheck

... General: !Greek\$10641 is not a valid index.

Out[*]=

$$a CR_{\text{Latin}\$10639}^{\text{Latin}\$10639} R_a^a + R_a^a CR_{\text{Greek}\$10641}^{\text{Greek}\$10641}$$

In[*]:= On[AutoFlag]

In[*]:= SumDum[a CR[1A, uA] × Cg[1B, 1C], {Capital, Latin, Greek}]

Out[*]=

$$a Cg_{BC} CR_a^a + a Cg_{BC} CR_{\alpha}^{\alpha}$$

In[*]:= SumDum[a CR[1A, 1B] × Cg[uA, uB], {Capital, Latin, Greek}]

Out[*]=

$$a Cg^{ab} CR_{ab} + a Cg^{a\alpha} CR_{a\alpha} + a Cg^{\alpha a} CR_{\alpha a} + a Cg^{\alpha\beta} CR_{\alpha\beta}$$

In[*]:= SumDum[a CCovD[1A, R[1a, uA]] × Cg[uB, 1B], {Capital, Latin, Greek}]

Out[*]=

$$a D_A R_a^A Cg_b^b + a D_A R_a^A Cg_{\alpha}^{\alpha}$$

In[*]:= SumDum[a CCovD[1A, CR[1a, uA]] × Cg[uB, 1B], {Capital, Latin, Greek}]

Out[*]=

$$a D_{\alpha} CR_a^{\alpha} Cg_b^b + a D_b CR_a^b Cg_c^c + a D_b CR_a^b Cg_{\alpha}^{\alpha} + a D_{\alpha} CR_a^{\alpha} Cg_{\beta}^{\beta}$$

In[*]:= SumDum[a CCovD[1A, CR[1a, uA]] × Cg[uB, 1B],
{Capital, Latin, Greek}, HeadQs → {IndexedOperatorQ}]

Out[*]=

$$a D_b CR_a^b Cg_A^A + a D_{\alpha} CR_a^{\alpha} Cg_A^A$$

Option HeadQs→{ObjectQ} is suppressed to fix mathematically illegal operations:

In[*]:= SumDum[a Log[CR[1A, 1B] × CR[uA, uB]], {Capital, Latin, Greek}]

Out[*]=

$$a \text{Log}[CR_{AB} CR^{AB}]$$

```
In[*]:= SumDum[a Log[CR[1A, 1B] × CR[uA, uB]], {Capital, Latin, Greek}, HeadQs → {ObjectQ}]
Out[*]=
a Log[CRAB CRAB]
```

RuleUnique

```
In[*]:= Tdefine /@ {f[], g[], h[]}; Tdefine[#, 1] & /@ {a, b, c};
```

Ref: MathTensor Book.

RuleUnique

```
In[*]:= RuleUnique::usage
Out[*]=
RuleUnique[lhs, rhs, cond] creates a delayed rule (lhs :=> rhs ;
cond) that automatically handles dummy indices on the right-hand side.
```

더미 인덱스를 고려한 Rule을 얻기 위한 함수이다. 첫 번째와 두 번째 인자는 각각 Rule의 왼쪽 표현과 오른쪽 표현이고, 옵션으로 세 번째 인자는 Rule의 조건이다. 돌려주는 Rule의 형태는 lhs :=> DumFresh[rhs] /; cond 형태이다.

```
In[*]:= aRule = RuleUnique[b[1a_, ub_], a[1a] × a[ub] × a[1c] × a[uc], PairIndexQ[{1a, ub}]]
Out[*]=
b1a_ ub_ :=> DumFresh[aa aLatin$10831 ab aLatin$10831] /; PairIndexQ[{1a, ub}]
```

```
In[*]:= expr1 = b[1a, ua] × b[1b, ub]
expr1 /. aRule
```

```
Out[*]=
baa bbb
```

```
Out[*]=
aa ab ac ad aa ab ac ad
```

```
In[*]:= bRule = RuleUnique[c[1a_, 1b_], 2 b[1a, 1b]]
```

```
Out[*]=
c1a_ 1b_ :=> DumFresh[2 bab]
```

```
In[*]:= rules = {aRule, bRule};
c[la, ua] × c[lb, ub] /. rules
% /. rules
```

```
Out[*]=
4 baa bbb
```

```
Out[*]=
4 aa ab ac ad aa ab ac ad
```

```
In[*]:= c[la, ua] × c[lb, ub] //. rules
```

```
Out[*]=
4 aa ab ac ad aa ab ac ad
```

Rule의 오른쪽 표현에 IndexedObject가 아닌 ScalarFunction들이 있는 경우 출력 모양이 (현재까지는) 좋지 않을 수 있다:

```
In[*]:= aRule = RuleUnique[b[la_, ub_],
  a[la] × a[ub] × Tscalar[a[lc] × a[uc]], PairIndexQ[{la, ub}]]
b[la, ua] × b[lb, ub] /. aRule
```

```
Out[*]=
bla_ua_ :=> DumFresh[aa ab (ac ac)] /; PairIndexQ[{la, ub}]
```

```
Out[*]=
aa ab aa ab (ac ac)2
```

```
In[*]:= cRule = RuleUnique[f[], b[la, lb] × b[ua, ub]]
f[] × b[la, ua] /. {aRule, cRule}
```

```
Out[*]=
f :=> DumFresh[bLatin$10936b bLatin$10936b]
```

```
Out[*]=
aa aa bbc bbc (ac ac)
```

```
In[*]:= {Log[f[]], Tscalar[f[] × g[]]^2} /. cRule
```

```
Out[*]=
{Log[bLatin$10955b bLatin$10955b], (bLatin$10958b bLatin$10958b g)2}
```

SyntaxCheck

```
In[*]:= SyntaxCheck::usage
```

```
Out[*]=
SyntaxCheck[expr] checks 'expr' for syntactical errors like free indices mismatch.
```

입력된 표현이 적합한 텐서 표현인가를 알아보는 함수이다. 옵션으로 HeadQs가 있다.

```
In[*]:= Tdefine[scrR[], PrintAs -> "R"];
Tdefine[F[1a, 1b], "-ba"];
Tdefine[R[1a, 1b], "ba"];
Tdefine[ε, "*-"];
SP`defineOperator[CovD, "∇", CD];
SP`defineOperator[LieD, "ℒ", LD]
```

Free Indices

텐서 합의 free 인덱스들은 동일해야 한다.

```
In[*]:= a scrR[] × F[1b, 1c] + b R[1b, 1a]
SyntaxCheck[%]
```

Out[*]=

$$b R_{ba} + a F_{bc} R$$

Msg: incompatible free indices: {1b, 1a} and {1b, 1c}

Out[*]=

$$b R_{ba} + a F_{bc} R$$

Duplicated Indices

텐서 곱 표현에서 동일한 인덱스는 허용되지 않는다.

```
In[*]:= R[1a, 1a]
SyntaxCheck[%]
```

Out[*]=

$$R_{aa}$$

Msg: duplicated indices: {1a, 1a}

Out[*]=

$$R_{aa}$$

```
In[*]:= R[1a, 1b] × F[1b, 1a]
SyntaxCheck[%]
```

Out[*]=

$$F_{ba} R_{ab}$$

Msg: duplicated indices: {1b, 1a, 1a, 1b}

Out[*]=

$$F_{ba} R_{ab}$$

```
In[*]:= CovD[1a, R[1b, 1c] × R[1c, 1b]]
SyntaxCheck[%]

Out[*]=
```

$$\nabla_a R_{bc} R_{cb}$$

Msg: duplicated indices: {1b, 1c, 1c, 1b}

```
Out[*]=
```

$$\nabla_a R_{bc} R_{cb}$$

IndexedObject 중에서 연산자 표현만 고려하도록 옵션을 조정하였다면 연산자 표현들만의 인덱스로 인덱스 중복을 확인한다. 따라서 아래 표현의 오류를 찾지 못한다:

```
In[*]:= CovD[1a, R[1b, uc] × R[1c, 1b]]
SyntaxCheck[%, HeadQs → {IndexedOperandQ}]

Out[*]=
```

$$\nabla_a R_b^c R_{cb}$$

```
Out[*]=
```

$$\nabla_a R_b^c R_{cb}$$

Number of Indices

텐서 인덱스의 수는 그 텐서의 rank와 같아야 한다.

```
In[*]:= R[1a]
SyntaxCheck[%]

Out[*]=
```

$$R_a$$

Msg: invalid number of indices for R: {1a}

```
Out[*]=
```

$$R_a$$

Epsilon 텐서는 DefaultKind의 차원이 정해져 있지 않을 때 임의의 (1 이상의) rank를 갖는 텐서이다:

```
In[*]:= SyntaxCheck /@ {ϵ[], ϵ[1a], ϵ[1a, 1b], ϵ[1a, 1b, 1c]}

Out[*]=
```

$$\{\epsilon, \epsilon_a, \epsilon_{ab}, \epsilon_{abc}\}$$

Msg: invalid number of indices for ϵ

Kind

인덱스 계열이 고려된다. 기본적으로는 DefaultKind 계열이다.

In[*]:=	DefaultKind
Out[*]=	Latin
In[*]:=	R[1a, 1b] SyntaxCheck[%]
Out[*]=	R _{aB}
	... General: 1B is not a valid index.
Out[*]=	R _{aB}
In[*]:=	CovD[1A, R[1a, 1c]] × R[1c, 1d] SyntaxCheck[%]
Out[*]=	∇ _A R _{ac} R _{cd}
	... General: 1A is not a valid index.
Out[*]=	R _{cd} ∇ _A R _{ac}

Dnup

DefaultKind 계열에 계량 텐서가 없도록 설정한다:

In[*]:=	Off[MetricgFlag]
	MetricSpaceQ를 만족하지 않는 인덱스 계열에서 인덱스는 IndexedObject를 정의할 때 결정한 위/아래 첨자의 위치를 따라야 한다.
In[*]:=	{R[1a, 1b], R[1a, ub]} // SyntaxCheck
	... Msg: R has wrong-dn/up indices: {1a, ub}
Out[*]=	{R _{ab} , R _a ^b }

CD type 연산자의 첫 번째 인자는 아래 첨자 인덱스이다:

In[*]:=	{CovD[1a, R[1b, 1c]], CovD[ua, R[1b, 1c]]} // SyntaxCheck
	... Msg: The index of CovD is not down: ua
Out[*]=	{∇ _a R _{bc} , ∇ ^a R _{bc} }

DefaultKind 계열에 계량 텐서를 설정한다:

In[*]:=	On[MetricgFlag]
---------	-----------------

계량 텐서가 있는 경우 위/아래 첨자를 자유롭게 변경시킬 수 있다. 따라서 비록 텐서를 정의할 때 인덱스의 기본 위치를 아래 첨자로 설정했을지라도 인덱스의 위 첨자 사용이 오류가 아니다:

```
In[*]:= {R[1a, ub], CovD[ua, R[1b, 1c]]} // SyntaxCheck
Out[*]= {Rab, ∇aRbc}
```

Operator

CD type 연산자의 첫 번째 인자는 그 연산자의 계열에 속한 인덱스이어야 한다:

```
In[*]:= CovD[1A, R[1b, 1c]] // SyntaxCheck
Out[*]= ... General: 1A is not a valid index.
```

∇_AR_{bc}

LD type 연산자의 첫 번째 인자는 기호이다:

```
In[*]:= LieD[V[1a], R[1a, 1b]]
SyntaxCheck[%]
```

ℒ_{V[1a]}R_{ab}

... Msg: non-symbolic argument V[1a] for an operator LieD

ℒ_{V[1a]}R_{ab}

ScalarFunction

ScalarFunction에 있는 IndexObject의 인덱스들이 적합한지 확인한다:

```
In[*]:= Log[R[1a, 1a]]
SyntaxCheck[%]
```

Log[R_{aa}]

... Msg: duplicated indices: {1a, 1a}

Log[R_{aa}]

In[*]:=	R[1a, 1b]² SyntaxCheck[%]
Out[*]=	R_{ab}^2
	 Msg: non-scalar expression R_{ab}
Out[*]=	R_{ab}^2
In[*]:=	Log[R[1a, 1b] × R[1d, 1c]] SyntaxCheck[%]
Out[*]=	$\text{Log}[R_{ab} R_{dc}]$
	 Msg: non-scalar expression $R_{ab} R_{dc}$
Out[*]=	$\text{Log}[R_{ab} R_{dc}]$

On/Off

In[*]:=	Tdefine[F[1a, 1b], "-ba"]; Tdefine[R[1a, 1b], "ba"]; SP`defineOperator[CovD, "∇", CD];
---------	---

AutoFlag

In[*]:=	AutoFlag::usage
Out[*]=	A flag that controls whether automatic post-processing, such as syntax checking and dummy index resetting, is performed on each output.
	출력 표현을 자동적으로 조정하려면 On 시키고, 그렇지 않으면 Off 시킨다. 이것을 On 시키면 SyntaxCheckFlag이 On인 경우의 문법 검사, ResetDummiesFlag이 On인 경우 ResetDummies 명령의 자동 실행이 이루어진다. 따라서 AutoFlag은 비록 연산 효율이 떨어지지만 On 시키는 것이 좋다. Default는 On이다.
In[*]:=	On[AutoFlag]
In[*]:=	{F[1b, 1a], F[1a, ua]} // Dum
Out[*]=	$\{F_{ba}, F_a^a\}$
In[*]:=	Off[AutoFlag]

```
In[*]:= {F[lb, la], F[la, ua]} // Dum
Out[*]=
```

```
{Fba, FLatin$17891Latin$17891}
```

CoordinateBasisFlag

```
In[*]:= CoordinateBasisFlag::usage
Out[*]=
```

```
A flag that controls whether the basis for a specific kind is a coordinate
basis. Use On[CoordinateBasisFlag] or On[CoordinateBasisFlag[kind]].
```

On인 경우 DefaultKind의 기준 벡터가 좌표 기준이고, Off인 경우 비좌표 기준이다. 기본적으로는 On이다.

```
In[*]:= Off[CoordinateBasisFlag]
CoordinateBasisQ[]
```

```
Out[*]=
False
```

비좌표 기준에서 Connection 계수는 무대칭이고, 연속한 BD 연산의 교환이 허용되지 않는다:

```
In[*]:= GetSymmetry[GammaCD]
BD[lb, la, F[uc, ud]] // TindexSort
```

```
Out[*]=
GenSet[]
```

```
Out[*]=
 $\hat{\partial}_b \hat{\partial}_a F^{cd}$ 
```

좌표 기준으로 바꾸려면 On 시킨다:

```
In[*]:= On[CoordinateBasisFlag]
CoordinateBasisQ[DefaultKind]
```

```
Out[*]=
True
```

좌표 기준에서 CD 연산자가 TorsionFree인 경우 커넥션 계수는 첫 번째와 두 번째 인덱스에 대한 교환 대칭을 갖는다:

```
In[*]:= TorsionFreeQ[CD]
GetSymmetry[GammaCD]
```

```
Out[*]=
True
```

```
Out[*]=
GenSet[{Cycles[{1, 2}], 1}]
```

좌표 기준에서 연속한 BD 연산을 교환할 수 있다:

In[*]:= **BD**[**lb**, **la**, **F**[**uc**, **ud**]] // **TindexSort**

Out[*]=

$$\partial_a \partial_b F^{cd}$$

비좌표 기준에서는 기준 벡터가 교환되지 않으므로 구조 상수(structure constants)가 0이 아니다:

In[*]:= **Off**[**CoordinateBasisFlag**]

Structuref[**la**, **lb**, **uc**]

Out[*]=

$$f_{ab}^c$$

그러나 좌표 기준에서 기준 벡터는 교환되므로 구조 상수가 필요 없다:

In[*]:= **On**[**CoordinateBasisFlag**]

Structuref[**la**, **lb**, **uc**]

Out[*]=

Structuref[**la**, **lb**, **uc**]

DefaultKind가 아닌 다른 Kind에 설정하려면 옵션으로 그 Kind를 입력한다:

In[*]:= **Off**[**CoordinateBasisFlag**[**Greek**]]

StructurefGreek[**lμ**, **lv**, **uρ**]

Out[*]=

$$f_{\mu\nu}^{\rho}$$

In[*]:= **On**[**CoordinateBasisFlag**[**Greek**]]

StructurefGreek[**lμ**, **lv**, **uρ**]

Out[*]=

StructurefGreek[**lμ**, **lv**, **uρ**]

MarkErrorFlag

텐서 표현에 오류가 있음을 발견했을 때 자동적으로 **ErrorT** 기호로 그 텐서를 둘러 싸려면 **On** 시키고, 그렇지 않으면 **Off** 시킨다. 이것을 **On** 시켜도 **AutoFlag**이 **Off**이면 아무런 효과가 없으므로 **AutoFlag**도 **On** 시켜야 한다. Default는 **On**이다.

In[*]:= **MarkErrorFlag::usage**

Out[*]=

A flag that controls whether expressions with errors are marked in red. When On, expressions wrapped in **ErrorT** are displayed in red.

In[*]:= **On**[**AutoFlag**]

In[*]:= **Off**[**MarkErrorFlag**]

```
In[*]:= {F[1b, 1b], F[1a, 1b]} // SyntaxCheck
```

```
Msg: duplicated indices: {1b, 1b}
```

```
Out[*]=
```

```
{Fbb, Fab}
```

```
In[*]:= ErrorT::usage
```

```
Out[*]=
```

```
ErrorT[expr] is a wrapper for expressions  
with errors, causing expr to be displayed in red.
```

```
In[*]:= On[MarkErrorFlag]
```

```
In[*]:= {F[1b, 1b], F[1a, 1b]} // SyntaxCheck
```

```
Msg: duplicated indices: {1b, 1b}
```

```
Out[*]=
```

```
{Fbb, Fab}
```

ResetDummiesFlag

출력 표현에 ResetDummies 명령을 실행시켜 더미 인덱스와 위/아래 첨자 쌍을 이루는 인덱스들을 자동적으로 재조정하려면 On 시키고, 그렇지 않으면 Off 시킨다. 이것을 On 시켜도 AutoFlag이 Off이면 출력에 대한 자동적인 재조정에 아무런 효과가 없으므로 AutoFlag도 On 시켜야 한다. Default는 On이다.

```
In[*]:= On[AutoFlag]
```

```
On[ResetDummiesFlag]
```

```
In[*]:= Dum[R[1a, 1b] × R[ua, ub]] // DumFresh
```

```
Out[*]=
```

```
Rab Rab
```

```
In[*]:= Off[AutoFlag]
```

```
Off[ResetDummiesFlag]
```

```
In[*]:= Dum[R[1a, 1b] × R[ua, ub]] // DumFresh
```

```
Out[*]=
```

```
RLatin$18033Latin$18035 RLatin$18033Latin$18035
```

SyntaxCheckFlag

텐서 표현이 문법적으로 적합한가를 자동적으로 검사하려면 On 시키고, 그렇지 않으면 Off 시킨다. 이것을 On 시켜도 AutoFlag이 Off이면 아무런 효과가 없으므로 AutoFlag도 On 시켜야 한다. Default는 Off이다.

In[*]:= Off[SyntaxCheckFlag]

In[*]:= CovD[1a, R[1b, uc] × R[1c, 1b]]

Out[*]=

$$\nabla_a R_b^c R_{cb}$$

In[*]:= R[1b, uc] × R[1c, 1b]

Out[*]=

$$R_b^c R_{cb}$$

In[*]:= On[AutoFlag]

On[SyntaxCheckFlag]

In[*]:= R[1b, 1a] × R[1c, 1b]

Msg: duplicated indices: {1b, 1a, 1c, 1b}

Out[*]=

$$R_{ba} R_{cb}$$

In[*]:= CovD[1a, R[1b, 1a] × R[1c, 1b]]

Msg: duplicated indices: {1b, 1a, 1c, 1b}

Out[*]=

$$\nabla_a R_{ba} R_{cb}$$